

## Python 入門

|                                          |    |
|------------------------------------------|----|
| 1 Python 周遊.....                         | 4  |
| 1-1 Python を学ぶ .....                     | 9  |
| 1-2 オブジェクトとは.....                        | 10 |
| 1-3 メソッド .....                           | 16 |
| 1-4 組み込み型.....                           | 20 |
| 1-4-1 数値型 .....                          | 22 |
| 1-4-2 ブーリアン型 .....                       | 26 |
| 1-4-3 テキストシーケンス（文字列）型 .....              | 31 |
| 1-4-4 そのほか特殊な組み込み型のオブジェクト .....          | 31 |
| 1-5 コンテナオブジェクトと繰り返し .....                | 35 |
| 1-5-1 シーケンス .....                        | 36 |
| ・ リスト .....                              | 36 |
| ・ タプル .....                              | 36 |
| ・ 文字列 .....                              | 37 |
| 1-5-2 集合(set)オブジェクト .....                | 38 |
| 1-5-3 辞書(dict)オブジェクト .....               | 38 |
| 1-5-4 繰り返し .....                         | 39 |
| ① for 文による繰り返し .....                     | 39 |
| ② 内包表記.....                              | 41 |
| ③ while 文 .....                          | 43 |
| 1-5-5 アンパック代入.....                       | 44 |
| 2 CUI アプリ開発 .....                        | 45 |
| 2-1 モジュール.....                           | 46 |
| 2-1-1 パッケージと__init__.py.....             | 50 |
| 2-1-2 Import Path の追加.....               | 51 |
| 2-2 オセロの駒 列挙型 .....                      | 52 |
| 2-2-1 ドキュメンテーションコメント .....               | 54 |
| 2-2-2 Top-level code .....               | 54 |
| 2-2-3 超簡易版の単体テスト .....                   | 54 |
| 2-3 オセロのボード テスト駆動開発 .....                | 57 |
| 2-3-1 単体テストフレームワーク [unittest] .....      | 58 |
| <b><i>Test Case 1: ボードの初期化</i></b> ..... | 58 |
| ① テストケースを作成する .....                      | 58 |

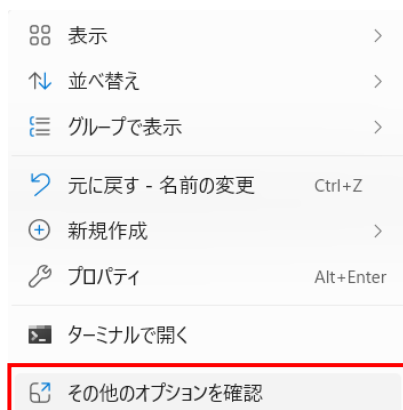
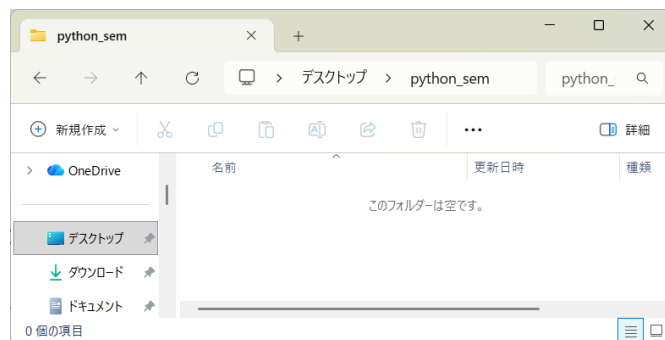
|                                             |           |
|---------------------------------------------|-----------|
| ② モジュールを作成する .....                          | 62        |
| <b>Test Case 2: 駒の数を数える .....</b>           | <b>68</b> |
| ① テストケースを作成する .....                         | 68        |
| ② モジュールを作成する .....                          | 69        |
| Test Case 3: 駒をおける箇所を判定する .....             | 70        |
| ① テストケースを作成する .....                         | 70        |
| ② モジュールを作成する .....                          | 71        |
| ③ リファクタリングする .....                          | 72        |
| Test Case 4: 駒を反転させる .....                  | 75        |
| ① テストケースを作成する .....                         | 75        |
| ② モジュールを作成する .....                          | 75        |
| Test Case 5: 駒をおける場所をすべてリストアップする .....      | 76        |
| ① テストケースを作成する .....                         | 76        |
| ② モジュールを作成する .....                          | 76        |
| 2-4 ゲームの進行を行うクラス 振舞駆動開発 .....               | 77        |
| 2-4-1 BDD の準備.....                          | 78        |
| <b>Feature 1: ゲームの初期化 関数.....</b>           | <b>80</b> |
| ① ふるまいを作成する.....                            | 80        |
| ② テストを作成する .....                            | 81        |
| ③ モジュールを作成する .....                          | 87        |
| <b>Feature 2: ターン進行.....</b>                | <b>89</b> |
| ① ふるまいを作成する.....                            | 89        |
| ② テストを作成する .....                            | 90        |
| ③ モジュールを作成する .....                          | 92        |
| <b>Feature 3: ゲームの終了と勝利判定.....</b>          | <b>93</b> |
| ① ふるまいを作成する.....                            | 93        |
| ② テストを作成する .....                            | 94        |
| ③ モジュールを作成する .....                          | 96        |
| 2-4-2 型ヒント .....                            | 97        |
| 2-5 オセロのプレイヤー 継承・多態性・カプセル化.....             | 100       |
| 2-5-1 プレイヤークラス 継承.....                      | 100       |
| 2-5-2 オーバーライドと多態性.....                      | 104       |
| 2-5-3 Player クラス作成 抽象クラス .....              | 108       |
| 2-5-5 strategy 関連クラスの作成 Strategy パターン ..... | 114       |
| 2-5-6 自作デコレータ.....                          | 117       |
| 3 付録.....                                   | 119       |

|       |                    |     |
|-------|--------------------|-----|
| 付録 A. | PlantUML.....      | 119 |
| 付録 B. | デコレータとその周辺知識 ..... | 122 |
| *     | ラムダ式 .....         | 122 |
| *     | 高階関数 .....         | 123 |
| *     | クロージャ .....        | 124 |
| *     | デコレータ .....        | 125 |

## 1 Python 周遊

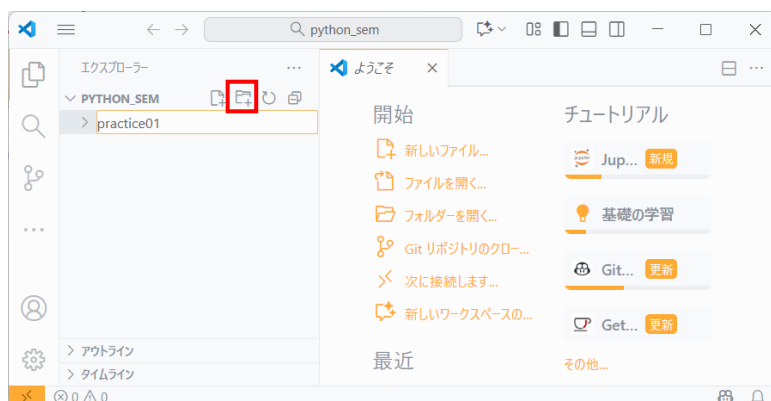
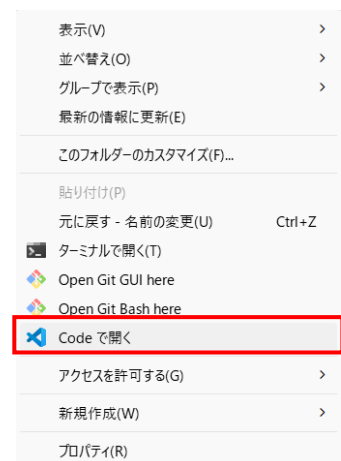
Python では操作の対象となるデータを「オブジェクト (Object)」と呼びます。また、オブジェクトを操作するための名札のような役割をするのが「変数」です。さっそく Visual Studio Code の Jupiter Notebook で Python 言語全体をサラッと触ってみましょう。

まずは練習用フォルダとしてデスクトップに「python\_sem」フォルダを作成します。



フォルダ内で右クリックして  
【その他のオプションを確認】  
をクリックします (左図)。

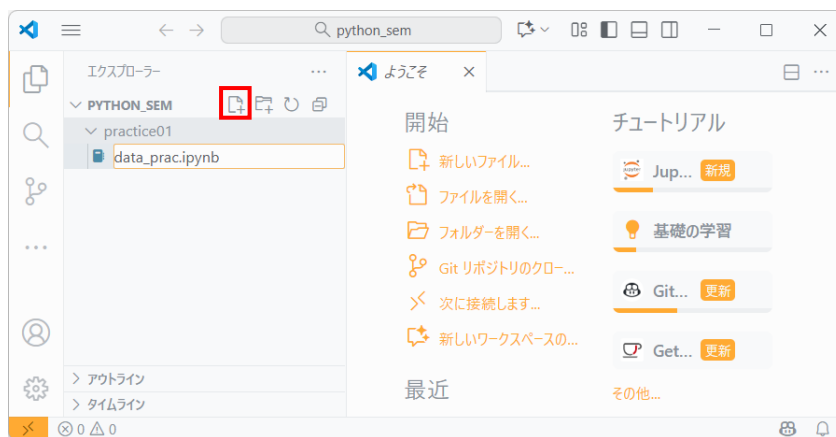
次に【Code で開く】を選択して  
Visual Studio Code を立ち上げ  
ます (右図)。



立ち上がった VSCode で  
【新しいフォルダ】をクリッ  
クします。

フォルダ名に「practice01」と  
入力して Enter キーを押しま  
す。



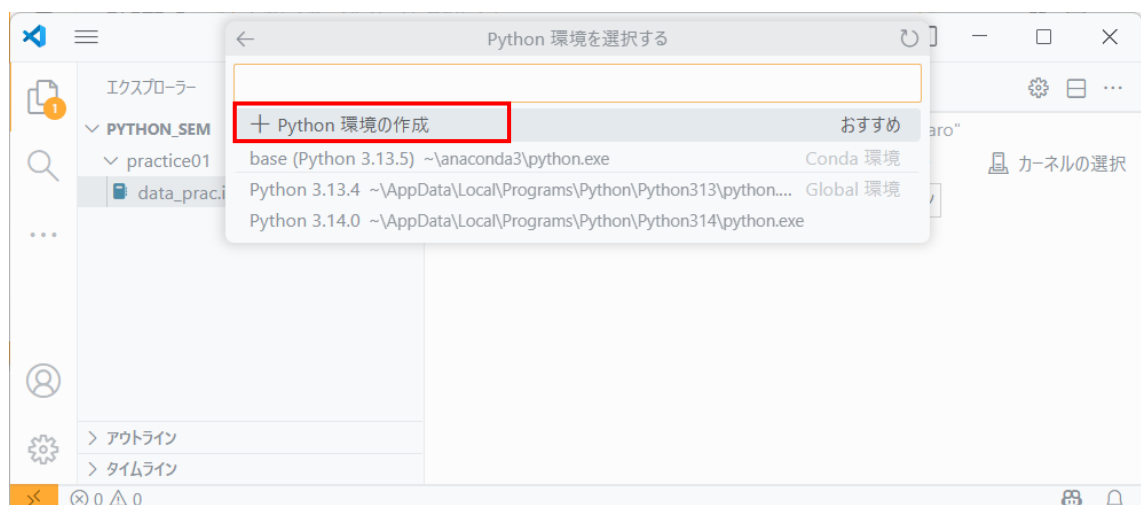


practice01 フォルダ  
を選択した状態で、  
【新しいファイル】を  
選択します。  
「data\_prac.ipynb」と  
入力して Enter キー  
を押します。

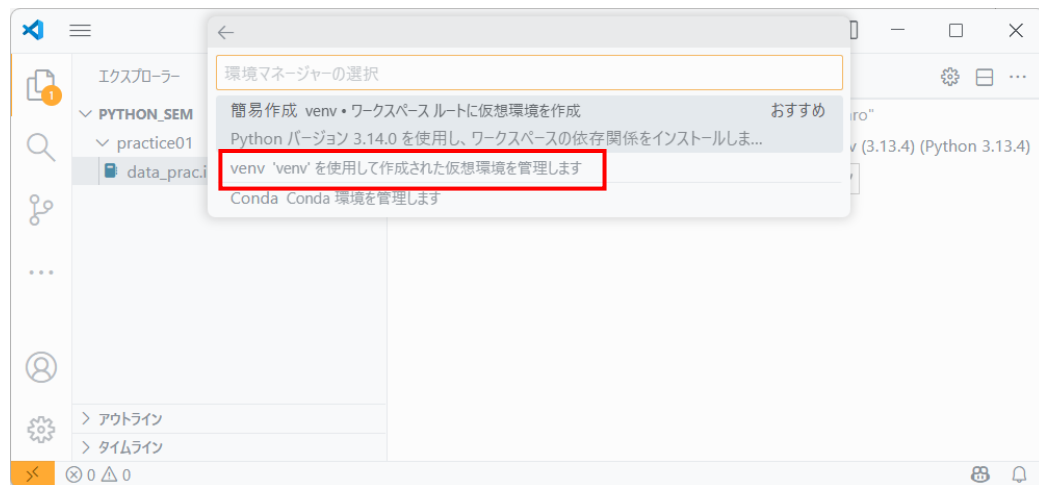
エディタの【カーネルの選択】をクリックして、【Python 環境…】をクリックします。



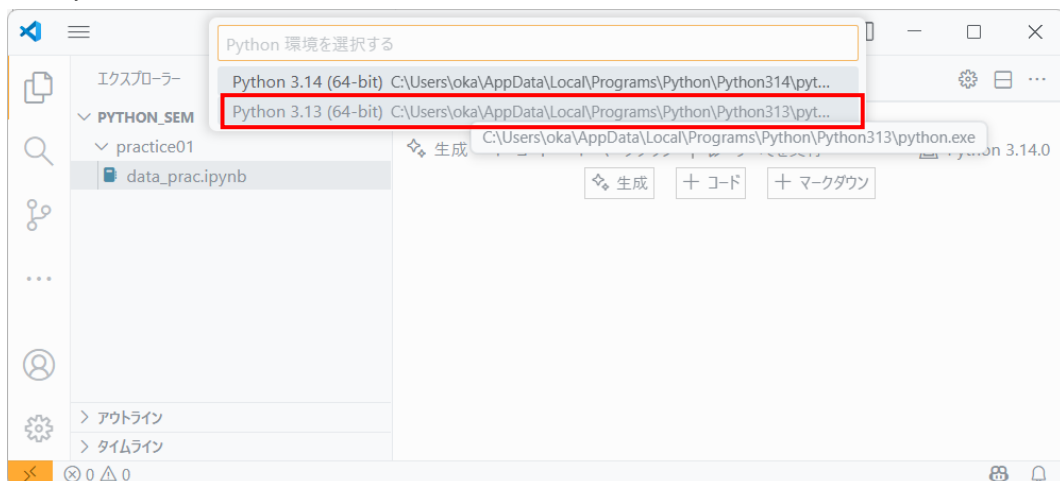
【+ Python 環境の作成】をクリックします。



【venv 'venv' を使用して作成された仮想環境を管理します】をクリックします。

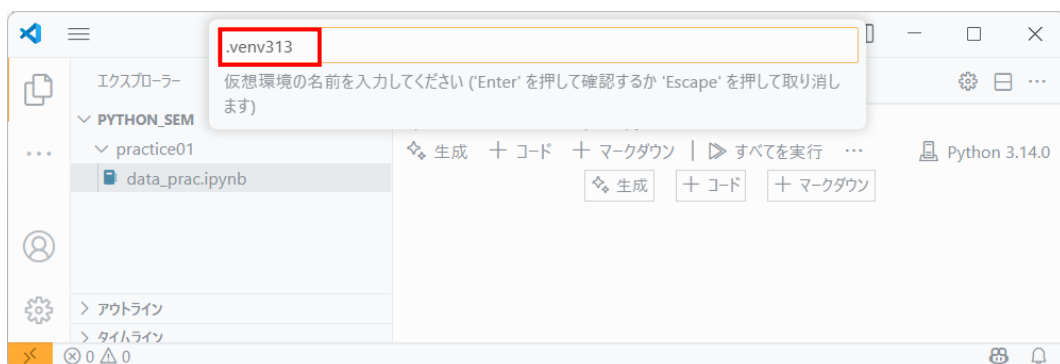


【Python 3.13 (64-bit) C:…】をクリックします。

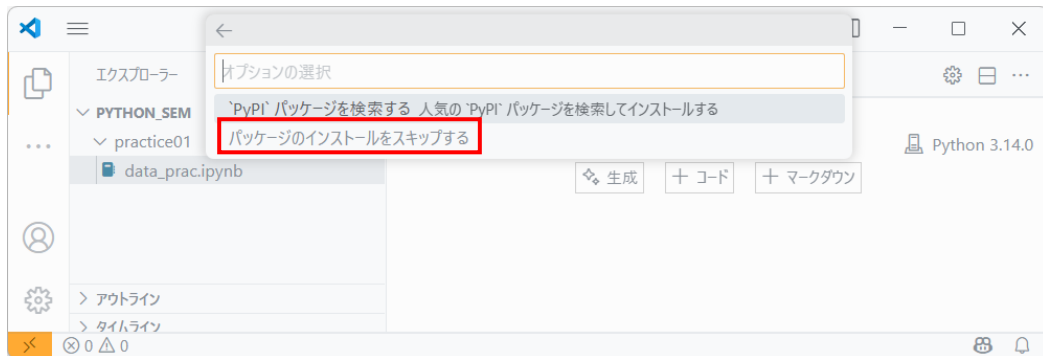


※1. Python のバージョンは 3.13 を選択してください！古いバージョンだと今後の説明で一部動作しないコードが現れます。また 3.14 が 2025 年 11 月 1 日現在の最新バージョンですが、VSCode 上の Notebook では不具合が見られたため、ここでは 3.13 を利用します。

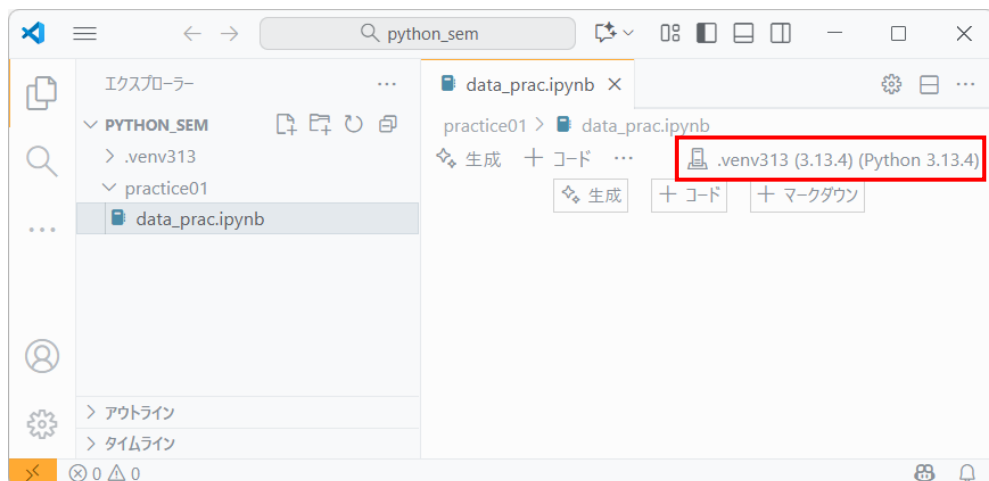
仮想環境の名前に「.venv313」と入力して Enter キーを入力します。



【パッケージのインストールをスキップする】をクリックします。



しばらくすると PYTHON\_SEM のルートディレクトリに .venv313 というディレクトリが作成されます。再度エディタ右上の【カーネルを選択】をクリックして、出てきた窓から【.venv313(3.13.4)(Python 3.13.4)】を選択します。下図のようにエディタ右上の表示が変われば OK です。



エディタの中の【+コード】を選択して、下記を入力します。



セルの左側の【▶】をクリックします。



下に「こんにちは、yamada tarou さん。」と出力されれば成功です。

## 1 - 1 Python を学ぶ

Python では操作の対象となるデータを「オブジェクト」と呼びます。先ほどのプログラムを再度見てみます。

```
name = "yamada tarou"
print("こんにちは、{}さん.".format(name))
```

このプログラムを詳しく見てみます。（今は理解できなくて大丈夫です。後で見直してみてください。）

- ① 1行目で `"yamada tarou"` というデータ（オブジェクト）に `name` という名札（変数名）を付けた。
- ② 2行目で `"こんにちは、{}さん。"` というオブジェクトが、`format(name)` という操作を行った（`"こんにちは、{}さん。".format(name)` ←この部分）
- ③ さらに `"こんにちは、{}さん。".format(name)` というオブジェクトを引数にして `print` という操作を行った。

非常に単純なプログラムですが、見てわかる通り Python を用いたプログラミングでは、**オブジェクト**と呼ばれる存在の理解が非常に重要です。

本セミナーを通して、下記の3項目の理解を目指します。

- ・ Python 言語の基礎
- ・ オブジェクト指向プログラミングの考え方
- ・ テスト駆動開発と振舞駆動開発

ではまずはオブジェクトとは何か、そして Python 言語の基礎について学んでいきます。

## 1-2 オブジェクトとは

オブジェクトとは、モノ・概念のことです。よりかみ砕くなら、広辞苑開いて名詞であるものはすべてオブジェクトです。

オブジェクト指向という考え方は、名詞で表されるすべてのモノをオブジェクトとして捉えて管理してソースコードをキレイにしようね※1！というプログラミングパラダイム※2の一つです。

※1. キレイなソースコード…というのは曖昧な言い方ですが、ざっくり言えば「ここ変更して!」とか「このバグ直して!」とか言われたときに、作った本人でなくとも容易に修正できるコードのことです。詳しくは Clean Code と検索すると出てきます。正しいオブジェクト指向を学ぶと、おのずとクリーンなコードを作成できます。

※2. プログラミングパラダイム … 例えば、「オセロゲーム作って」と100人のプログラマーに頼んだとします。まったく同じコードを書いてくることは(裏で繋がってるか、カンニングしていない限り)まずありえません。中には、後から修正は不可能なコードを出す人もいるでしょう。それでは困るということで、頭のいいプログラマーさん達が「こんな設計・考え方でプログラムすると、キレイなソースコードになるよ!」と書き方を提唱してくれました。見本になるような、模範的なプログラミング手法をまとめてプログラミングパラダイムと言います。オブジェクト指向プログラミングは、プログラミングパラダイムの一つです。

では、これから実際にオブジェクト指向プログラミングに触れてみます。見たことない文がたくさん出てきますが、一旦全体像に触れておきましょう。詳細についての学習は後に行います。

実際に現実のモノを Python のオブジェクトに落とし込んでみます。Python でオブジェクトを扱うには、オブジェクトを生成するための設計図であるクラス (Class) が必要になります。クラスはざっくり以下のような構造になっています。

### クラスのひな型

```
class ClassName(object):  
    <statement-1>  
    ...  
    <statement-N>
```

class というキーワードの後に半角スペースを入れて、任意のクラス名 (ここでは例として ClassName と書いています) を指定します。クラス名の後に(object): を書いて改行し、

そのクラスの定義を記述します。

クラスでは、オブジェクトが持つ**特性（内包するデータ）**と、オブジェクトが**できること・ふるまい・操作**と、オブジェクトの**生成時の初期化処理**を定義します。

オブジェクトを扱いたい！と言ってるのに、なぜクラスなんていう余計な概念が入ってくるんだ！という感じですが、世の中のモノの構造を広い視点でとらえるとその理由が見えてきます。

オセロを題材に考えてみます。オセロというゲームを観察すると存在するモノは、オセロの駒が 64 個、プレイヤーが 2 人、オセロのボードが 1 つです。オセロの駒というモノをオブジェクトにすると、プログラム内に 64 か所も似通ったコードが登場しそうですね。無駄無駄。

クラスという設計図を使うと、「オセロの駒を作るクラス」から、実際にオセロの駒のオブジェクトを 64 個生成するという方式をとることができます。

なんとも便利です。ではオセロの駒を Python のクラスとして定義してみましょう。エディタの【+コード】をクリックして、そこに下記のコードを入力します。（段々になっている文は tab キーを使用します）



```
practice01 > data_prac.ipynb > # オセロの駒のクラスを定義
生成 + コード + マークダウン | ▶ すべての実行 ... .venv313 (3.13.4) (Python 3.13.4)

# オセロの駒のクラスを定義
class Disc(object):
    def __init__(self, state):
        self.s = state

disc1 = Disc("白")
print("状態が{}のオセロの駒を生成しました".format(disc1.s))

[12] ✓ 0.0s Python
... 状態が白のオセロの駒を生成しました
```

入力したら【▶】ボタンを押して実行します。オセロの駒が一つ生成できました。作成したプログラムの中身を見てみます。

```
# オセロの駒のクラスを定義
class Disc(object):
    def __init__(self, state):
        self.s = state
```

- 1行目 `# オセロの駒のクラスを定義`

Python のプログラムで「#」から始まる 1 行をコメントと呼びます。コメントはプログラムの実行に影響を与えません。プログラムを見返したときや、人にプログラムを見せるときは、積極積に説明文となるコメントを書くの良いです。

- 2行目 `class Disc(object):`

`class` の後に半角スペースを空けてクラス名を定義できます。ここではオセロの駒 (Othello Disc) の設計図として Disc クラスを作成しています。クラス名の後にある () の中には、クラス名に出てくる名詞の上位概念のクラス名の名詞を記述します。上位概念や下位概念といった言葉は後述します。今は

```
class クラス名(object):
```

で好きな名前のクラスを作ることができると覚えてください。

- 3行目 `[tab]def __init__(self, state):`

先頭に `[tab]` を挿入して、コンストラクタを `def __init__(self):` で定義します。コンストラクタとは、ざっくり言えばオブジェクトの生成後の初期化処理を書く部分です。コンストラクタの中では、オブジェクトが内包すべきデータの初期化処理を書きます。 `__init__(self, state):` になっている理由は次で説明します。

- 4行目 `[tab][tab]self.s = state`

コンストラクタの中の処理の内容です。 `self` はオブジェクト自身を指します。ドット `.` はドットアクセス演算子とか、メンバアクセス演算子とか呼ばれる演算子で、オブジェクトが持つデータやふるまいにアクセスします。

つまり `self.s` では、オブジェクトが持つデータ `s` にアクセスしていることを指します。

イコール `=` は代入演算子と呼ばれるもので、右辺のデータを左辺の変数に代入 (コピー) する演算子です。

つまり今回の `self.s = state` は、「このオブジェクト (`self`) が持つ `s`」に「`state`」の値をコピーするという意味になります。なお「`state`」の値はオブジェクト生成時に指定することができます。 (`__init__(self, state):`)

次にオセロの駒オブジェクトを生成している部分を見ます。



```
disc1 = Disc("白")
print("状態が{}のオセロの駒を生成しました".format(disc1.s))
```

- ・ 1行目 `disc1 = Disc("白")`

クラスのオブジェクトを生成するには `クラス名()` と書きます。今回は `Disc` クラスのオブジェクトを生成したいので `Disc("白")` と書いています。クラス名の後のカッコには、コンストラクタに渡すデータを指定できます。

```
def __init__(self, state):
    self.s = state
```

コンストラクタを見ると `__init__(self, state)` と定義されています。 `self` は無条件で自分自身のオブジェクトを指すため、実際に渡せるデータは `state` です。

なので `Disc("白")` と書くと、「このオブジェクトが持つ `self.s` というデータ」に、「`state` (今回は「白」というデータ)」をコピーして、オブジェクトを生成するという意味になります。

クラスのオブジェクトは **インスタンス** とも呼ばれます。オブジェクト (インスタンス) が持つ固有のデータを **インスタンス変数** と呼びます。今回の `Disc` クラスのインスタンス変数は `s` のみです。

`disc1 = Disc("白")` は代入演算子が使われていることからわかるように、`Disc("白")` というオブジェクトを `disc1` にコピーしています。この時、「**変数** `disc1` に `Disc("白")` オブジェクトの参照を代入 (コピー) する」という言い方をします。

この代入によって、今生成した `Disc("白")` というオブジェクトに `disc1` という変数 (名札) でアクセスできることになります。

- ・ 2行目 `print("状態が{}のオセロの駒を生成しました".format(disc1.s))`

まずは `print()` についてです。`print()` はカッコ内に書いた **文字列** というオブジェクトの内容を表示する命令です。文字列オブジェクトはダブルクォーテーション `"` かシングルクォーテーション `'` で囲って生成します。

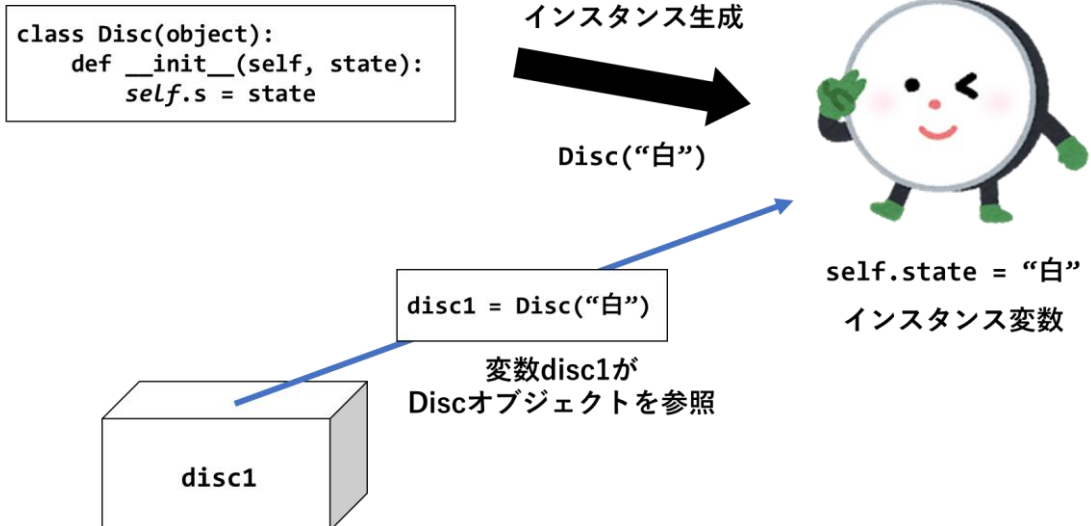
今回生成した文字列オブジェクトは `"状態が{}のオセロの駒を生成しました"` というものです。文字列というオブジェクトのふるまいの一つに「`{}`」がある部分に後から文字を入れることができる」というものがあります。

後から文字を入れるには、`.format()` を使います。`.format()` の中に文字列を入れて使います。今回は、`.format(disc1.s)` と書いているので、`disc1` に代入されたオブジェクト (`Disc("白")`) の `s` というデータにアクセスしていることになります。

クラスからオブジェクトを生成することを、**クラスのインスタンスを生成する** とも言い

ます。また、インスタンスが持つデータをインスタンス変数、インスタンスが持つふるまいをメソッドと言います。

# オセロの駒を定義する設計図（クラス）



今回作成した Disc クラスによって、Python プログラムの世界に「オセロの駒」(のインスタンス)を作り出すことができるようになりました。Disc クラスのインスタンスは Python プログラム内では Disc 型として処理されます。これを**オブジェクトのデータ型**と言います。Python では扱うデータの種類をデータ型で区別します。

#### Disc クラスの定義と、Disc 型インスタンスの生成 (完成版)

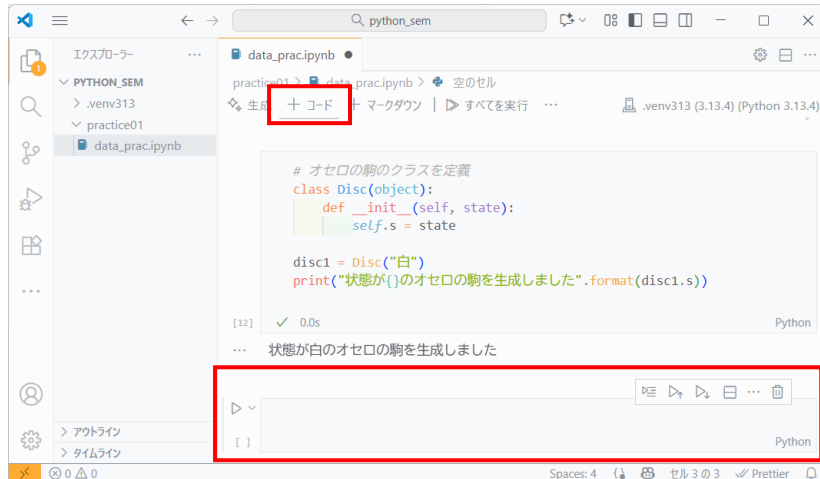
```
# オセロの駒のクラスを定義
class Disc(object):
    def __init__(self, state):
        self.s = state

disc1 = Disc("白") # 一個目のオセロの駒を生成。
print("状態が{}のオセロの駒を生成しました".format(disc1.s))

# オセロの駒は Disc 型オブジェクトとして処理される！
disc2 = Disc("黒") # 二個目のオセロの駒を生成
disc3 = Disc("白") # 三個目のオセロの駒を生成
disc4 = Disc("黒") # 四個目のオセロの駒を生成
print("追加で状態が{}, {}, {}の3つのオセロの駒を生成しました".format(disc2.s, disc3.s, disc4.s))
```

### 1-3 メソッド

オブジェクトのふるまいを定義するのがメソッドです。メソッドは引数と戻り値という二つの要素で構成されます。早速作ってみます。

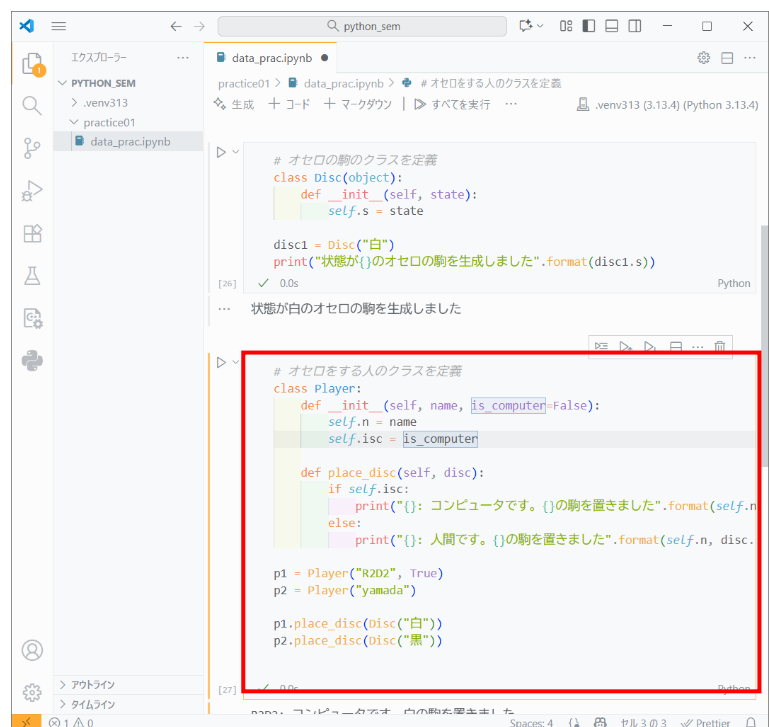


VSCodeで【+コード】をクリックします。

先ほど作成したプログラムの下に新しくセルができます。次はこのセルでプログラムを作ります。

次ページにプログラムが書いてあるのでそれを入力します。入力後は右図のようになります。

右図赤枠部分が追記箇所です。



```
# オセロをする人のクラスを定義
class Player(object):
    def __init__(self, name, is_computer=False):
        self.n = name
        self.isc = is_computer

    def place_disc(self, disc):
        if self.isc:
            print("{}: コンピュータです。{}の駒を置きました".format(self.n, disc.s))
        else:
            print("{}: 人間です。{}の駒を置きました".format(self.n, disc.s))

p1 = Player("R2D2", True) # Player 型インスタンスを生成(R2D2って名前のコンピュータ)
p2 = Player("yamada") # Player 型インスタンスを生成(yamada という名前で人間)

p1.place_disc(Disc("白")) # R2D2 が動いて駒(Disc 型で白)を置く
p2.place_disc(Disc("黒")) # yamada が動いて駒(Disc 型で黒)を置く
```

- ・ 1 行目 # オセロをする人のクラスを定義

これはコメントです。プログラムの実行に影響を与えません。

- ・ 2 行目 class Player(object):

ここで Player クラスというクラス名を与えています。「対戦選手」というオブジェクトを生成するためのクラスです。

最初の 2 行はオセロの駒クラス (Disc クラス) でやった内容と同じです。新しい文の説明の前にメソッドについて詳しく説明します。

メソッドとはオブジェクトのふるまいをプログラムにしたものです。書き方は以下の通りです。

#### メソッドのひな型

```
class A(object):
    def MethodName(self, args):
        # ふるまいを書く
        return X
```

クラスの宣言の後に tab キーで字下げして、**def** と書いてメソッドを定義します。メソッドには**引数リスト**を指定する必要があります。引数リストの中には、**メソッド内の処理で必要な変数**（**仮引数**や**ローカル変数**と呼ばれます）を列挙します。

メソッド内での処理が終了したら、データを返却する **return** 文を記述します。返却するデータがない場合には **return** 文を省略します。

- ・ 3行目 **def \_\_init\_\_(self, name, is\_computer=False):**

初めこれをコンストラクタと呼んでいましたが、厳密には **\_\_init\_\_**メソッド（声に出すときは**イニットメソッド**）と呼ばれるメソッドの一つです※。

**\_\_init\_\_**メソッドの引数リストは (**self, name, is\_computer=False**) になっています。ここで現れる、**self, name, is\_computer=False** は実際にその操作を行うまでデータの中身が確定しません。メソッドの中では、仮にデータが入ってきたら…という考え方で内容を記述します。

引数リストの中でデータの代入を行うことができます。今回、「プレイヤーがコンピュータか否か」を表す変数として **is\_computer** という変数が引数リストにあります。今回 **is\_computer=False** と書いていますがこれは、メソッドの呼び出しの際にデータを代入しなかった場合の**デフォルト値**が **False** という意味です。

※ 実は Python の公式ドキュメントでも、コンストラクタなんて呼ばれていません。最初にこれをコンストラクタと呼んだのは、「C++や Java のコンストラクタと似ている役割をしているから、しょっちゅう間違えてコンストラクタと呼ばれるため」です。実際の Python プログラマーでも Java 出身とかだと、間違えてイニットメソッドをコンストラクタと呼ぶ人も多いです。どっちでも伝わるようにどっちとも覚えておくと良いと思います。

- ・ **def place\_disc(self, disc):**

「プレイヤーがオセロの駒を置く」というふるまいをメソッドにしたものです。引数リストは(**self, disc**)ですが、**self** がオブジェクト自身（この Player 型インスタンス）、**disc** が実際に置こうとするオセロの駒（Disc 型インスタンス）を指しています。

```
if self.isc:
    print("{}: コンピュータです。{}の駒を置きました".format(self.n, disc.s))
else:
    print("{}: 人間です。{}の駒を置きました".format(self.n, disc.s))
```

`if self.isc:` はこのインスタンス(`self`)の内包するデータ(`isc...`コンピュータかどうか)を見て、もしコンピュータなら直後の文を実行、コンピュータじゃないなら `else:`以下を実行するという意味です。

- `p1 = Player("R2D2", True)`

`Player` 型のインスタンスを生成しています。インスタンスの生成時に `"R2D2"` と `True` というデータを渡しています。これが `__init__` メソッドに渡され、`__init__` メソッドの内部の処理 `self.n = name` と `self.isc = is_computer` が実行され、`Player` ができます。また、

`p2 = Player("yamada")`

という文でも `Player` 型インスタンスを生成しています。`__init__` メソッドには名前 `"yamada"` だけ渡されます。`self.isc` には `is_computer` のデフォルト値 `False` がコピーされます。

- `p1.place_disc(Disc("白"))`

メソッドの呼び出し (コール) を行っているところです。

「p1 よ、`place_disc(Disc("白"))`と動け！」と命令していると解釈しても良いです。これによって `p1` が参照するオブジェクト (`Player("R2D2", True)`) が持つふるまいの一つ `place_disc` を実行できます。引数には `Disc("白")` と書いて、状態が白のオセロの駒を渡しています。

## 1 - 4 組み込み型

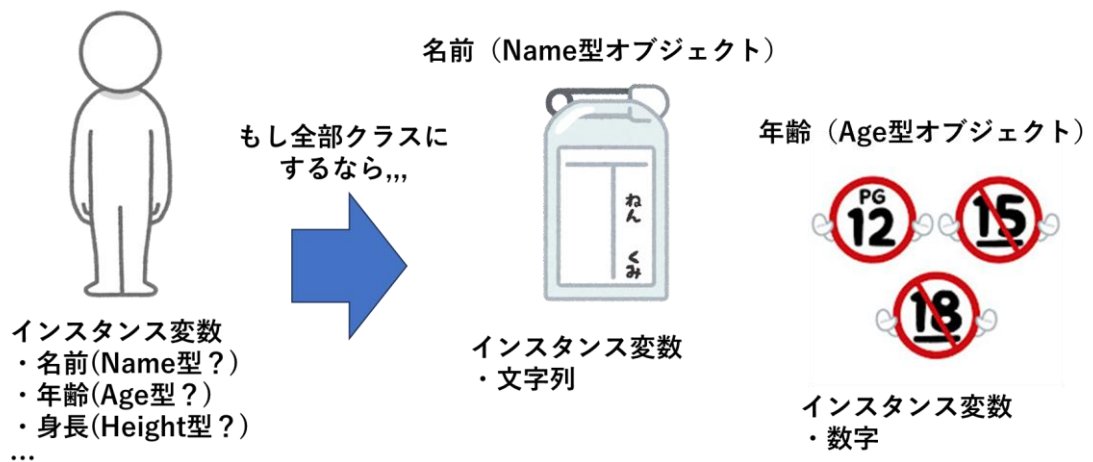
クラスを定義することでオブジェクトを生成する準備ができます。クラスを作ってしまったあとは クラス名() と書くだけです。

ところで、クラスが持つデータを考えると、例えば「人間」を定義した Human クラスなんかは、名前という文字列や、年齢という整数の数値や、身長という小数点を含む数値をもって至ります。Python でプログラムをするのだからこれらもクラスにしてあげる必要があるように感じますね（名前なら Name クラス、年齢なら Age クラスみたいな）。

しかし、それらを突き詰めると必ず限界に到達します。年齢というオブジェクトを生成する Age クラスを考えてみます。「年齢」が内包するデータは何でしょうか？ 整数値です。「年齢」という名詞が持つデータは最終的にただの整数値になります。

では今度は整数値をクラスにする必要があります。「整数値」という名詞が内包するデータは何でしょうか？「整数値」は整数値というデータそのものですね。これ以上深掘するのは賢明ではありません。

### 人間（Human型オブジェクト）



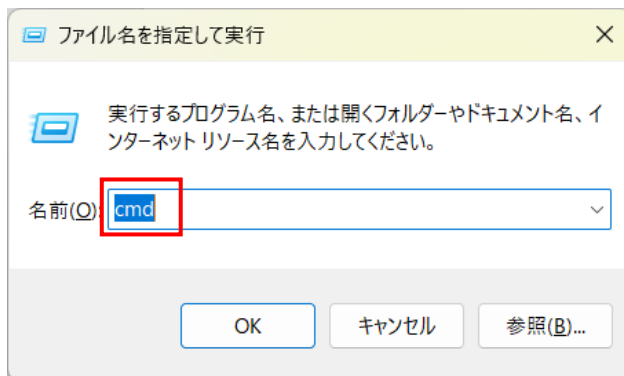


Python は**組み込み型**と呼ばれるクラスのデータ型をはじめから提供してくれています。これは突き詰めた結果、データそのものになってしまったクラスたちだと考えると良いでしょう。

組み込み型の例（一部抜粋）

| データ型      | 内容                     | 例                    |
|-----------|------------------------|----------------------|
| int 型     | 整数値を表すオブジェクト           | 100, 200, -1, 0      |
| float 型   | 小数点数を表すオブジェクト          | 1.1, 3.14, -10.0     |
| str 型     | 文字列を表すオブジェクト           | "Hello", 'こんにちは'     |
| bool 型    | Yes(真), No(偽)を表すオブジェクト | True, False (←2 つだけ) |
| complex 型 | 複素数を表すオブジェクト           | 1+2j, -2.3 + 4j      |

Python でクラスを作成する際には組み込み型を必ずと言っていいほど利用します。組み込み型について詳しく見ていきます。今回は Visual Studio Code ではなく、Python インタプリタを利用します。

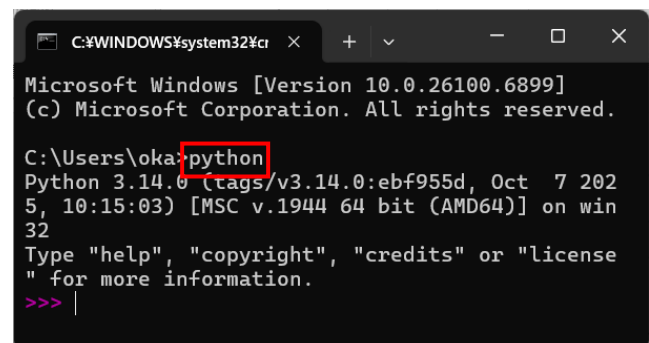


Windows キー+R で現れるウィンドウ（左図）に、「cmd」と入力します。

入力したら Enter キーを押します。コマンドプロンプトが起動します。

起動したコマンドプロンプトで  
「python」  
と入力します。入力したら Enter キーを押します。右図のように

>>>



という入力ラインが現れます。ここに Python のコードを入力すると、1 行単位で手軽にプログラムを実行することができます。以降の組み込み型の学習ではここに文字を入力していきます。

## 1-4-1 数値型

数値型には主に次の3つがあります。

| データ型             | 内容            | 例                |
|------------------|---------------|------------------|
| <b>int</b> 型     | 整数値を表すオブジェクト  | 100, 200, -1, 0  |
| <b>float</b> 型   | 小数点数を表すオブジェクト | 1.1, 3.14, -10.0 |
| <b>complex</b> 型 | 複素数を表すオブジェクト  | 1+2j, -2.3 + 4j  |

基本的に数値型のオブジェクトの生成は、直接数字を打つことで可能です。

```

>>> 100
100
>>> i = 10
>>> print(i)
10
>>> gamma = 0.57721
>>> s = 0.5 + 14j
>>>

```

上図では、

- ・ 最初に「100」という int 型オブジェクトの生成
- ・ 次に変数 i に「10」という int 型オブジェクトの参照をコピー
- ・ print(i) で変数 i の内容を表示
- ・ 変数 gamma に「0.57721」という float 型オブジェクトの参照をコピー
- ・ 変数 s に「0.5 + 14j」という complex 型オブジェクトの参照をコピー

基本的にオブジェクトを生成する場合 クラス名() で生成します。組み込み型はあまりにもよく使うため、特別にクラス名を省略して書くことができます✖。プログラム内で直接書かれた固定の値をリテラルという言い方をします。上の例にある、「i = 10」では、i が変数で 10 がリテラルです。

一応、普通のオブジェクトのように生成することも可能です✖。

```

>>> x = int(3)
>>> print(x)
3
>>>

```

次に数値のふるまいを考えます。数値には何ができるのでしょうか。パッと思いつくのは「数値は演算が定義できる」ということですね。

数値は四則演算が可能です。また、整数÷整数の時は「○ あまり△」みたいな答え方もします。割り算の商と余りというものです。また累乗なんかも計算できます。3 の 2 乗は 9 みたいな。

整数と小数点数と複素数で、若干できることは異なります（複素数を割った余りなんて定義できない）が、だいたいできることはそのくらいかなと思います。実際に数値が持つふるまい（メソッド）とその意味を列挙します。

| 使える型                | メソッド                         | 対応する演算子               | 意味                         |
|---------------------|------------------------------|-----------------------|----------------------------|
| int, float, complex | <code>__add__(x)</code>      | +                     | 自身の値に x の値を足す              |
| int, float, complex | <code>__sub__(x)</code>      | -                     | 自身の値から x の値を引く             |
| int, float, complex | <code>__mul__(x)</code>      | *                     | 自身の値に x の値をかける             |
| float, complex      | <code>__truediv__(x)</code>  | /                     | 自身の値を x で割る                |
| int, float          | <code>__floordiv__(x)</code> | //                    | 自身の値を x で割る<br>(小数点以下切り捨て) |
| int, float          | <code>__mod__(x)</code>      | %                     | 自身の値を x で割った余り             |
| int, float, complex | <code>__pow__(x)</code>      | **                    | 自身の値を x 乗する                |
| int, float, complex | <code>__neg__()</code>       | -                     | 自身の値の符号を反転する               |
| int, float, complex | <code>__abs__()</code>       | <code>abs(...)</code> | 自身の値の絶対値                   |

試しに何個か使ってみます。なお Python ではリテラルと判断されるとメソッドを利用できないため、あえて `int()` で int 型オブジェクトを生成しています。

#### int 型オブジェクトで遊ぶ

```

C:\WINDOWS\system32\cmd  ×  +  -  □  ×
>>> x = -1
>>> x.__add__(5)
4
>>> x.__sub__(5)
-6
>>> y = int(2).__add__(4).__add__(2)
>>> print(y)
8
>>> int(3).__neg__()
-3
>>> |

```

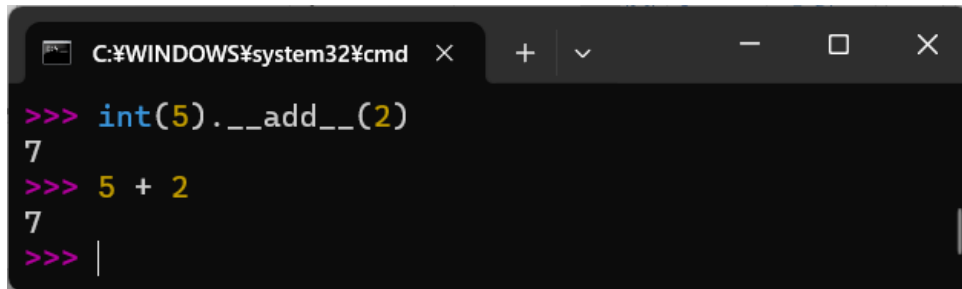
float 型や complex 型でも試してみましょう。

- ・ 10m の高さからモノを落とすと最終的に  $\sqrt{2 \cdot 9.8 \cdot 10}$  m/s の速さになります
- ・ 斜辺以外の 2 辺が 3.0, 4.0 の直角三角形の斜辺の長さは  $|3.0 + 4.0j|$  です

確かにこれで数値型の演算を行うことができますが、少し面倒に感じますね。実際に計算を行う場合、非常に便利な**演算子**を使うのが一般的です。

演算子は何かしらの記号で、左右に演算の対象（**オペランド**）を取ります。Python では内部的に、オブジェクトに定義された特殊メソッド（`__add__`など）の呼び出しを行います。

#### `__add__`メソッドと `+` 演算子



```
C:\WINDOWS\system32\cmd
>>> int(5).__add__(2)
7
>>> 5 + 2
7
>>> |
```

上の図はどちらも全く同じこと（足し算）をしています。演算子を使うとかなり簡潔して、なおかつ我々の日常的な感覚に近い形で演算を行うことができます。

メソッドと対応した演算子は前ページの表にまとめてあります。

※1. Python コードの実行の流れは、大きく次のようになっています。

1. **字句解析(Lexing)** … コードに書かれた文字を字句解析機 (lexical analyzer、**tokenizer**) で解析。要は半角スペースとか Tab 文字とかカッコから、意味のある単語 (**トークン**) を見つける作業。
2. **構文解析(Parsing)** … 字句解析で見つかったトークンから抽象構文木 (**AST**) を構築する作業。AST は人間が書く Python コードと、コンパイラが作成する 01 のバイトコードの中間の役割をするコード。
3. **コンパイル(Compilation)** … AST を 01 からなるバイトコードの変換する作業。
4. **実行(Execution)** … バイトコードを Python 仮想マシン (**PVM**) が実行する。

Python では「i = 10」というコードをみたときに、まず字句解析で i 、 = 、 10 というトークンを見つけてます。

次の構文解析で下記のような AST が生成されます。

```
Assign(      ← 代入(assign)するという意味のノード
    targets=[Name(id='i')],      ← 代入の対象(target)が i という変数
    value=Constant(value=10)     ← 代入する値(value)が Constant(value=10)というインスタンス
)
```

なお、Python3.7 以前だと Constant の部分は Num になっています。Constant は Python の ast モジュールで定義された Constant クラスのインスタンスです。この時点（構文解析の時点）で「10」という値が Constant クラスのインスタンスとして解釈されます。

ということで、省略して書くことができるというのは、正確には「リテラルを書いた場合、Python の構文解析の時に自動的に Constant インスタンスに変換される」というのが正しいです。

※2. 正確には int クラスのコンストラクタ呼び出しです。int クラスのコンストラクタに入れられる引数は、\_\_int\_\_メソッドか、\_\_index\_\_メソッドか\_\_trunc\_\_メソッドを持ったオブジェクトである必要があります。

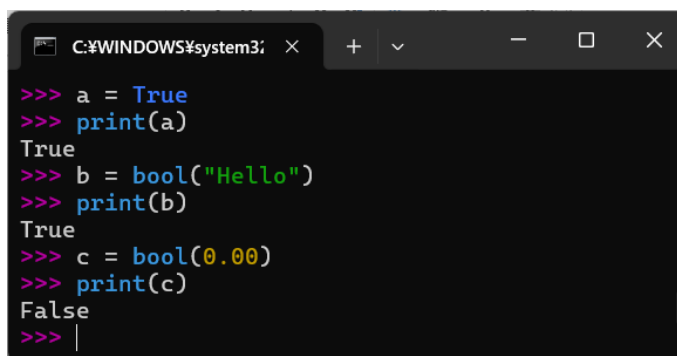
(Python3.14 以降、\_\_int\_\_か\_\_index\_\_メソッドに限定され、\_\_trunc\_\_メソッドだけを実装したクラスを int クラスのコンストラクタに渡すと TypeError が生じるようになりました)

## 1-4-2 ブーリアン型

真と偽、Yes と No、といった**真理値**を定義するデータ型がブーリアン型です。真と偽しか存在しないので、ブーリアン型のオブジェクトは Python で（この実世界でも）二つのみです。「そう」か「そうじゃない」かだけです。

ブーリアン型のリテラルは真の場合に **True**、偽の場合に **False** と書きます。明示的に bool クラスのコンストラクタを呼び出して生成することも可能です。その場合、bool クラスのコンストラクタに渡すのは、「真とみなされるオブジェクト」か「偽とみなされるオブジェクト」です。

| 真とみなされるオブジェクトの例           | 偽とみなされるオブジェクトの例           |
|---------------------------|---------------------------|
| 0 じゃない数値 (1, -100, 3.14)  | 0 の数値 (0, 0.0, -0.000000) |
| 空じゃない文字列 ("Hello", 'ABC') | 空の文字列 (" ", "")           |



```

C:\WINDOWS\system32\cmd.exe
> python
Python 3.10.0 Shell
>>> a = True
>>> print(a)
True
>>> b = bool("Hello")
>>> print(b)
True
>>> c = bool(0.00)
>>> print(c)
False
>>> |

```

True と書くとブーリアン型オブジェクト（真）が生成される。

bool()で明示的に生成することもできる。

ブーリアン型オブジェクトは数値としての側面も持ちます。True は整数の 1、False は整数の 0 です。ただし実際の演算で True とかが出てくるのは不自然なので、ブーリアン型オブジェクトには専用の演算を用いるのが一般的です。

| 演算子        | 意味                  | 使用した場合の変化                                                                                       |
|------------|---------------------|-------------------------------------------------------------------------------------------------|
| <b>not</b> | 論理値の反転              | not True → False , not False → True                                                             |
| <b>or</b>  | 論理和<br>どっちな正しければ正しい | True or True → True , True or False → True<br>False or True → True , False or False → False     |
| <b>and</b> | 論理積<br>どっちも正しいなら正しい | True and True → True , True and False → False<br>False or True → False , False or False → False |

ブーリアン型オブジェクトを生成するには

- ① True か False というリテラルを書く
- ② bool() でコンストラクタに「真または偽とみなされるオブジェクト」を渡す

という 2 種類がありましたが、実際には

- ③ オブジェクト間を比較して真か偽を生成する

という手法をとることが一般的です。そのためのメソッドを下記の表にまとめます。

| メソッド                             | 演算子                | 意味                      |
|----------------------------------|--------------------|-------------------------|
| <code>__eq__(self, other)</code> | <code>==</code>    | self と other が等しいかどうか   |
| <code>__ne__(self, other)</code> | <code>!=</code>    | self と other が等しくないかどうか |
| <code>__lt__(self, other)</code> | <code>&lt;</code>  | self が other より小さいかどうか  |
| <code>__le__(self, other)</code> | <code>&lt;=</code> | self が other 以下かどうか     |
| <code>__gt__(self, other)</code> | <code>&gt;</code>  | self が other より大きいかどうか  |
| <code>__ge__(self, other)</code> | <code>&gt;=</code> | self が other 以上かどうか     |

数値オブジェクトならイメージが付きやすいので、まずは数値で試してみます。

```

C:\WINDOWS\system32\cmd.exe
>>> int(5).__eq__(5)
True
>>> int(5).__eq__(3)
False
>>> int(5).__ne__(-10)
True
>>> int(5).__lt__(10)
True
>>>

```

int クラスでは我々の直感的なイメージ通りに各種メソッドが定義されています。

文字列の場合は辞書順になります。“こんにちは”は“さようなら”よりも小さい（辞書で手前）です。

これらには対応する演算子があり、先の表にまとめています。

```

C:\WINDOWS\system32\cmd.exe
>>> "こんにちは".__eq__("Hello")
False
>>> "こんにちは".__eq__("こんにちは")
True
>>> "Hello".__lt__("Hey")
True
>>> "こんにちは".__gt__("さようなら")
False
>>>

```

条件演算子を使って True か False を生成する

```
C:\WINDOWS\system32\cmd.exe
>>> x = 10
>>> 3 <= x
True
>>> -15 <= x and x <= 15
True
>>> name = "Bob"
>>> "D" <= name
False
>>> |
```

ブーリアン型オブジェクトの使い道として最もメジャーなのが**条件分岐**です。

「もし今日の天気が雨なら、傘をさす」

みたいな条件によって動作が変わるものを条件分岐と言います。条件分岐の実装は if 文を使用します。

```
C:\WINDOWS\system32\cmd.exe
>>> weather = "雨"
>>> if weather == "雨":
...     print("傘を指します")
... else:
...     print("...")
...
傘を指します
>>> |
```

*if 文のひな型*

```
if 条件式:
    条件式が True の時の処理
else:
    条件式が False の時の処理
```



また、下記の条件演算子という演算子的な使い方もよくします。

|                                                |
|------------------------------------------------|
| X = 条件式が True の時の値 if 条件式 else 条件式が False の時の値 |
|------------------------------------------------|

```
>>> weather = "雨"
>>> motimono = "傘" if weather == "雨" else "なし"
>>> print(motimono)
傘
>>> weather = "晴れ"
>>> motimono = "傘" if weather == "雨" else "なし"
>>> print(motimono)
なし
```

自分で `__eq__` メソッドを実装する例を下記に示します。(追記箇所を赤枠で囲っています。)

```
# オセロの駒のクラスを定義
class Disc(object):
    def __init__(self, state):
        self.s = state

    def __eq__(self, obj):
        if isinstance(obj, Disc):
            return self.s == obj.s
        else:
            return False

disc1 = Disc("白") # 一個目のオセロの駒を生成。
print("状態が{}のオセロの駒を生成しました".format(disc1.s))

# オセロの駒はDisc 型オブジェクトとして処理される！
disc2 = Disc("黒") # 二個目のオセロの駒を生成
disc3 = Disc("白") # 三個目のオセロの駒を生成
disc4 = Disc("黒") # 四個目のオセロの駒を生成
print("追加で状態が{}, {}, {}の3つのオセロの駒を生成しました".format(disc2.s,
disc3.s, disc4.s))

if disc1 == disc3:
    print("disc1 と disc3 は同じ色の駒です。")
```

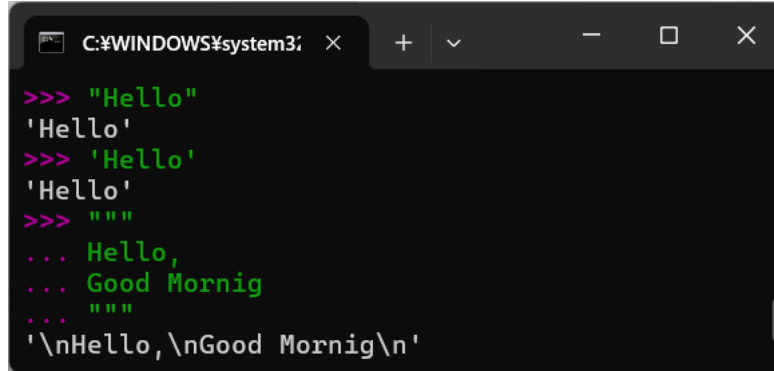
`__eq__` メソッドは **`isinstance`** という関数とセットで使われることが多いです。`isinstance` は第一引数のオブジェクトが第二引数のクラスなら `True` を返し、それ以外なら `False` を返します。

異なるクラスのオブジェクトと比較したらとにかく「違う！」と出している感じです。

```
def __eq__(self, obj):
    if isinstance(obj, Disc): # ==で比較したオブジェクトがDisc 型なら
        return self.s == obj.s # 二つのオブジェクトの色を比較
    else:
        return False # 違うオブジェクトが比較されたらすべて False
```

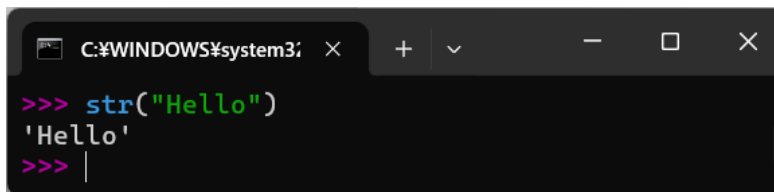
### 1-4-3 テキストシーケンス（文字列）型

「文字列」や「テキスト」を表すオブジェクトは **str 型** です。str 型オブジェクトの生成にはダブルクォーテーションかシングルクォーテーションか三連引用符を用います。



```
C:\WINDOWS\system32\cmd.exe
>>> "Hello"
'Hello'
>>> 'Hello'
'Hello'
>>> """
... Hello,
... Good Mornig
... """
'\nHello,\nGood Mornig\n'
```

また、str クラスのコンストラクタで明示的に文字列型オブジェクトを生成することもできます。



```
C:\WINDOWS\system32\cmd.exe
>>> str("Hello")
'Hello'
>>> |
```

「文字列」という名詞ができる操作を考えると、それはもう無数にありそうですね。例えば、特定の文字を置換したり、特定の文字を探したり、文字列を結合したり、文字数を数えたり。かなりの数があるのでここでは紹介せず使用時にたびたび使い方を紹介する形にします。

### 1-4-4 そのほか特殊な組み込み型のオブジェクト

ほかにも組み込み型が存在するのでまとめておきます。（実際によく使うコレクション系の型は後で扱います。）

#### 【空オブジェクト】

ヌル(Null)オブジェクトとも呼ばれます。「無」とか「空(から)」とかといった「**データが存在しない**」を表すオブジェクトです。None というリテラルを書くと、Python はそれをヌルオブジェクトとして扱います。「無」にはなんのふるまいもありません。

「データがあるかどうか」の判定に使うのがメインの使い方、**is 演算子**とセットで使います。

```
C:\WINDOWS\system32\cmd
>>> x = 10
>>> x is None
False
>>> x = None
>>> x is None
True
>>> |
```

x = 10 だとデータ  
があるので  
x is None は False

### 【Ellipsis オブジェクト】

Ellipsis 型のオブジェクトは「省略」を表すオブジェクトです。Ellipsis というリテラルを書くかドットを3つ並べて ... と書くと、Python はそれを Ellipsis インスタンスとして認識します。Ellipsis は日本語で省略記号という意味です。

ついでに似たものに **pass 文** というものがあります。pass 文は「何もしない」という動作を表す文です。何か定義するけどいったん省略するなら Ellipsis、何もしない事を定義したいなら pass 文、という使い分けになります※。

```
C:\WINDOWS\system32\cmd
>>> class Test(object):
...     def foo(self):
...         # いったん省略。後で内部を実装
...         ...
...
...     def bar(self):
...         if x:
...             print("barbar")
...         else:
...             # else句では何もしない
...             pass
>>> |
```

Test クラスというクラスを作成しているときに、foo メソッドの中身を「あとでしっかり書くよ〜」って時は Ellipsis を使うのが適当。

条件分岐とかで多いが、「何もしないよ〜」って時は pass 文を使うのが適当。

※ Ellipsis は「省略」を表すオブジェクトですが、「あとで実装するからいったん省略しとくね」という使い方以外に、「たくさんあるけど全部列挙するのは無理だから省略するね」という使い方があります。

### 【NotImplemented オブジェクト】

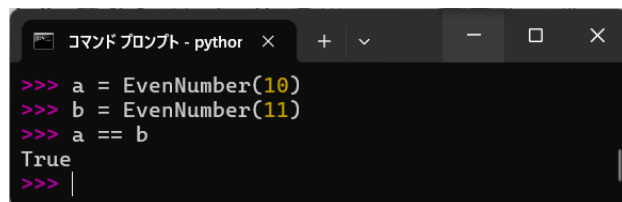
`__eq__` メソッドは演算子 `==` と対応します。そのため、`a == b` というような等しいかどうかの演算を行うと、実質 `a.__eq__(b)` が実行されます。ここで重要になるのが、二つのデータの比較は必ずしも同じデータ型とは限らない、ということです。

文字列と数字の比較とかもあり得てしまうのですね。もしかしたらオセロの駒とパソコンが比較されることもあるかもしれません。普通はただ `False` を返せばいいだけに思えますが、例えば、「偶数」を定義するクラスを作成することを考えます。

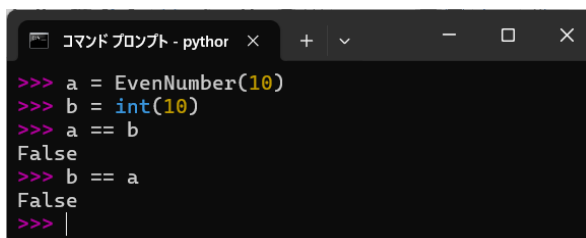
#### 偶数を定義するクラス(?)

```
>>> class EvenNumber(object):
...     def __init__(self, num):
...         if num % 2 == 0:
...             self.n = num
...         else:
...             self.n = num - 1
...     def __eq__(self, obj):
...         if isinstance(obj, EvenNumber):
...             return self.n == obj.n
...         else:
...             return False
```

実際にインスタンスを生成して比較してみます (右図)。まあ予想通りって感じですね。次に `int` 型オブジェクトと比較してみます。



```
コマンド プロンプト - pythor
>>> a = EvenNumber(10)
>>> b = EvenNumber(11)
>>> a == b
True
>>> |
```



```
コマンド プロンプト - pythor
>>> a = EvenNumber(10)
>>> b = int(10)
>>> a == b
False
>>> b == a
False
>>> |
```

左図で「偶数」の 10 と「整数」の 10 を比較しています。確かに「偶数」と「整数」では異なる概念ですが、どちらも我々の感覚では 10 に違いありません。

このような事態が起きる理由は「異なる概念(クラス)の比較結果は、`False` ではなく実装できない」ためです。`isinstance` を使って、異なるクラス型のオブジェクトを比較した結果はいつでも `False` と書いていますが、これは誤りです。

```

...     def __eq__(self, obj):
...         if isinstance(obj, EvenNumber):
...             return self.n == obj.n
...         else:
...             return False

```

←ここが違う

違う概念の比較については、毎度 False を出すのは正しくありません。視点によっては一見違うものも等しいと出したい場面もあり得ます。

このような時に使えるのが、「実装不可」を定義した **NotImplemented** 型オブジェクトです<sup>\*</sup>。NotImplemented というリテラルでオブジェクトを生成することができます。

改めて偶数を定義した EvenNumber クラスの \_\_eq\_\_ メソッドを見てみます。int 型オブジェクトとの比較は可能なので、これは elif を使って分岐させるのが正解ですね。次に EvenNumber 型オブジェクトから見て、EvenNumber 型と int 型以外のオブジェクトとの比較について考えます。これは「実装できない」ですね。もしかしたら未来で、「4 の倍数」を表す Multiple4 クラスや、「0 以上の整数」を表す FromZero クラスなどが作成されるかもしれません。当然、作成されないかもしれません。elif で実装するのは不可能です。

よって、\_\_eq\_\_ メソッドの中身は

```

EvenNumber 型オブジェクトなら self.n と obj.n を比較
int 型オブジェクトなら self.n と obj を比較
それ以外については実装できない (NotImplemented)

```

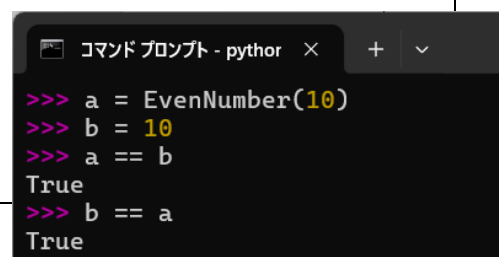
となります。

偶数を定義するクラス(完全版)

```

>>> class EvenNumber(object):
...     def __init__(self, num):
...         if num % 2 == 0:
...             self.n = num
...         else:
...             self.n = num - 1
...     def __eq__(self, obj):
...         if isinstance(obj, EvenNumber):
...             return self.n == obj.n
...         elif isinstance(obj, int):
...             return self.n == obj
...         else:
...             return NotImplemented

```



```

コマンド プロンプト - pythor
>>> a = EvenNumber(10)
>>> b = 10
>>> a == b
True
>>> b == a
True

```

※ 実装できない、とはいえ何かしらを実装しないと True も False も返ってこないです。

NotImplemented は正確には「**私では実装できません**」という意味です。クラス A とクラス B があり、a = A()、b = B()の時に a == b が実行されたとします。

内部的には a.\_\_eq\_\_(b)が実行されますが、

クラス A の\_\_eq\_\_メソッドから NotImplemented が return された場合、クラス B の\_\_eq\_\_メソッドが呼び出されます。

NotImplemented の本質は「**当該クラスで定義できないので、引数のオブジェクトのメソッドを呼び出す**」と

いうものです。なおどちらも NotImplemented の場合、両メソッドを呼び出してから False を返します。

```
>>> class A:
...     def __eq__(self, obj):
...         print("Aの eq")
...         return NotImplemented
...
>>> class B:
...     def __eq__(self, obj):
...         print("Bの eq")
...         return NotImplemented
...
>>> a = A()
>>> b = B()
>>> a == b
Aの eq
Bの eq
False
>>> b == a
Bの eq
Aの eq
False
```

## 1 - 5 コンテナオブジェクトと繰り返し

オセロアプリの続きを作成します。オセロの駒とオセロのプレイヤーが（形だけ）できたので、次にオセロのボードを作成します。オセロのボードには 8×8 のオセロの駒を置く場所があります。

```
# オセロのボードを定義
class OthelloBoard(object):
    def __init__(self, square1_1, square1_2, square...):
        # 各マスの状態を初期化(最初は何も置かれていない)
        ...

    # 盤上のオセロの駒を数える
    # 盤上でおける場所を調べる などなど
```

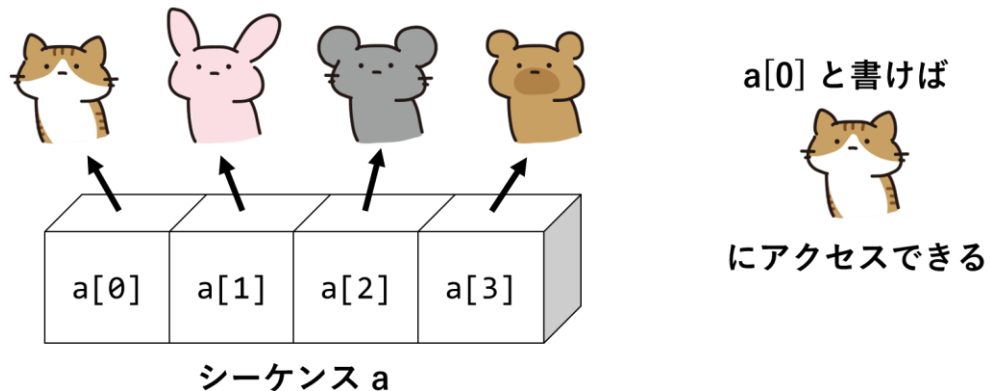
盤上のマス一つ一つにオセロの駒を置いていくとなると、都合 64 行も同じようなコードが必要になります。また、64 個もインスタンス変数を定義していく必要があります。

「オブジェクトをまとめて複数個扱いたい」という時に便利なのが、Python の組み込み型のコンテナオブジェクトと呼ばれるオブジェクトです。オセロのボードクラスを作成する前にコンテナオブジェクトについてさらっておきます。

## 1-5-1 シーケンス

複数のデータをまとめて扱うことができるのがコンテナオブジェクトですが、中でも**順序を持つ一続きのもの**をシーケンスオブジェクトと言います。

シーケンスオブジェクトには、リスト、タプル、文字列が該当します。シーケンスオブジェクトの特徴は、**各データに整数値でインデックスがある**という部分です。



- リスト

[ ] を使ってカンマ区切りでデータを並べると、0 から順番にインデックスを振られたリストオブジェクトを作成することができます。

```

コマンドプロンプト - python
>>> num_strs = ["ZERO", "ONE", "TWO", "THREE"]
>>> num_strs[2]
'TWO'
>>>

```

上記では文字列オブジェクトを4つ格納した num\_strs というリストオブジェクトを作成しています。リスト内の特定のデータにアクセスするには [整数値] と書きます。上記では num\_strs のインデックスが2のオブジェクトにアクセスしています。

- タプル

( ) を使ってカンマ区切りでデータを並べると、0 から順番にインデックスを振られたタプルオブジェクトを作成することができます。

```

コマンドプロンプト - python
>>> primes_lt_15 = (2, 3, 5, 7, 11, 13)
>>> primes_lt_15[3]
7
>>>

```

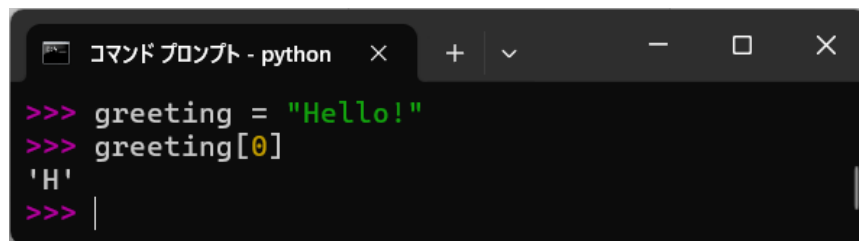


上記では 15 未満の素数を `primes_lt_15` というタプルオブジェクトに入れています。リストと同じく、タプル内のデータにアクセスするには [整数値] と書きます。上記では `primes_lt_15[3]` と書いているのでインデックスが 3 のオブジェクトにアクセスしています。

なお、ただ単にカンマ区切りでデータを並べるとタプルオブジェクトとして認識されます。後述するアンパック代入ではよく利用します。

- 文字列

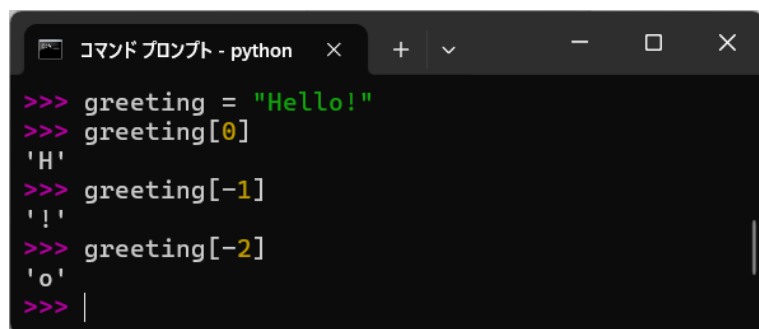
ダブルクォーテーションかシングルクォーテーションか三連引用符を使って、任意のテキストデータを囲うと、**各文字に 0 からインデックスがふられた文字列オブジェクト**を作成することができます。



```
コマンド プロンプト - python
>>> greeting = "Hello!"
>>> greeting[0]
'H'
>>> |
```

上記では Hello! という文字列オブジェクトを作成し、変数 `greeting` に格納しています。また、`greeting[0]` とすることで "Hello" のインデックスが 0 の文字にアクセスしています。

インデックスは 0 から順にふられていますが、末尾から負の整数を使うこともできます。

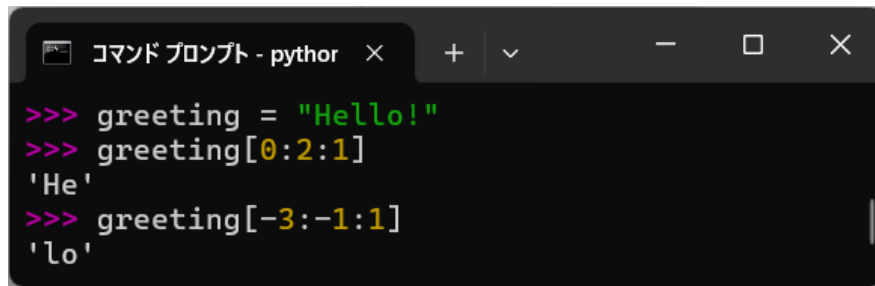


```
コマンド プロンプト - python
>>> greeting = "Hello!"
>>> greeting[0]
'H'
>>> greeting[-1]
'!'
>>> greeting[-2]
'o'
>>> |
```

シーケンス特有の便利な操作にスライスというものがあります。スライスはシーケンスの**部分**を抽出します。指定の仕方は

[開始インデックス : 終了インデックス : 増分値]

という書き方です。



```

>>> greeting = "Hello!"
>>> greeting[0:2:1]
'He'
>>> greeting[-3:-1:1]
'lo'

```

リストとタプルは非常によく似ていますが、リストの方は**ミュータブル（変更可能）**、タプルは**イミュータブル（変更不可能）**という特徴があります。

### 1-5-2 集合(set)オブジェクト

集合(set)は順序を持たず重複を許さないコンテナオブジェクトです。 集合オブジェクトの生成には `{ }` を使用します。

```

>>> favorite_color = {"赤", "水色"}
>>> print(favorite_color)
{'水色', '赤'}

```

### 1-5-3 辞書(dict)オブジェクト

キー (Key) と値 (Value) でデータを管理できるコンテナオブジェクトが**辞書型オブジェクト**です。

```

>>> color_code = {"RED": "#FF0000", "IVORY": "#FFFFFF0"}
>>> print("赤のカラーコードは " + color_code["RED"])
赤のカラーコードは #FF0000

```

シーケンスが整数インデックスでデータにアクセスするのに対して、辞書では各データにキーを用いてアクセスします。

## 1-5-4 繰り返し

コンテナオブジェクトは繰り返しによって内部のデータを**走査**することができます。繰り返しには2種類あって、コンテナオブジェクトの内部を走査する場合には**for 文**を利用するのが一般的です。ついでにもう一つの繰り返しの仕方である**while 文**についてもここで説明しておきます。

## ① for 文による繰り返し

for 文は次の構文で使います。

```
for ループ変数 in イテラブルオブジェクト:
    ...
```

イテラブルオブジェクトの中の要素が一つずつ、ループ変数に格納されて操作が行われます。イテラブルオブジェクトとは、「繰り返し処理が可能な」オブジェクトのことです。コンテナオブジェクトはイテラブルオブジェクトの一種です。

下記にいくつか例を書いておきます。

文字列に特定の文字が何個含まれるか数える

```
>>> strs = "Hello World!"
>>> cnt = 0
>>> for s in strs:
...     if s == "l":
...         cnt += 1
...
>>> print("「 {}」に 含まれる l の数は {} 個です ".format(strs, cnt))
「 Hello World!」に 含まれる l の数は 3 個です
```

辞書の内容を表示する

(辞書型オブジェクトに.keys()を使用することですべてのキーを取得できる)

```
>>> color_code = {"RED": "#FF0000", "IVORY": "#FFFFF0"}
>>> for cc in color_code.keys():
...     print("{}の カラー コード は {} です ".format(cc, color_code[cc]))
...
REDの カラー コード は #FF0000です
IVORYの カラー コード は #FFFFF0です
```

for 文はよくイテラブルオブジェクトの一種である **range オブジェクト** と使用されます。range オブジェクトは「繰り返し可能な整数値の集まり」で、特定の回数の繰り返しの実現に使われます。

```
>>> for i in range(3):  
...     print(i)  
...  
0  
1  
2
```

左の図では、range オブジェクトに対して for 文で走査を行っています。

for 文の繰り返しの中では print を使って要素の中身を表示しています。

実際に使ってみると、range オブジェクトの要素が 0 から引数の数未満までであることが分かります。

```
>>> for i in range(10):  
...     print(" if i % 2 == 0 else i")  
...  
1  
3  
5  
7  
9
```

range オブジェクトを使って奇数の場合のみ数字を表示するように作成した for 文です。今回は 10 未満の奇数をすべて表示しています。

Range オブジェクトはその性質上、ループ変数に意味がない場合も多いです。

「Hello…」と 3 回表示する for 文。ループ変数 i はループ内で使われない。

```
>>> for i in range(3):  
...     print("Hello...")  
...  
Hello...  
Hello...  
Hello...
```

意味のない変数の名前に気を遣うのもばからしいので、ループ変数に意味がない場合には慣習的に変数名を `_` (アンダースコア) とします。人によってはこれを捨て変数と呼びますが、特に決まった呼び方はありません。

「Hello!」と3回表示する for 文

```
>>> for _ in range(3):
...     print("Hello!")
...
Hello!
Hello!
Hello!
```

「変数名をアンダースコアにすると、代入した値を廃棄する」みたいな誤った話がありますが、ふつうにただの変数なので、代入も演算も可能です。

```
>>> for _ in range(5):
...     a = _ + 1
...     b = _.__add__(5)
...     print(f"{a = }, {b = }")
...
a = 1, b = 5
a = 2, b = 6
a = 3, b = 7
a = 4, b = 8
a = 5, b = 9
```

## ② 内包表記

任意のイテラブルオブジェクトからコンテナオブジェクトを簡単に生成することができる内包表記という記法があります。繰り返し構文とは言えませんが、for 文に非常に似ているのでここで紹介します。

内包表記の使い方は下記のようになります。下記はリストの内包表記の例ですが、すべてのコンテナオブジェクトで共通します。

[リストに入れたい値 **for** ループ変数 **in** イテラブルオブジェクト **if** 条件]

最後にある条件の if は省略することができます。いくつか例を見ておきます。

偶数のリスト

```
>>> even_num = [i * 2 for i in range(10)]
>>> print(even_num)
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
```

平方数の集合

```
>>> sq_num = {i**2 for i in range(5)}
>>> print(sq_num)
{0, 1, 4, 9, 16}
```

range オブジェクトは引数を二つ指定すれば初期値を決めることができます。

1 月から 12 月までの月を格納した集合

```
>>> month = {"{}月".format(i) for i in range(1,13)}
>>> print(month)
{'7月', '10月', '8月', '9月', '2月', '4月', '12月',
'1月', '5月', '11月', '6月', '3月'}
```

内包表記は入れ子にすることも可能です。

九九の表を定義する辞書を内包表記で作成。

キーが”〇〇の段”、値がリストでその段の九九の値になっている。

```
>>> times_table = {f"{i}の段": [i * j for j in range(1, 10)] for i in range(1, 10)}
>>> print(times_table["3の段"])
[3, 6, 9, 12, 15, 18, 21, 24, 27]
>>> print(times_table["9の段"])
[9, 18, 27, 36, 45, 54, 63, 72, 81]
```

## ③ while 文

イテラブルオブジェクトの中を走査できるのが for 文でした。もう一つの繰り返しは「条件が True の間繰り返す」という条件に依存した繰り返しです。

```
while 条件式:
```

```
...
```

ユーザからの入力で -1 が入力されるまで繰り返す。

```
>>> x = 0
>>> while x != -1:
...     x = int(input("-1を 入力したら 終了"))
...
-1を 入力したら 終了5
-1を 入力したら 終了3
-1を 入力したら 終了-1
>>> |
```

ユーザからの入力で"exit"と打たれるまで、入力文字をオウム返しする。

```
>>> while (line := input("*** ")) != "exit":
...     print(f"入力: {line}")
...
*** Hello!
入力: Hello!
*** Good Morning!
入力: Good Morning!
*** exit
```

上記で `:=` はセイウチ演算子 (Walrus Operator) と呼ばれるもので、代入しながら式に使えるというものです。

## 1-5-5 アンパック代入

変数に値を代入する際に、右辺にイテラブルオブジェクトを取ると**アンパック代入**という代入が可能です。複数の変数にまとめてデータを代入することができ、非常に便利です。アンパック代入ではイテラブルオブジェクトの要素数と左辺の変数の数が一致する必要があります。もし、複数変数の代入の際に足りないデータがあれば習慣的に `_` に入れます。

```
>>> a, b, c = [1, 3, 5]
>>> print(a)
1
>>> _, x, _ = [1, 3, 5]
>>> print(x)
3
```

アンパック代入はメソッドと利用することが多いです。

平方数の和と平均を計算する

```
>>> class DataAnalysis:
...     def sum_ave(self, nums):
...         sum = ave = 0
...         for n in nums:
...             sum += n
...         ave = sum / len(nums)
...         return sum, ave
...
>>> sq_nums = [i**2 for i in range(1, 10)]
>>> s, a = DataAnalysis().sum_ave(sq_nums)
>>> print("1から10までの平方数の和:{}".format(s))
1から10までの平方数の和:285
>>> print("1から10までの平方数の平均:{}".format(a))
1から10までの平方数の平均:31.666666666666668
```



## 2 CUI アプリ開発

実際に Python で CUI アプリケーションを開発します。CUI (Character User Interface) とは、PC とユーザ間の情報のやり取りを文字(Character)で行うものです。CUI アプリケーションとはキーボードと文字が出てくる画面だけで遊べるアプリケーションのことです。

この CUI アプリケーションの作成を通して、オブジェクト指向プログラミングの全体像と、そのほかの Python 周辺知識を学んでいきます。

今回はオブジェクト指向プログラミングの考えに沿って、オセロアプリを開発します。CUI のオセロアプリでは下記の機能を実装します※。

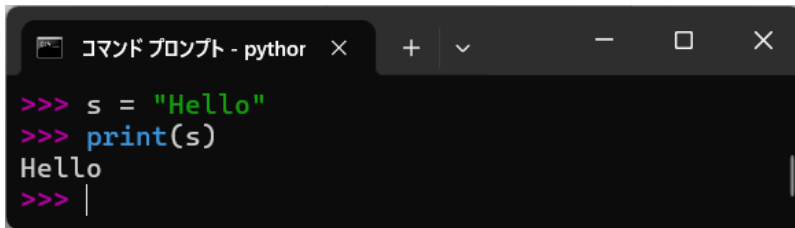
- ① 初めに、人間 vs 人間 か 人間 vs コンピュータ のいずれかを選択する
- ② コンピュータであれば強さを選択する
- ③ ゲームが開始したらオセロの駒を置いていく
  - ・ルール上、駒をおけない場所が指定された場合は選択しなおし
  - ・自分の手番でおける場所がない場合は置ける場所ができるまで相手の手番になる
  - ・片方のオセロの駒の色が盤面空なくなるか、盤面がすべて埋まったら終了
- ④ 終了時に盤面の駒の数を表示する

登場する主なオブジェクトは、人間、コンピュータ、オセロの駒、オセロの盤面あたりです。ほかに必要なものがあればその都度追加していきます。

※ 今回作成するプログラムのより詳細な設計書は別紙でお配りします。

## 2-1 モジュール

これから CUI アプリケーションを作成するにあたって、Python プログラムの種類についてまとめておきます。Python はインタプリタ型で 1 文ずつプログラムを実行することが可能です。



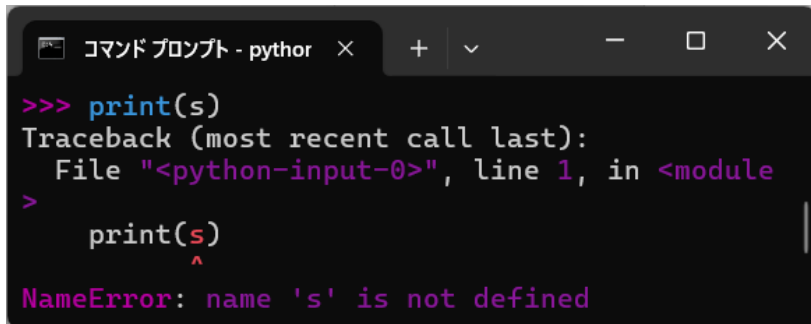
```
>>> s = "Hello"
>>> print(s)
Hello
>>> |
```

“Hello” という文字列型のオブジェクトを生成して、変数 `s` で参照する

`print(s)` で表示する

しかし作成するプログラムが複雑になると、どうしても複数行まとめて書きたい場合が出てきます。Python のインタプリタで入力したプログラムは、インタプリタの終了と同時に消えるので、再び起動したインタプリタでまた同じコードを書く必要があります。

一度コマンドプロンプトを終了させてから、再度 Python で `print(s)` を入力したところ、`NameError` というエラーが発生！→

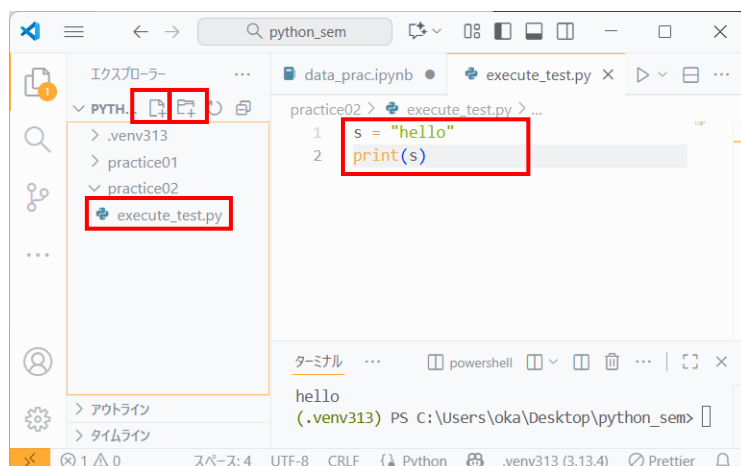


```
>>> print(s)
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    print(s)
    ^
NameError: name 's' is not defined
```

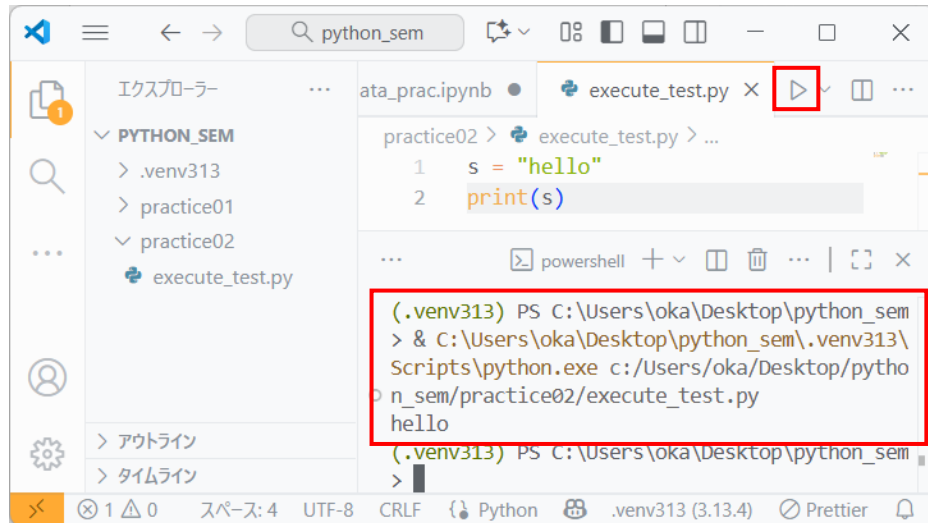
これが 2 行程度であればまあ何とか毎回やれないこともないですが、100 行のクラスとかを毎回書くとなると本当に面倒です。Python では実行可能な複数行を定義したファイルを **スクリプト** と呼びます。スクリプトは `.py` という拡張子でファイルを作成します。

Visual Studio Code を起動し、「practice02」というフォルダを作成します。新しく作成したフォルダ内にスクリプト「execute\_test.py」というファイルを作成します

ファイル内に先述の 2 行を入力します。



execute\_test.py を作成したら実行します。Visual Studio Code 上部の【▶】を押します。

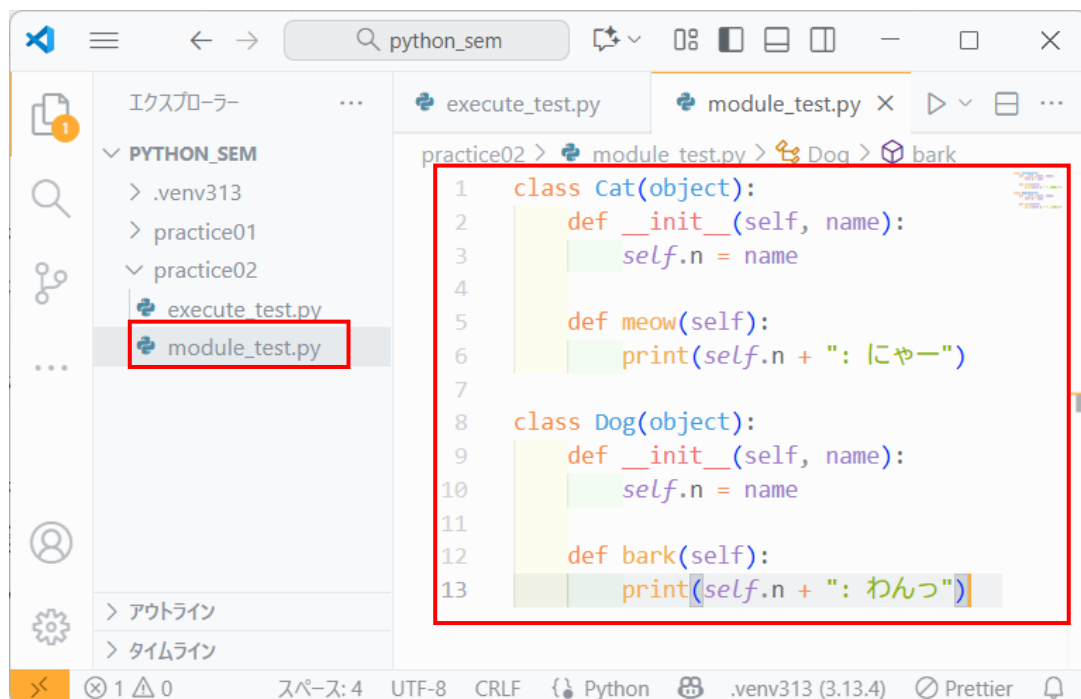


スクリプトは実行可能な.py ファイルです。実行の際には python コマンドをターミナル（コマンドプロンプト）に打ち込みます。

### *python* スクリプト名.py

Visual Studio Code では【▶】を押すと自動で上記のコマンドを入力してくれます。

スクリプトは実行の流れをズラッと書いたものですが、別の.py ファイルで使用することを念頭に置いた.py ファイルをとくにモジュールと言います。モジュールにはクラスの定義や関数（後述します）を書きます。

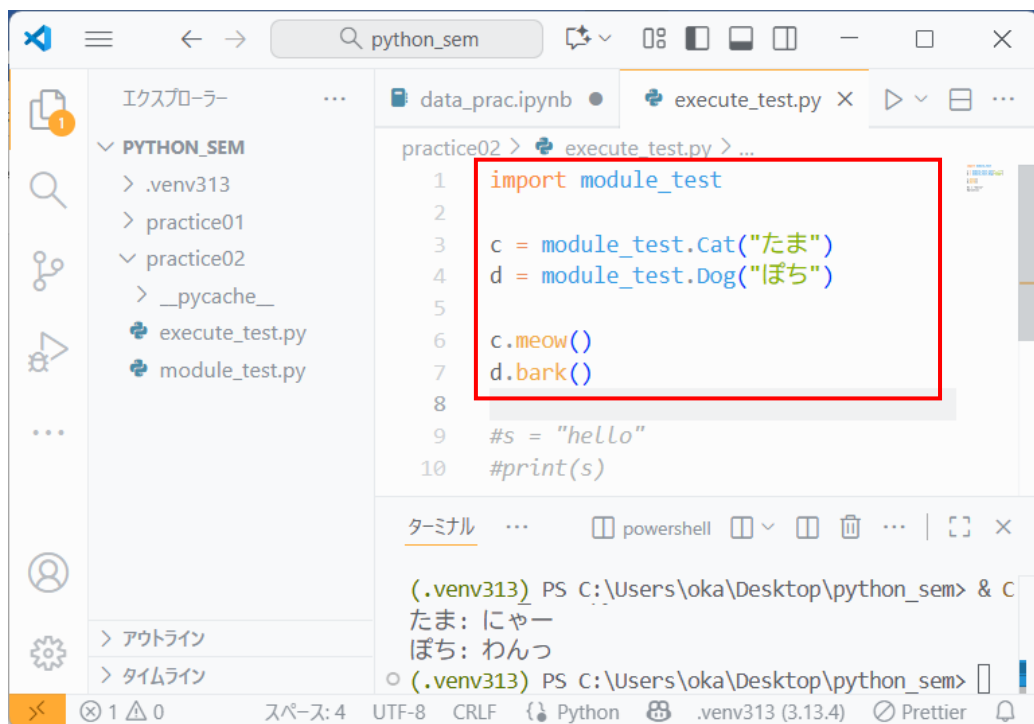


モジュールを使うには **import 文** というモノを使用します※。

### *import* モジュール名

※ インポート文は正確には、現在の**名前空間 (namespace, ns)** に、import の直後の名前を追加する文です。名前空間についての詳細は省きますが、このテキストでの import 文の説明はかなりざっくりしたものだと思います。

前ページのモジュール (module\_test.py) を作成後、execute\_test.py の内容を下記のように変更します。

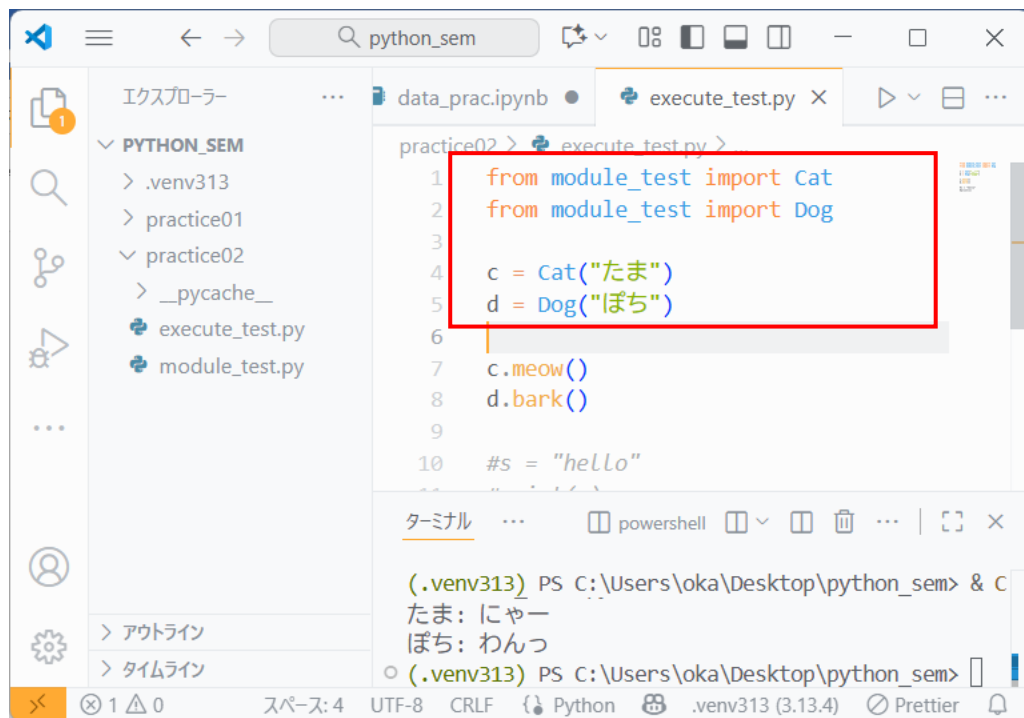


実行すると、import 文を使用することによって、確かに Cat クラスと Dog クラスのオブジェクトを使用できていることがわかります。しかし現在のインポートの仕方ではクラスを使用するときに毎度毎度 module\_test. と書く必要があって面倒です。

import 文ではモジュール内のクラスや関数の名前を直接追加することも可能です。書き方は下記のようになります。

### *from* モジュール名 *import* クラス名

このように書くと実際に使用するときモジュール名を省略することができます。



モジュール名を省略してクラスを使用することができました。ついでに紹介だけしておくと、

### *from モジュール名 import \**

という書き方をするとそのモジュール内の、名前が `_` (アンダーバー) から始まるもの以外のすべての名前を `import` します。非常に楽ができますが、名前の衝突の危険性などの理由から推奨されません。

【参考】Python の公式ドキュメント

<https://docs.python.org/ja/3.13/tutorial/modules.html#more-on-modules>

`import` 文でモジュールを追加する際に、「特定のフォルダのモジュールしかインポートできない」ことを知っておきましょう。自分でモジュールを作成した場合、それをインポートするなら下記の3か所のいずれかにモジュールを配置します。

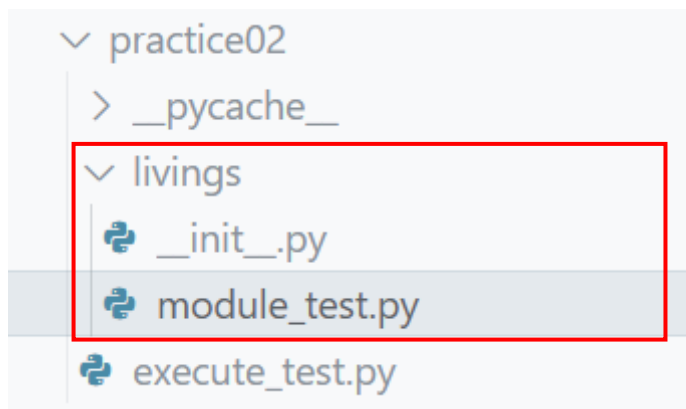
- ・ 実行時に自分がいるフォルダ（カレントディレクトリ）にあるモジュール
- ・ 実行ファイルがあるフォルダのモジュール
- ・ 環境変数 `PYTHONPATH` で指定したフォルダのモジュール

### 2-1-1 パッケージと\_\_init\_\_.py

モジュールはパッケージに配置します。パッケージは少し特殊なフォルダで、Python の世界では **\_\_init\_\_.py** というファイルが置いてあるフォルダのことを指します。

Python3.3 以降のバージョンでは **\_\_init\_\_.py** が無くてもパッケージとして認識されるようになりましたが、過去バージョンとの互換性のためにも作成しておくのが無難です。

practice02 フォルダの下に「livings」フォルダを作成します。その中に「**\_\_init\_\_.py**」という Python ファイルを作成します。中身は空で構いません。次に「module\_test.py」をドラッグアンドドロップで livings フォルダに移動します。最終的に下記の図のようになります。



パッケージに属するモジュールをインポートする場合、下記の import 文を書きます。

*from パッケージ名.モジュール名 import クラス名*

execute\_test.py の import 文を書きのように修正します。

```
from practice02.livings.module_test import Cat
from practice02.livings.module_test import Dog
```

修正したら再度実行します。正しく実行されます。

**\_\_init\_\_.py** にはモジュールをインポートしたときの初期化処理を書くことができます。Print 文を追加して再度プログラムを実行してみましょう。

### 2 - 1 - 2 Import Path の追加

Python で自作のモジュールを作成したとき、それを使用するには特定のフォルダをインポートパスに追加する必要があります。追加するには環境変数 PYTHONPATH に入れるか、一時的なものであれば下記の文でパスの追加を行うことができます。

```
import sys
import os
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '追加したいパス')))
```

モジュールインポートの前にこの文を追加することで、任意のディレクトリにあるパッケージをインポートすることができます。

## 2-2 オセロの駒 列挙型

まずはオセロの駒を定義するクラスを作成します。オセロの駒は、ゲームの最中にボードに置かれます。ボードから見れば「白い駒」「黒い駒」「何もない」の**3つの状態を取る**ことさえできれば良いですね。

逆に言えば、それは現実問題ホントにオセロの駒である必要はないということになります。10 円玉でも、会社の名刺でも、なんならサイコロでも、オセロというゲームは成立させることができます（サイコロなら例えば、1 の目が「何もない」、6 の目が「白の駒」、3 の目が「黒の駒」のように定義すればよいのです）。

ここにおいて、オセロの駒のオブジェクトは、実質「白」「黒」「無」という 3 状態を定義できる何かなら何でも良いということがわかります。このように特定の定数しかとらないクラス型を**列挙型**と言います。「オセロの駒」のオブジェクトは「白」「黒」「無」の 3 つです。列挙型のクラスとする例を下記に書いておきます。

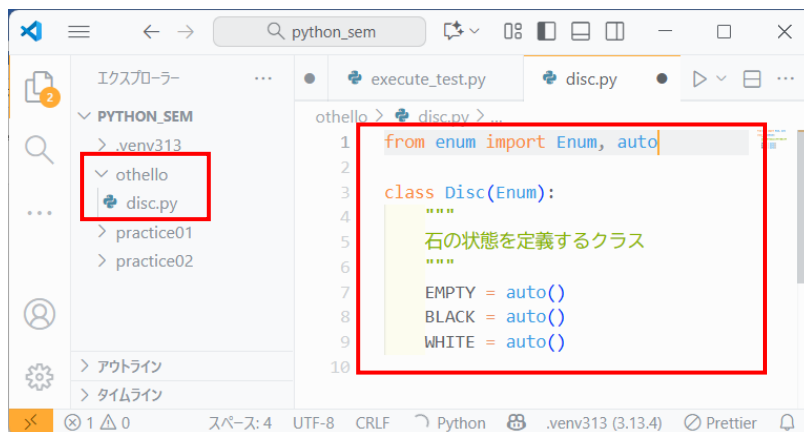
- ・ 「**光の三原色**」という名詞をクラスにするとき、これには「**赤**」「**緑**」「**青**」というオブジェクトしか存在しません。
- ・ 生物分類学での「**界**」をクラスにするなら日本では「**動物界**」「**植物界**」「**菌界**」「**原生生物界**」「**モネラ界**」の五界のみがオブジェクトです。
- ・ 物理学の標準模型で定義される「**ゲージ粒子**」をクラスとすれば、「**グルーオン**」「**光子**」「**Z ボソン**」「**W ボソン**」の 4 つのみがオブジェクトです。
- ・ 麻雀の「**三元牌**」をクラスとすると、「**白**」「**發**」「**中**」がオブジェクトです。

このように、クラスのオブジェクトの種類が有限個に制限されている場合に列挙型を使用します✖。ポーカーの役（ワンペア、ツーペア、…、ロイヤルストレートフラッシュ）、トランプのスート（スペード、ハート、クラブ、ダイヤ）、住宅の構造（木造、鉄骨、RC）など、列挙型は非常によく使用します。

それでは実際に「オセロの駒」のクラス（列挙型）を作成します。列挙型のクラスを作成するには Enum クラスを利用します。Enum クラスは enum モジュールで提供されています。

列挙型のクラスでは定数群（オセロの駒なら「白」「黒」「無」）を、クラス内に列挙していきます。これらの定数には**メンバー値**を定義する必要があります。メンバー値は列挙型内の識別コードですが、基本的には auto クラスのコンストラクタを使用して自動で割り振ります。





「othello」フォルダを作成し「disc.py」というモジュールを作成します。

「disc.py」の中に下記のコードを入力します。

*disc.py*

```
from enum import Enum, auto

class Disc(Enum):
    """
    駒の状態を定義するクラス
    """
    EMPTY = auto()
    BLACK = auto()
    WHITE = auto()
```

上記のコードで、オセロの駒(`Disc(Enum)`クラス)のオブジェクトが3つ(`EMPTY`、`BLACK`、`WHITE`)できました。ほかにも例として、トランプのスイートなら次のようになります。

トランプのスイートの列挙型

```
from enum import Enum, auto

class Cardsuit(Enum):
    """
    トランプのスイートを定義するクラス
    """
    SPADES = auto()
    HEARTS = auto()
    CLUBS = auto()
    DIAMONDS = auto()
```

以上で「オセロの駒」は完成しましたが、次に`""`と単体テストについて説明します。

※ 列挙型を使う時に注意すべき点は、「**列挙型のオブジェクトはこの世界でただ一つであるべき**」というものです。

「スペード」というオブジェクトはこの世でただ一つです（もちろん「スペードの 5」であるトランプカードは無数にあるでしょう）。「赤」も「木造」もこの世でただ一つです（「赤いリンゴ」や「木造アパート」はこの世界にたくさんありますが、「赤」や「木造」が指す実態は目に見えないただ一つのものであります）。

### 2-2-1 ドキュメンテーションコメント

今回のオセロの駒クラスでは`""`という三連引用符が出てきました。これは文字列型インスタンスを生成するものでしたが、クラス宣言の直後かメソッド宣言の直後に`""`を書くと、ドキュメンテーションコメント (docstring) になります。

クラスのドキュメントには **クラス名.\_\_doc\_\_** で、メソッドのドキュメントには **クラス名.メソッド名.\_\_doc\_\_** でアクセスできます。

基本的には <https://peps.python.org/pep-0257/> に書かれた規約に沿ってドキュメントを作成します。しかし広く波及させる予定のないプログラムであれば、上記のように簡単なコメントで問題ないでしょう。

### 2-2-2 Top-level code

モジュールをインポートすると、`__name__` にモジュール名が格納されます。

先述の Cat と Dog クラスを使用する `practioce02/execute_test.py` に次のコードを入力して試してみましょう。

```
print(Cat.__name__)
```

「Cat」という出力が得られます。

実行する際に指定するモジュール（`【>】`を押したとき開いている.py ファイル）を**トップレベルのコード (Top-level code)** と言います。トップレベルのコードはそのアプリケーションの**エントリーポイント**とも呼ばれます。

トップレベルで実行されたモジュールの `__name__` には `__main__` という文字列が入ります。これを利用して、そのモジュールの**単体テスト**を記述することができます。

[https://docs.python.org/ja/3.13/library/\\_\\_main\\_\\_.html](https://docs.python.org/ja/3.13/library/__main__.html)

### 2-2-3 超簡易版の単体テスト

トップレベルで実行したモジュールの `__name__` には `__main__` という文字列が入ります。これを利用して、`disc.py` というモジュールの中で「もしこのモジュールが直接実行されたら…」という if 文を追記できます。

今回は「もし `disc.py` モジュールが直接実行されたら … Disc クラスの単体テストをする」という風に追記します。

*disc.py*

```
from enum import Enum, auto
```

```
class Disc(Enum):
```

```
    """
```

```
    石の状態を定義するクラス
```

```
    """
```

```
    EMPTY = auto()
```

```
    BLACK = auto()
```

```
    WHITE = auto()
```

下記を追記↓

```
# 簡単な単体テストコード
```

```
if __name__ == "__main__":
```

```
    print(Disc.__doc__) # ドキュメントの表示
```

```
    assert Disc.BLACK is not None, "「BLACK」の駒がないのは困ります!!"
```

```
    assert Disc.WHITE.name == "WHITE", "「WHITE」の駒の名前は 'WHITE' です!!"
```

追記した文を見てみます。 `if __name__ == "__main__"` で「このモジュールを直接実行されたら…」という文になります。

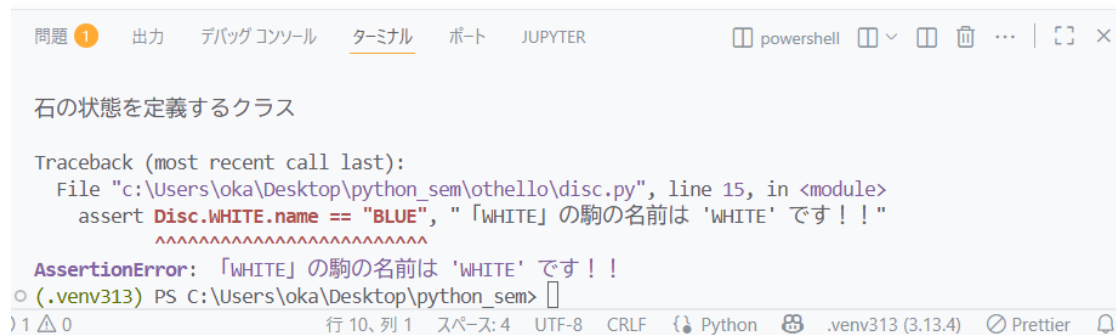
次の1行で `print(Disc.__doc__)` と書いていますが、これによって Disc クラスのドキュメントを表示しています（この1文はテストとは無関係です）。

次の `assert` 文がテストケースを記述している部分です。書き方は次の通りです。

**assert 条件式, 条件が False の時のメッセージ**

条件式の部分が False の時、Python のプログラムは実行中にエラーとなり、コンソールに上記で書いたメッセージが表示されます。今回のテストケースはすべて成功するため、実行すると Disc クラスのドキュメントだけ表示されます。

わざとテストに失敗するように `assert Disc.WHITE.name == "BLUE"` と変更して実行してみましょう。下記のエラーが表示されます。



The screenshot shows a JupyterLab interface with a terminal window open. The terminal displays a traceback for an `AssertionError`. The error message is: `AssertionError: 「WHITE」の駒の名前は 'WHITE' です!!`. The traceback points to line 15 of `disc.py`, where the assertion `assert Disc.WHITE.name == "BLUE", "「WHITE」の駒の名前は 'WHITE' です!!"` is located. The terminal window has tabs for '問題 1', '出力', 'デバッグ コンソール', 'ターミナル', and 'ポート'. The status bar at the bottom shows '行 10、列 1', 'スペース: 4', 'UTF-8', 'CRLF', 'Python', and '.venv313 (3.13.4)'.

```
問題 1 出力 デバッグ コンソール ターミナル ポート JUPYTER powershell
石の状態を定義するクラス

Traceback (most recent call last):
  File "c:\Users\oka\Desktop\python_sem\othello\disc.py", line 15, in <module>
    assert Disc.WHITE.name == "BLUE", "「WHITE」の駒の名前は 'WHITE' です!!"
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: 「WHITE」の駒の名前は 'WHITE' です!!
(.venv313) PS C:\Users\oka\Desktop\python_sem>
```

非常にシンプルなクラス（列挙型など）であれば、そのクラスのテストコードは同一クラス内で記述することも多いです。

## 2-3 オセロのボード テスト駆動開発

次にオセロのボードを定義するクラスを作成します。オセロのボードにはオセロのルールも詰め込みます。イメージは、オセロの駒を置いたらルールに則って自動で駒がひっくり返るようなボードです。

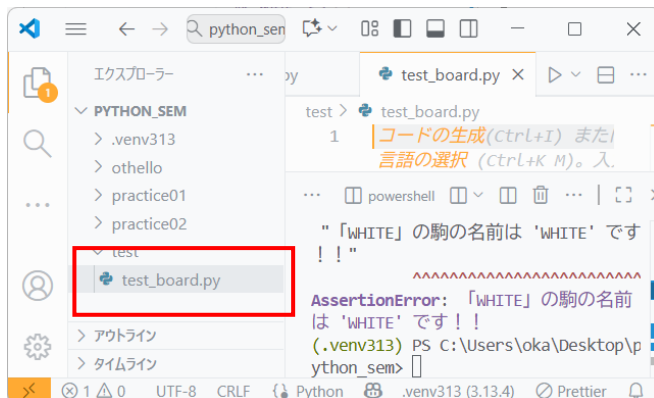
ゲームのルールは複雑なロジックを持つ場合が多いです。この複雑なルールを場当たりにメソッドとして実装しては、後から「これが動かない!」「あれがうまくいかない!」という事態も起こりかねません。複雑なメソッドの修正は最小限に抑えたいものです。

今回のオセロのボードクラスは、多くの**アジャイルソフトウェア開発手法**で推奨されている**テスト駆動開発** (test-driven development; **TDD**) で作成します。TDD による開発手順はざっくり次の通りです。

- ① 各機能について先にテストコードを書く (テストファースト)
- ② テストをパスする最小限のコードを書く
- ③ コードをキレイにする (リファクタリング)

先にテストコードを書くので、テストコードを書いた時点では必ずテストに失敗(**RED**)します。②で書く最小限のコードとはテストに成功(**GREEN**)すればどんな書き方でも構いません。③でコードのリファクタリングを行いますが、リファクタリングとは「**その機能を保持したまま、コードを見やすくする**」行為を指します。

### 2-3-1 単体テストフレームワーク [unittest]



まずはテストコードを格納するためのフォルダを作成します。名前を「test」とします。

test フォルダの中にモジュール「test\_board.py」を作成します。

これから単体テストを記述しますが、Python で単体テストを記述する場合にいくつかの選択肢があります。一つはオセロの駒の時に書いたような assert 文を列挙するやり方です。しかしこれはかなり煩雑になるうえ、実行時のエラーメッセージも見にくいです。

一般的に単体テストを記述する場合はテストフレームワークを使用します。今回は最もメジャーなテストフレームワークである **unittest** を利用します。

ではこれから Board クラスを、unittest を使用した TDD で開発していきます。

#### **Test Case 1:**      ボードの初期化

初めに書くテストは初期化に関するテストです。オセロのボードが正しく初期化されているかをテストします。

##### ① テストケースを作成する

test\_board.py に下記のテストクラスのひな型を書きます。

test\_board.py

```
# othello パッケージへ Path を通す
import sys
import os
sys.path.append(os.path.abspath(os.path.join(os.path.dirname(__file__), '..')))

# テストコードを書いていく
import unittest
from othello.board import Board
```

```

from othello.disc import Disc

class TestBoard(unittest.TestCase):
    def test_initial_board_setup(self):
        """オセロボードの初期化が正しく行われているかのテスト
        """
        ...

```

Path を通す部分はパッケージのところで説明したとおりです。

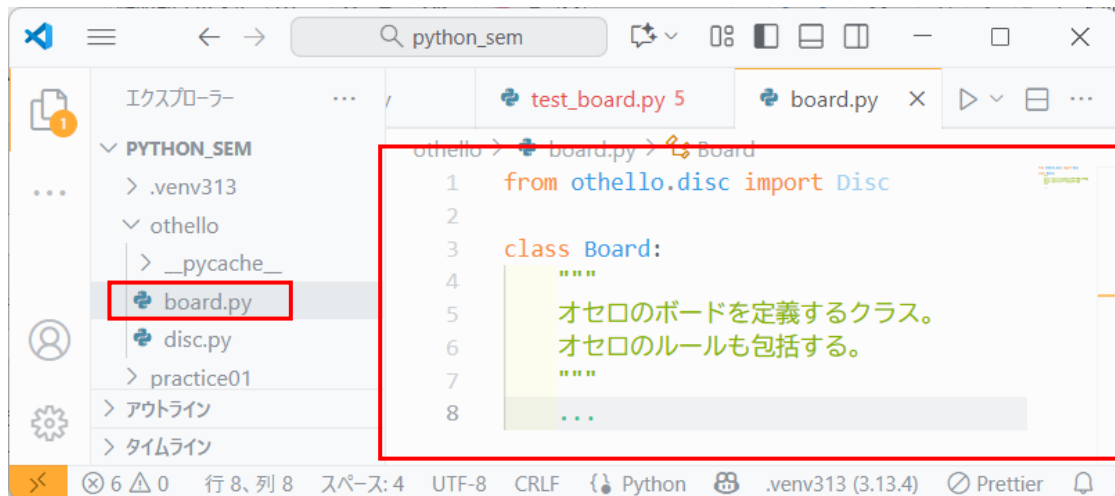
次に `import unittest` です。これによってテストフレームワーク unittest を利用できるようになります。

次に class 宣言部ですが、テストを行うクラスは一般的に **Test[テスト対象]** という名前のクラスになります。現場によっては **[テスト対象]TestCase** や **[テスト対象]Tester** などのバリエーションが見られますが、いずれにせよ「あるクラスをテストするクラス」であることが明確な名前を付けます。

そしてメソッド `def test_initial_board_setup(self):` 部について、テストを行うメソッド名は必ず **test** から始まる名前にします。これは unittest が test から始まるメソッドだけをテストケースとして認識するためです。

メソッド内に実際にテストを書いていきますが、一旦コメントで何をするテストケースなのかを書いて Ellipsis で中身を省略しています。先に board モジュールのひな型を作っておきます（board モジュール自体がないと import 文でエラーになるため）。

othello フォルダの下に board.py というモジュールを作成し、下記のコードを入力します。



こちらでもドキュメントだけ書いて本文は(Ellipsis 使って)省略しています。

開発するクラス (Board) とそれをテストするクラス (TestBoard) を作成したら、テストケースを入力していきます。

*test\_board.py* (import 文や path を通す文を省略)

```

class TestBoard(unittest.TestCase):
    def test_initial_board_setup(self):
        """オセロボードの初期化が正しく行われているかのテスト
        """
        board = Board()
        self.assertEqual(board.grid[3][3], Disc.WHITE)
        self.assertEqual(board.grid[4][4], Disc.WHITE)
        self.assertEqual(board.grid[3][4], Disc.BLACK)
        self.assertEqual(board.grid[4][3], Disc.BLACK)
        # 中央の4マス以外は空であることも確認
        self.assertEqual(board.grid[0][0], Disc.EMPTY)

```

`board.grid[3][3]` は3行3列目に置かれたオセロの駒を表します (行と列のインデックスは0から開始します)。オセロのボードは最初、真ん中の4マスに白と黒の駒が2個ずつ置かれます。最初のテストは「オセロのボードを作成した段階で、ちゃんとその初期状態になっているのか」です。

```
self.assertEqual(board.grid[3][3], Disc.WHITE)
```



`self.assertEqual` は、引数の二つのオブジェクトが等しくないときに `AssertionError` というエラーを送出してテストの失敗を知らせます。上記は

「3行3列目の駒 (`board.grid[3][3]`) が白の駒 (`Disc.WHITE`) じゃなきゃダメ」というテストコードになります。

今回勝手に `board.grid[3][3]` という書き方でマスの上の駒を見ていますが、ここは赤線が引かれていると思います。なぜなら `Board` クラスの中身は現段階でまっさらだからです。

では一度テストを実行してみましょう。下記のコードを `test_board.py` の末尾に追記します。unittest の実行は `unittest.main()` で行います。

*test\_board.py (末尾に追記)*

```
if __name__ == '__main__':
    unittest.main()
```

末尾に追記したら実行ボタン【▶】を押します。下記が出力されます。



```
問題 6 出力 ターミナル ... powershell
(.venv313) PS C:\Users\oka\Desktop\python_sem> & C:\Users\oka\Desktop\python_sem\Scripts\python.exe c:/Users/oka/Desktop/python_sem/test/test_board.py
setup)
オセロボードの初期化が正しく行われているかのテスト
-----
Traceback (most recent call last):
  File "c:\Users\oka\Desktop\python_sem\test\test_board.py", line 16, in test_initial_board_setup
    self.assertEqual(board.grid[3][3], Disc.WHITE)
                      ^^^^^^^^^
AttributeError: 'Board' object has no attribute 'grid'
-----
Ran 1 test in 0.002s

FAILED (errors=1)
(.venv313) PS C:\Users\oka\Desktop\python_sem>
```

テストに通らない状態を **RED** と言います。現状、テストに失敗したため `board` モジュールの `Board` クラスは RED です。エラーメッセージを見て修正していきます。

今回のエラーメッセージは

```
AttributeError: 'Board' object has no attribute 'grid'
```

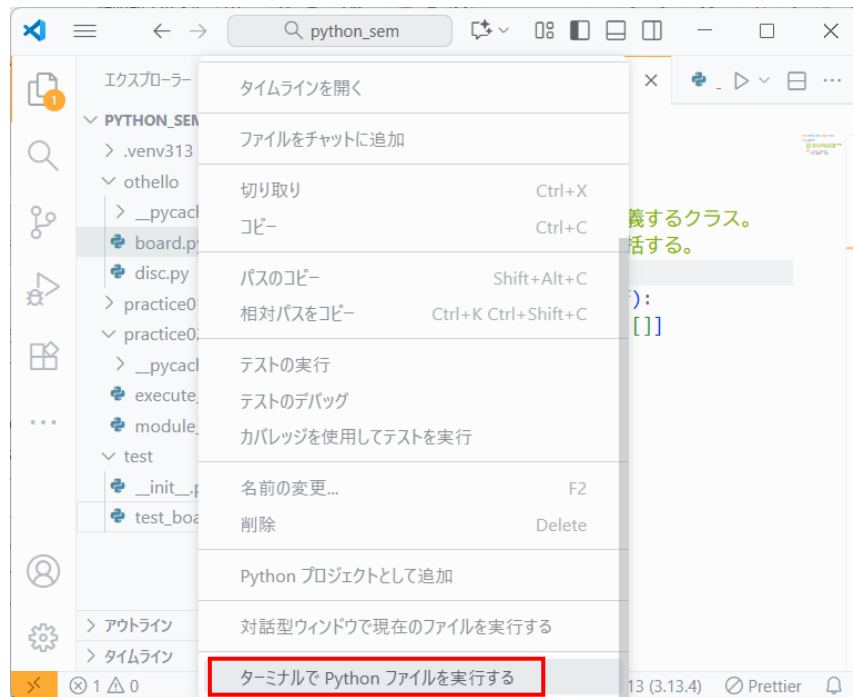
というものです。AttributeError はオブジェクトに存在しない属性にアクセスしたときに生じるエラーです。'Board' object has no attribute 'grid' からわかるように、Board クラスに grid というインスタンス変数がないことに起因するエラーです。

## ② モジュールを作成する

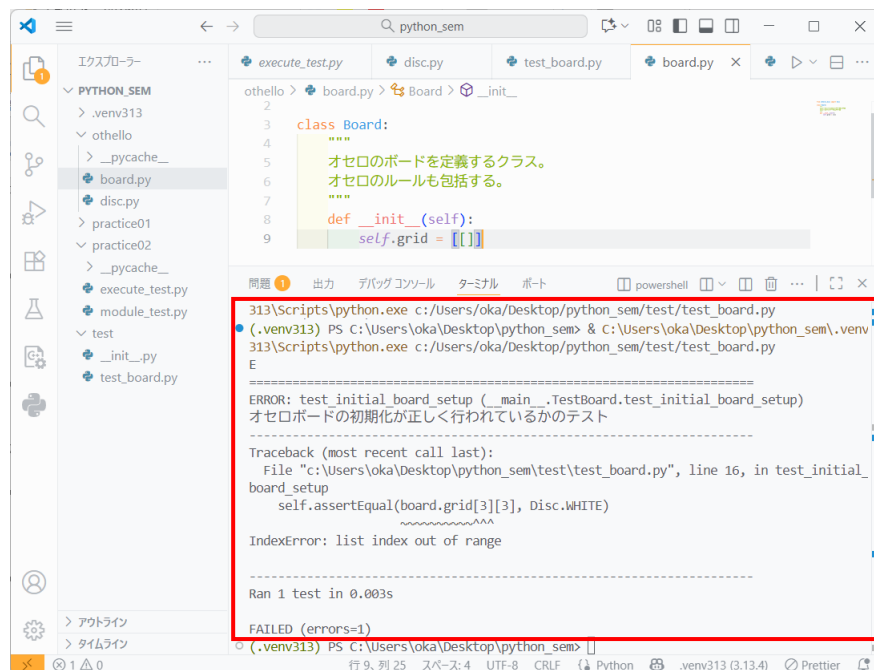
エラーメッセージに合わせて、Board クラスにコンストラクタを追加し、grid というインスタンス変数を持たせます。アクセスの仕方から明らかなですが、grid は 2 次元のリストになります。いったん空のリストにしています。

```
othello > board.py > Board
2
3 class Board:
4     """
5     オセロのボードを定義するクラス。
6     オセロのルールも包括する。
7     """
8     def __init__(self):
9         self.grid = [[]]
```

再度テストを実行しますが、毎度毎度 test\_board.py を開いて実行して、また board.py に戻って…とやるのは面倒です。Visual Studio Code では任意のファイルを右クリックから実行できるので、その方法で test\_board.py を実行します。



board.py を開いた状態で、Visual Studio Code の左にある test\_board.py を右クリックします。【ターミナルで Python ファイルを実行する】をクリックすると test\_board.py が実行されます（下図）。



今度は IndexError というエラーが発生しています。これは存在しないリストのインデックスを指定したときに生じるエラーです。今回の Board クラスでは grid を空にしていたの

でマス目が存在しない状態になっているためです。

エラーを修正します。オセロのマス目は8×8なのでリストをそのサイズにします。また、各マスにはいったん「無」のオセロの駒を配置しておきます。



```

othello > board.py > Board > __init__
2
3 class Board:
4     """
5     オセロのボードを定義するクラス。
6     オセロのルールも包括する。
7     """
8     def __init__(self):
9         self.grid = [[Disc.EMPTY for _ in range(8)] for _ in range(8)]

```

修正したらファイルを保存 (ctrl + s) して再度テストを実行します。

```

F
=====
FAIL: test_initial_board_setup (_main_.TestBoard.test_initial_board_setup)
オセロボードの初期化が正しく行われているかのテスト
=====
Traceback (most recent call last):
  File "c:\Users\oka\Desktop\python_sem\test\test_board.py", line 16, in test_initial_board_setup
    self.assertEqual(board.grid[3][3], Disc.WHITE)
    ~~~~~^~~~~~
AssertionError: <Disc.EMPTY: 1> != <Disc.WHITE: 3>
=====
Ran 1 test in 0.003s

FAILED (failures=1)

```

今度は AssertionError が発生します。これは自分で書いた下記のテストケースに違反したということです。

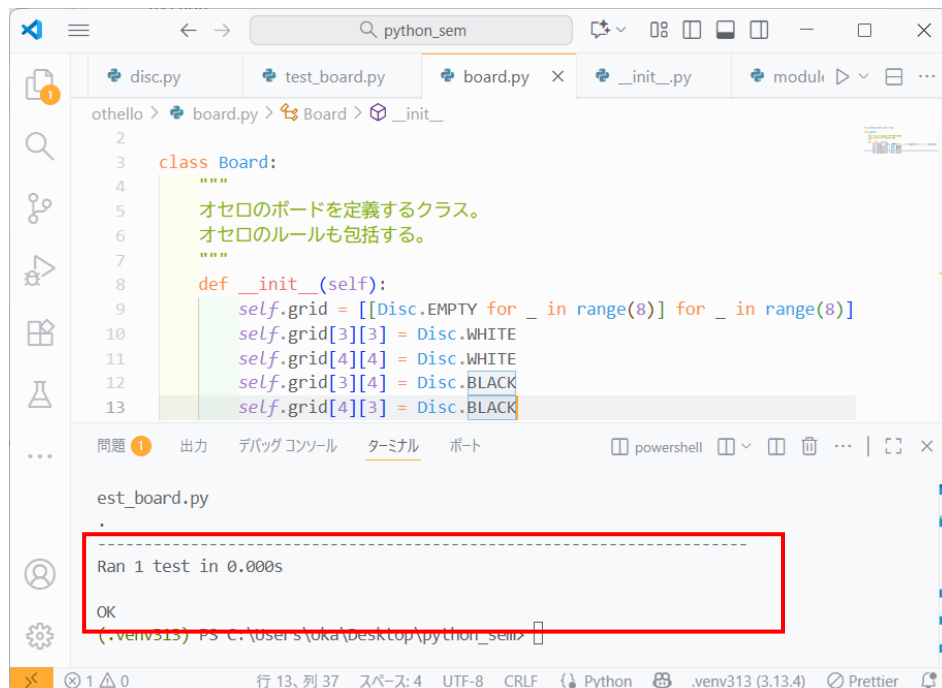
```
self.assertEqual(board.grid[3][3], Disc.WHITE)
```

ボードの初期状態は真ん中の4マスに駒を置く必要があります。これを実装します。

*board.py* (*Board* クラスの `__init__` メソッド)

```
def __init__(self):
    self.grid = [[Disc.EMPTY for _ in range(8)] for _ in range(8)]
    self.grid[3][3] = Disc.WHITE
    self.grid[4][4] = Disc.WHITE
    self.grid[3][4] = Disc.BLACK
    self.grid[4][3] = Disc.BLACK
```

`__init__` メソッドの中身が完成したら保存して、再度 *test\_board.py* を実行します。



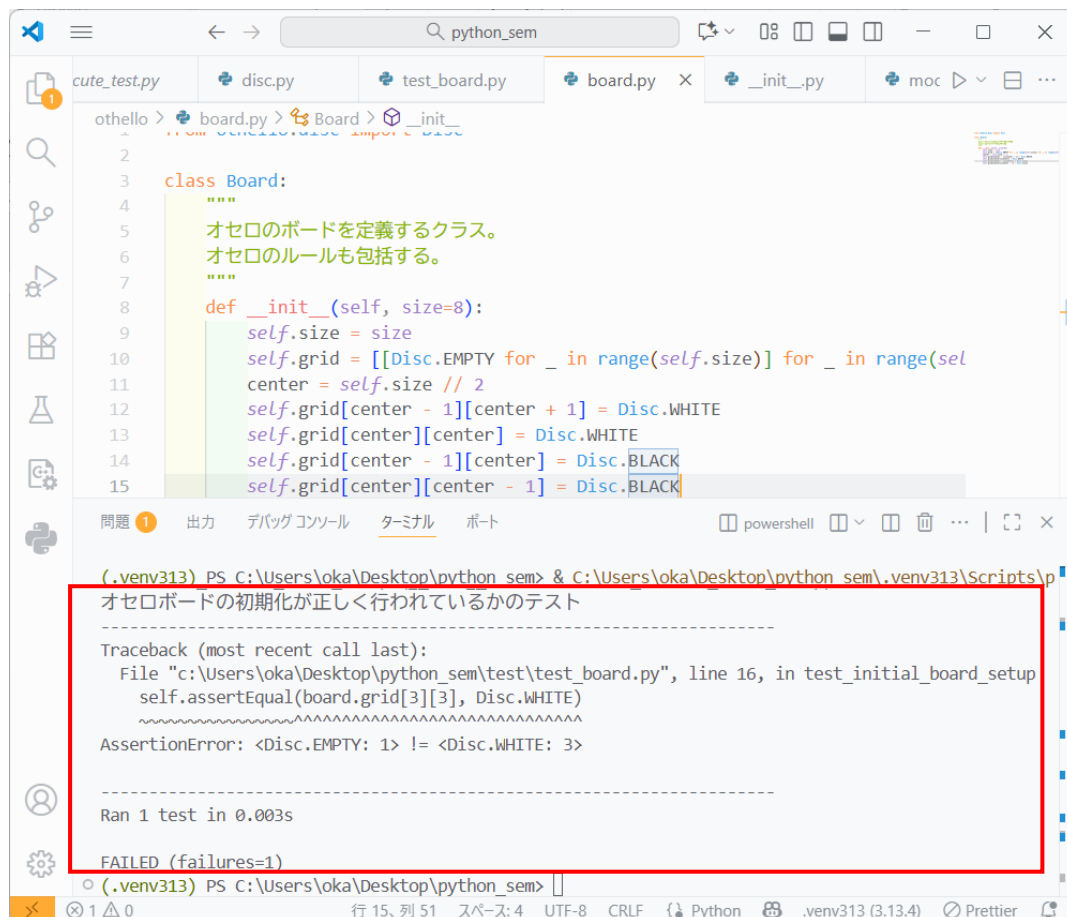
「OK」という表示が現れればテストに通ったことになります。テストをクリアした *Board* クラスは **GREEN** な状態になったと表現されます。

ほかのテストケースも記述しますが、まずは **GREEN** な *Board* クラスに対してリファクタリングを行います。例えば「ボードのサイズはデフォルトで  $8 \times 8$  だけ変更できるようにする」といった拡張で次のようなコードを入力したとします。

board.py (Board クラスの\_\_init\_\_メソッド)

```
def __init__(self, size=8):
    self.size = size
    self.grid = [[Disc.EMPTY for _ in range(self.size)] for _ in range(self.size)]
    center = self.size // 2
    self.grid[center - 1][center + 1] = Disc.WHITE
    self.grid[center][center] = Disc.WHITE
    self.grid[center - 1][center] = Disc.BLACK
    self.grid[center][center - 1] = Disc.BLACK
```

大幅に修正が行われました。再度テストを行います。



テストに失敗したことがわかります。基本的にリファクタリングは非常に小さいステップで行うことが推奨されます。リファクタリングは、プログラムの動作自体は変更しないことが前提です。修正した結果 RED 状態になったなら、それはリファクタリングではなく単

に RED なモジュールを作成しただ、再度 GREEN になるまで修正する必要があります。

今回は例としてわざと間違えましたが、テスト駆動開発では「大幅なリファクタリング (のようなもの)」は非推奨です。慣れるまではリファクタリングとはクラス名やメソッド名などの名前を適切に変える作業 (Rename Symbol) だと思っておくと良いと思います※。大規模なプロジェクトについてリファクタリングのつもりが、大幅な修正をしてしまいテストが通らないなどがありえて非常に危険です。

※ もちろん、実際にはほかにもリファクタリングのやり方が存在します。名前の変更(Rename Symbol)以外では、主にマジックナンバーの置き換え、複数個所で重複する部分の切り出し等があります。リファクタリングはアプリケーションの実行に影響を与えないという特徴から、「いつやるべきか (そもそも必要なのか)」「どの行為をリファクタリングというのか」がよく議論になります。リファクタリングにかけた時間はまったくもってアプリケーションに影響を与えないため、リターンがなくコストだけがかかるという意見もあります。しかし長い目を見たときに、「可読性が高く機能拡張性を有し、品質が高いために追加要件にかかる開発難易度が軽減する」というのはメリットであると思います。ある種、「未来での開発・生産性と品質改善」を良好にする投資の役割を果たすのがリファクタリングだと思います。

単純にコード修正をする作業では先述の例のようにテストが通らなくなる可能性があります。テスト駆動開発ではこまめにテストをすることで、「修正するだけで満足しない」状況を作ることができます。今回、テストが通らなかったですが、テスト結果を見れば修正箇所は明らかです。

【修正版】 `board.py` (`Board` クラスの `__init__` メソッド)

```
def __init__(self, size=8):
    self.size = size
    self.grid = [[Disc.EMPTY for _ in range(self.size)] for _ in range(self.size)]
    center = self.size // 2
    self.grid[center - 1][center - 1] = Disc.WHITE
    self.grid[center][center] = Disc.WHITE
    self.grid[center - 1][center] = Disc.BLACK
    self.grid[center][center - 1] = Disc.BLACK
```

上記のように修正することでテストに通るようになります。

**Test Case 2: 駒の数を数える**

次に駒の数を数えるテストです。初期状態で 2 つの白の駒と 2 つの黒の駒があるはずで  
す。これをテストします。

## ① テストケースを作成する

次のコードを test\_board.py に追記します。

test\_board.py (追記部分のみ。TestBoard クラスの最後に追記。)

```
def test_count_disc_initial(self):
    """オセロボードの上の駒の数を数えるテスト
    """
    board = Board()
    black_disc, white_disc = board.count_discs()
    self.assertEqual(black_disc, 2)
    self.assertEqual(white_disc, 2)
```

board.count\_discs() でボードの上の駒の数を、黒い駒・白い駒の順にリターンしてく  
れることを期待しています。初期状態ではいずれの色の駒も 2 個ずつであることをテスト  
しています。(self.assertEqual(black\_disc, 2)の部分)

実行するとエラーであることがわかります (RED)

```
E.
=====
ERROR: test_count_disc_initial (_main_.TestBoard.test_count_disc_initial)
オセロボードの上の駒の数を数えるテスト
-----
Traceback (most recent call last):
  File "c:\Users\oka\Desktop\python_sem\test\test_board.py", line 27, in test_count_disc_initial
    black_disc, white_disc = board.count_stones()
                                ^^^^^^^^^^^^^^^^^
AttributeError: 'Board' object has no attribute 'count_stones'

-----
Ran 2 tests in 0.002s

FAILED (errors=1)
```



## ② モジュールを作成する

GREEN にするために board.py を修正します。Board クラスの最後に下記を追記します。

```
def count_discs(self):
    black_discs = 0
    white_discs = 0
    for row in self.grid:
        for disc in row:
            if disc == Disc.BLACK:
                black_discs += 1
            elif disc == Disc.WHITE:
                white_discs += 1
    return black_discs, white_discs
```

再度テストを行います。最初のテストも併せて2つのテストに合格したことがわかります。

```
..
-----
Ran 2 tests in 0.000s

OK
```

Test Case 3: 駒における箇所を判定する

オセロのロジックの中で最も複雑なのが駒をおける場所を判定するロジックです。

### ① テストケースを作成する

まずはテストケースを作成します。Board クラスでは（まだ実装していませんが）置く場所を判定するメソッドで `is_valid_move` が存在するとしています。

`test_board.py` (追記部分のみ。 `TestBoard` クラスの最後に追記。)

```
def test_is_valid_move_at_start(self):
    board = Board()
    self.assertTrue(board.is_valid_move(3, 2, Disc.BLACK))
    self.assertFalse(board.is_valid_move(0, 0, Disc.BLACK)) # 置けない場所
    self.assertFalse(board.is_valid_move(3, 3, Disc.BLACK)) # 既に石がある場所
```

`self.assertTrue`、`self.assertFalse` はそれぞれ、引数が True、False になっているかを調べるメソッドです。期待する値ではない場合に `AssertionError` を送出します。

作成したらテストを実行します。下記のようにエラーが発生します。

```
self.assertTrue(board.is_valid_move(3, 2, Disc.BLACK))
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

AttributeError: 'Board' object has no attribute 'is\_valid\_move'

-----

Ran 3 tests in 0.002s

FAILED (errors=1)

## ② モジュールを作成する

```

def is_valid_move(self, x, y, disc_color):
    if not (0 <= x < self.size and 0 <= y < self.size):
        return False

    if self.grid[y][x] != Disc.EMPTY:
        return False

    opponent_color = Disc.WHITE if disc_color == Disc.BLACK else Disc.BLACK
    flippable_discs = []

    # 8 方向を定義
    directions = [(0, 1), (1, 1), (1, 0), (1, -1), (0, -1), (-1, -1), (-1, 0), (-1, 1)]

    for dx, dy in directions:
        line = []
        nx, ny = x + dx, y + dy
        while 0 <= nx < self.size and 0 <= ny < self.size:
            if self.grid[ny][nx] == opponent_color:
                line.append((nx, ny))
            elif self.grid[ny][nx] == disc_color:
                flippable_discs.extend(line)
                break
            else: # 空白または盤外
                break
            nx, ny = nx + dx, ny + dy

    return len(flippable_discs) > 0

```

上記のコードを入力後、再度テストを実行します。テストにパスしたことが分かります。

```
Ran 3 tests in 0.000s
```

```
OK
```

## ③ リファクタリングする

テスト駆動開発では GREEN なコードに対してリファクタリングを行いコードの品質を向上させます。今回作成したメソッド `is_valid_move` は非常に長く、今後コードを保守するのが困難です。

リファクタリングでよく行われる行為がメソッドの抽出 (**Extract Method**) です。複雑だったり行数が多いコードはそれだけで解釈に時間がかかります。

メソッドの切り出しは基本的にコピー＆ペーストで行います。先ほどのメソッドの中を見て、論理的に分割できそうなところを探します。

```
def is_valid_move(self, x, y, disc_color):
    if not (0 <= x < self.size and 0 <= y < self.size):
        return False
    if self.grid[y][x] != Disc.EMPTY:
        return False
```

駒が置けるかチェック

```

    opponent_color = Disc.WHITE if disc_color == Disc.BLACK else Disc.BLACK
    flippable_discs = []
    # 8 方向を定義
    directions = [(0, 1), (1, 1), (1, 0), (1, -1), (0, -1), (-1, -1), (-1, 0), (-1, 1)]

    for dx, dy in directions:
```

特定の方向に駒が置けるかチェック

```

        line = []
        nx, ny = x + dx, y + dy
        while 0 <= nx < self.size and 0 <= ny < self.size:
            if self.grid[ny][nx] == opponent_color:
                line.append((nx, ny))
            elif self.grid[ny][nx] == disc_color:
                flippable_discs.extend(line)
                break
            else: # 空白または盤外
                break
            nx, ny = nx + dx, ny + dy

    return len(flippable_discs) > 0
```

メソッドの切り出しを行う場所は、ほとんどの場合「メソッド内で定義された変数の値を求める部分」になります。今回は `is_valid_move` メソッドの中で定義された二つの変数、`flippable_discs` と `line` について求めている部分を切り出します。

まずは赤枠部分を切り出します。下記のようになります。(対応する部分は赤枠で囲んでいます)

```
def is_valid_move(self, x, y, disc_color):
    if not (0 <= x < self.size and 0 <= y < self.size):
        return False
    if self.grid[y][x] != Disc.EMPTY:
        return False
    flippable = self._get_flippable_discs(x, y, disc_color)
    return len(flippable) > 0

def _get_flippable_discs(self, x, y, disc_color):
    opponent_color = Disc.WHITE if disc_color == Disc.BLACK else Disc.BLACK
    flippable_discs = []

    # 8 方向を定義
    directions = [(0, 1), (1, 1), (1, 0), (1, -1), (0, -1), (-1, -1), (-1, 0), (-1, 1)]

    for dx, dy in directions:
        line = []
        nx, ny = x + dx, y + dy
        while 0 <= nx < self.size and 0 <= ny < self.size:
            if self.grid[ny][nx] == opponent_color:
                line.append((nx, ny))
            elif self.grid[ny][nx] == disc_color:
                flippable_discs.extend(line)
                break
            else: # 空白または盤外
                break
            nx, ny = nx + dx, ny + dy
    return flippable_discs
```

切り出したメソッドのメソッド名の先頭に `_` がついています。先頭にアンダースコアがついたメソッドは慣習で「内部で利用するメソッドだから呼び出さないでね」という意味になります。(もともと「メソッドの中で宣言した変数の値を求めるためのメソッド」であり、外部から直接呼ばれるのは好ましくないためこのようにしています)

クラスに変更を加えたら、どんなに小さい変更でも再度テストを実行します。テストにパスしたら次に青枠部分を切り出します。下記のようになります。

```
def is_valid_move(self, x, y, disc_color):
    変更なし

def _get_flippable_discs(self, x, y, disc_color):
    directions = [(0, 1), (1, 1), (1, 0), (1, -1), (0, -1), (-1, -1), (-1, 0), (-1, 1)]
    flippable_discs = []
    for dx, dy in directions:
        line = self._check_direction(x, y, dx, dy, disc_color)
        if line:
            flippable_discs.extend(line)
    return flippable_discs

def _check_direction(self, x, y, dx, dy, disc_color):
    opponent_color = Disc.WHITE if disc_color == Disc.BLACK else Disc.BLACK
    line = []
    nx, ny = x + dx, y + dy
    while 0 <= nx < self.size and 0 <= ny < self.size:
        if self.grid[ny][nx] == opponent_color:
            line.append((nx, ny))
        elif self.grid[ny][nx] == disc_color:
            return line # 挟めているので、裏返せる駒のリストを返す
        else:
            return [] # 空白または盤外
        nx, ny = nx + dx, ny + dy
    return [] # 盤の端まで相手の石しなかった場合
```

再度テストを実行します。うまく通ればリファクタリングは成功です。

## Test Case 4: 駒を反転させる

## ① テストケースを作成する

駒を反転させる際のテストケースを記述します。

```
def test_place_stone(self):
    board = Board()
    # 黒が(3, 2)に置く -> (3, 3)の白が裏返る
    board.place_stone(3, 2, Stone.BLACK)
    self.assertEqual(board.grid[2][3], Stone.BLACK) # 置いた石
    self.assertEqual(board.grid[3][3], Stone.BLACK) # 裏返った石
```

## ② モジュールを作成する

```
def place_stone(self, x, y, stone_color):
    if not self.is_valid_move(x, y, stone_color):
        return False

    flippable = self._get_flippable_stones(x, y, stone_color)
    self.grid[y][x] = stone_color
    for fx, fy in flippable:
        self.grid[fy][fx] = stone_color
    return True
```

モジュールを作成したら再度テストを行い、テストをパスしたか確認します。

Test Case 5: 駒をおける場所をすべてリストアップする

① テストケースを作成する

```
def test_get_valid_moves(self):  
    board = Board()  
    moves = board.get_valid_moves(Stone.BLACK)  
    self.assertEqual(moves, [(2, 3), (3, 2), (4, 5), (5, 4)])
```

② モジュールを作成する

```
def get_valid_moves(self, stone_color):  
    moves = []  
    for y in range(self.size):  
        for x in range(self.size):  
            if self.is_valid_move(x, y, stone_color):  
                moves.append((x, y))  
    return moves
```

以上で Board クラスの作成は終了です。今後の機能追加でも今までに作成したテストクラスは流用できるので、安心して変更を加えることができます。



## 2-4 ゲームの進行を行うクラス 振舞駆動開発

オセロの駒とオセロのボードができました。後はプレイヤーがいればオセロができる、とはならないです。オセロのボードは置かれたオセロの駒を見て、オセロのルールに反しないかを監視していました。しかし、このままではプレイヤーに制限がかかっていません。現実問題、オセロの規則的に置いちゃダメな場所に、プレイヤーは勝手に駒をおけます。さらに連続で同じ色の駒もおけるし、勝敗が付く前に勝ったことにして逃げることもできます。

実際に我々がオセロをやったときにそうならないのは、ゲームの「**進行役**」がいるからです。2連続で駒を置いたら「ダメ!」と相手に言われます。自分も「ダメ」と思います。これは内心にルールを守ってプレイヤーの動きを制限させる「進行役」がいるからと考えられます。

この正義の「進行役」をクラスにします。「進行役」は主に次の事柄に着目します。

- ・ どんなオセロボード (Board 型オブジェクト) を使用するか
- ・ 現在の置くべき駒は白 (Disc.WHITE) か黒 (Disc.Black) か
- ・ 駒が置かれたとき、次に置くべき駒は何か
- ・ 駒が置ける場所があるかどうか
- ・ 白と黒の駒の数を数えてどっちの駒を置いているプレイヤーが勝利したのか

上記の事柄を進行役が持つべき機能要件であると認め、**アジャイルソフトウェア開発手法**で近年着目されている**振舞駆動開発** (behavior driven development; BDD) で開発していきます。振舞駆動開発はビヘイビア駆動開発とも呼ばれ、「そのプログラムの振る舞い」を先に記述して開発を行います。これを**スペックファースト**と言います。

振舞駆動開発の最大の利点は、システムの「振る舞い」に焦点を当て、エンジニア、QA、企画担当者など、チームの誰もが理解できる自然言語で仕様を記述・共有する開発スタイルにあります。

では、実際に開発を行います。

### 2-4-1 BDD の準備

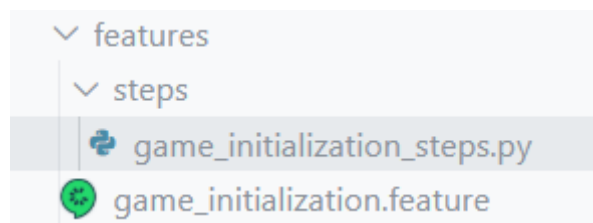
まずは Python で振舞駆動開発を行うのに必要なツールをインストールします。振舞駆動開発をサポートするライブラリは多数ありますが、今回は直感的にわかりやすく使用できる **behave** を使用します。Visual Studio Code のターミナルに下記を入力します。

```
(.venv313) PS C:\Users\oka\Desktop\python_sem> pip install behave
```

次にフォルダの作成を行います。behave というツールでは特定のフォルダ構造を規約とします。基本的には次のような構造です。

```
features/  
├─game_initialization.feature      # ゲーム進行に関する振る舞いを記述  
└─ steps/  
    └─game_initialization_steps.py
```

これに合わせてディレクトリとファイルをプロジェクトの直下に作成します（ファイルの中身はいったん空で作ります）。



Visual Studio Code のプラグインに BDD をサポートするものがあるので、それをインストールします。Visual Studio Code の左にある【拡張機能】のアイコンをクリックして、検索窓に「Cucumber (Gherkin) Full Support」と入力します。



「Cucumber (Gherkin) Full Support」というプラグインが現れるのでそれをインストールします。

インストールが終わると .feature ファイルのアイコンが変化します。

BDD での開発手順はざっくり、シナリオ作成 → テスト作成 → クラス作成です。テスト駆動開発にワンステップ追加された形になります。

シナリオはなるべく小分けにします。複雑なシナリオにするとそのテストをパスするクラスの作成が困難になるためです。シナリオは **Gherkin(ガーキン)記法** で作成します。Gherkin 記法についての詳細はのちに解説します。

では、BDD でオセロの「進行役」を表す Game クラスを作成していきます。

## Feature 1: ゲームの初期化 関数

### ① ふるまいを作成する

ふるまいを定義します。features フォルダに新規で game\_initialization.feature というファイルを作成し、次のふるまいを記述します。(字下げは tab キーで行うのが一般的です。)

features/game\_initialization.feature

**Feature:** ゲームの初期化

**Scenario:** 新しいゲームが開始される

**Given** 新しいゲームオブジェクトを生成する

**Then** ゲーム盤が生成されている

**And** 最初の手番は "黒" である

Gherkin 記法は特定のキーワードと、その内容の記述で構成されます。現れたキーワードについてそれぞれ見ておきます。

- **Feature:**  
機能を完結に書くパートです。主にテストの目的かドキュメントのタイトルを記述します。
- **Scenario:**  
個々のテストケースを書くパートです。一つの **Feature:** に複数の **Scenario:** があることも多いです。

上記の二つが.feature ファイルの根幹的な部分です。次に **Scenario:** の中に書かれたキーワードを見ていきます。

- **Given**  
そのシナリオの前提条件を記述します。ゲーム開始時に「ゲーム」そのものがないのは困るので、今回は **Given** で「新しいゲームオブジェクトを生成する」としています。
- **Then**  
期待される結果を記述します。ゲーム開始と同時にゲーム盤(Board 型オブジェクト)が生成されていることを期待しています。
- **And**  
直前のキーワード（今回は **Then**）に追加で条件を与えます。

ほかにもキーワードが現れますがその都度紹介します（最後にまとめて表にしています）。Gherkin 記法によるシナリオの記述は、非常に直感的にわかりやすいという特徴があります。今回の `game_initialization.feature` は次の日本語と対応します。

まずはゲームの初期化について（**Feature:** ゲームの初期化）、  
新しいゲームが開始されたらどうなってほしいか考える（**Scenario:** 新しいゲームが開始される）。  
まずそもそもゲーム自体が始まっているのが前提なんだけど（**Given** 新しいゲームオブジェクトを生成する）、  
ゲームが始まったのと同時にゲーム盤が生成されて（**Then** ゲーム盤が生成されている）、  
最初の手番は黒の駒からにしたい（**And** 最初の手番は "黒" である）。

## ② テストを作成する

`.feature` ファイルを作成したら、今度は**ステップを実装**します。`.feature` ファイルに基づいてそれをテストする `python` ファイルを作成する工程です。テスト駆動開発ではこのテストファイルの作成から作業が開始していましたが、ふるまいを先に定義することでテストケースの見落としを防ぐことができます。

BDD では `.feature` ファイルを作成した後に、**`behave --dry-run`** を実行してからテストケースを記述した `Python` ファイルを作成するのが定石です。Visual Studio Code のターミナルに **`behave --dry-run`** と入力しましょう。

```

features > game_initialization.feature
1 Feature: ゲームの初期化
2
Scenario: 新しいゲームが開始される
問題 1 出力 デバッグコンソール ターミナル テスト結果 ポート
(.venv313) PS C:\Users\oka\Desktop\python_sem> behave --dry-run
Then ゲーム盤が生成されている # None
And 最初の手番は "黒" である # None

Errored scenarios:
  features/game_initialization.feature:3 新しいゲームが開始される

0 features passed, 0 failed, 1 error, 0 skipped
0 scenarios passed, 0 failed, 1 error, 0 skipped
0 steps passed, 0 failed, 0 skipped, 3 undefined
Took 0min 0.000s

You can implement step definitions for undefined steps with these snippets:

from behave.api.pending_step import StepNotImplementedError
@given(u'新しいゲームオブジェクトを生成する')
def step_impl(context):
    raise StepNotImplementedError(u'Given 新しいゲームオブジェクトを生成する')

@then(u'ゲーム盤が生成されている')
def step_impl(context):
    raise StepNotImplementedError(u'Then ゲーム盤が生成されている')

@then(u'最初の手番は "黒" である')
def step_impl(context):
    raise StepNotImplementedError(u'Then 最初の手番は "黒" である')

(.venv313) PS C:\Users\oka\Desktop\python_sem>

```

長い出力が出ます。**Dry Run**とは**予行演習**という意味で、「もしそのコマンドを実行したら、こんなこと実行するつもりだよ」という情報を得ることができます。Git などにもあるオプションです。

**behave** コマンドを実行すると **.feature** ファイルに定義されたシナリオを逐次テストしていきます。その際にテストケースが記述された Python ファイルを見に行くのですが、そこで定義されていないシナリオの内容があった場合、**StepNotImplementedError** というエラーが発生します。今回の予行演習(Dry Run)でもエラーが表示されているのがわかります(上図)。

実装すべき内容が出てくるので、それをコピペするかそれを見ながらテスト用の **.py** ファイルのひな型を作成します。

*game\_initialization\_steps.py*

```
from behave import given, then
from othello.game import Game
from othello.board import Board
from othello.disc import Disc

@given('新しいゲームオブジェクトを生成する')
def step_impl(context):
    ...

@then('ゲーム盤が生成されている')
def step_impl(context):
    ...

@then('最初の手番は "{color}" である')
def step_impl(context, color):
    ...
```

詳しいテストの内容は Ellipsis で省略しています。

behave を使用する場合、`from behave import given, then` で必要なモジュールをインポートする必要があります。behave では、feature のシナリオの内容を Python のデコレータ機能で実装します。

デコレータとは@から始まる文のことで、次に来るメソッドやクラスや関数に装飾する役割を持ちます。`@given('新しいゲームオブジェクトを生成する')` は直後のメソッドが feature ファイルの `Given 新しいゲームオブジェクトを生成する` に対応していることを表します。

`@then('最初の手番は "{color}" である')` の `"{color}"` はプレースホルダーです。feature ファイルの方で `And 最初の手番は "黒" である` と書いてありましたが、上記のプレースホルダーを利用すると `color` という変数に `"黒"` という値が渡されます。変数 `color` はデコレータ直後の関数の引数にすることで、その関数内で使用できます。

関数名は任意です。なんなら同じ関数名でも構いません。behave の実行はデコレータを見て行うので、関数名には何の規約もないです。今回はすべてに `step_impl` しました。実際は、可読性と保守性の観点から関数名はそれぞれ異なるものにする方が無難です。

**関数**とは一連の処理の流れを独立して定義するものです。よく似たものにメソッドがありますが両者は、**クラス内部で定義されればメソッド、クラス外部で定義されれば関数**と呼ばれます。

オブジェクト指向プログラミングでは組み込み関数（Python が標準で提供してくれている関数、`print` など）とライブラリで指定された関数（今回の `step_impl` など）以外で関数を使う機会は少ないです※。

最後に各関数の引数である `context` についてです。`context` は日本語で「文脈」という意味ですが、ソフトウェア開発の世界では「**現状で関心のあるモノを扱うのに必要なデータ**」を表します。今回でいえば「テスト」を行うわけですが、`context` にはテストに必要なモノを任意に保存できます。

また、`context` に保存したデータは別のステップでも取り出すことができます。データの保存は `context.【識別名】 = 【保存したいデータ】` という形式で行います。

※ メソッドと関数の大きな違いは、以下の部分です。

**メソッド … オブジェクトの状態に依存した処理を実行するため、同一の入力でも出力が変わる**

**関数 … 状態を持たないため、同じ入力にはいつでも同じ出力となる**

同じ「人間」に「挨拶する」という動作をさせることを考えます。人間のオブジェクト（私やあなた）によって挨拶の内容が変化すると思います。これを簡単なコードで書くなら、

```
me = Human("私")
you = Human("あなた")

me.greet() == you.greet() … 出力結果 False
```

という感じです(あくまでイメージです)。関数を用いる場合は `greet` 関数が同じ入力に対していつでも同じ出力になるように設計します。

イメージでは

```
g1 = greet("私")
g2 = greet("あなた")

g1 == g2 … 出力結果 False
```

みたいな感じです。オブジェクト指向の時は「オブジェクトが違うから同じメソッドでも結果が違うのは当たり前だね」というニュアンスです。対して、関数を使用したほうは「`greet` 関数の入力が違う（「私」と「あなた」）から結果が違うのは当たり前だね」というニュアンスになります。

このように状態を持たない関数が主役のプログラミングパラダイムを**関数型プログラミング**と言います。オブジェクト指向プログラミングが「現実のモノをオブジェクトとして扱う」のに対して、関数型プログラミングは「データの推移を関数で表現する」ものです。よくオブジェクト指向プログラミングと対を成すみたいな言い方がされますが、目的によって使い分けるのが普通です。



では、テストファイルを完成させます。

*game\_initialization\_steps.py*

```
from behave import given, then
from othello.game import Game
from othello.board import Board
from othello.disc import Disc

@given('新しいゲームオブジェクトを生成する')
def step_impl(context):
    context.game = Game()

@then('ゲーム盤が生成されている')
def step_impl(context):
    assert isinstance(context.game.board, Board)

@then('最初の手番は "{color}" である')
def step_impl(context, color):
    expected_color = Disc.BLACK if color == "黒" else Disc.WHITE
    assert context.game.current_player == expected_color
```

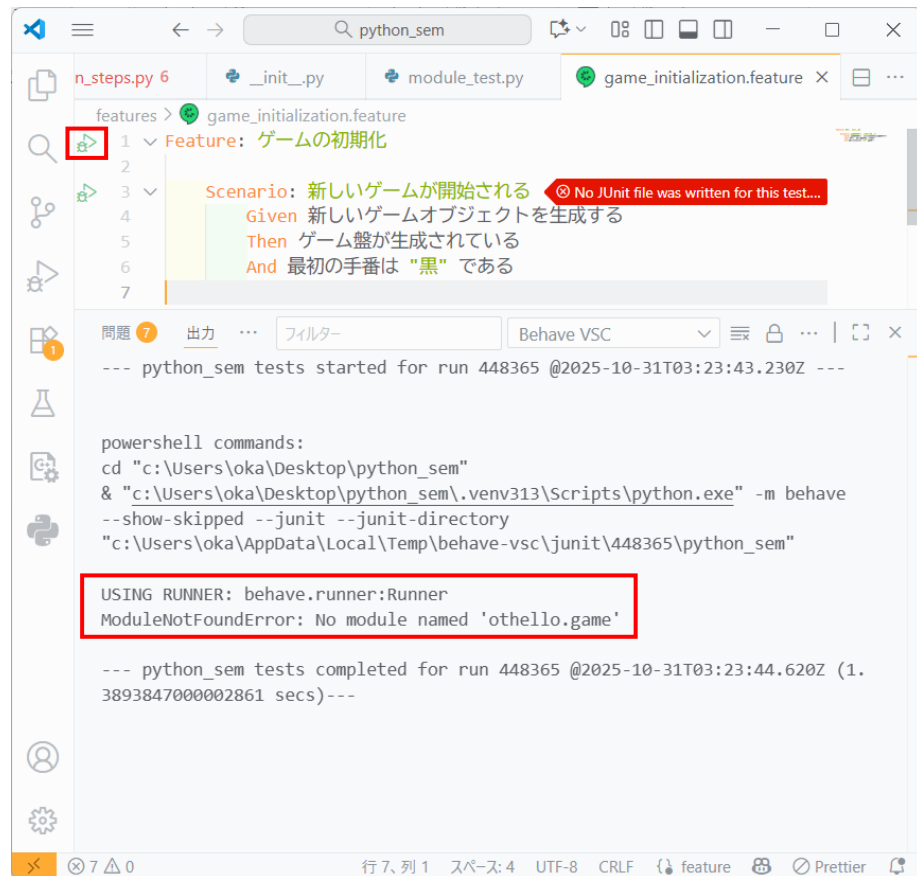
最初のステップで `context.game = Game()` という文を書いています。今後のステップでも `context.game` と書けばここで生成した `Game` 型オブジェクトを使用できます。

次のステップでは `assert isinstance(context.game.board, Board)` と書いています。assert 文では直後の真理値が `False` だと例外を送出します。今回は、最初のステップで作ったゲームのボード (`context.game.board`) がちゃんと `Board` 型オブジェクトなのかを調べるテストになっています。

次のステップで `expected_color = Disc.BLACK if color == "黒" else Disc.WHITE` という文がありますが、これは `feature` ファイルの方で作成した文章とプログラムの整合性を保つための一文です。`feature` ファイルでは 黒 という言葉で黒い駒を指していましたが、現在作成するプログラムでは `Disc.BLACK` で黒い駒を指します。

次の `assert context.game.current_player == expected_color` で現在の手番（実質ゲーム開始時の手番）と期待する駒の色が同じかをテストしています。

テスト用の Python ファイルが作成できたらテストを実行します。Visual Studio Code で game\_initialization.feature を開いて、エディタの左側に現れる【○】をクリックします（下図では【▷】になっていますがアイコンの場所は同じです）。



テスト結果が出力されました。今回は ModuleNotFoundError でテストが失敗しました。othello.game というモジュールを作成していないことが原因です。ふるまいに従ってテストファイルの作成まで済んだら、RED 状態を GREEN にするためにモジュールを作成していきます。

## ③ モジュールを作成する

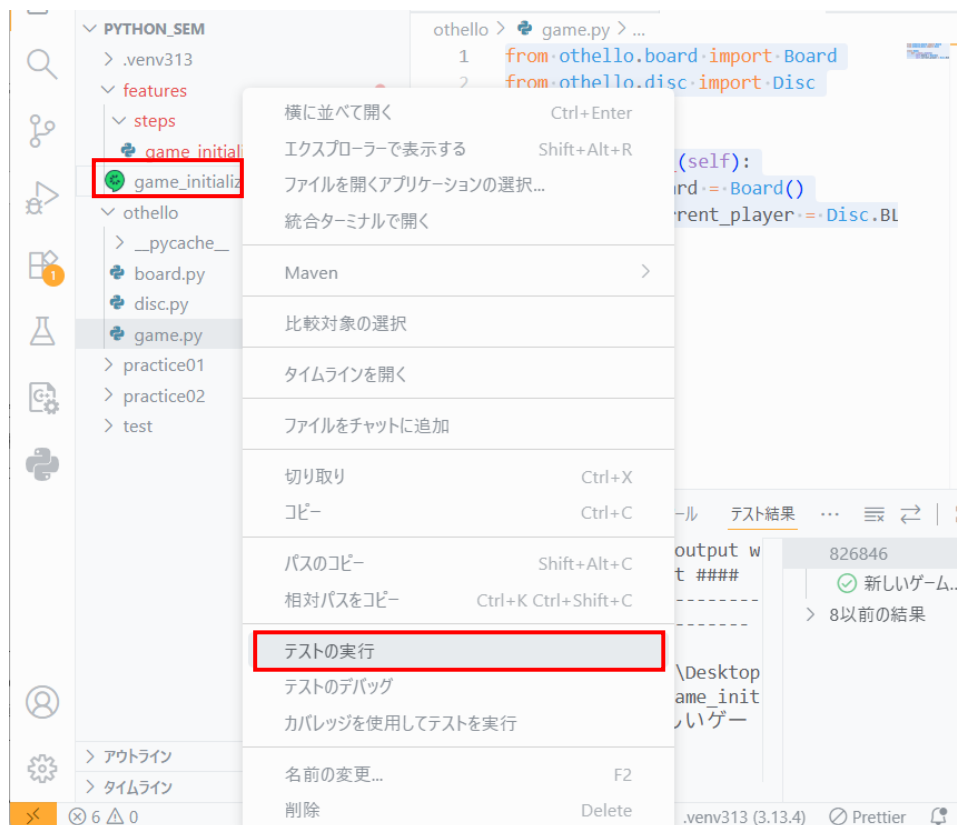
モジュールを作成します。othello フォルダの下に game.py を作成し下記のコードを入力します。

othello/game.py

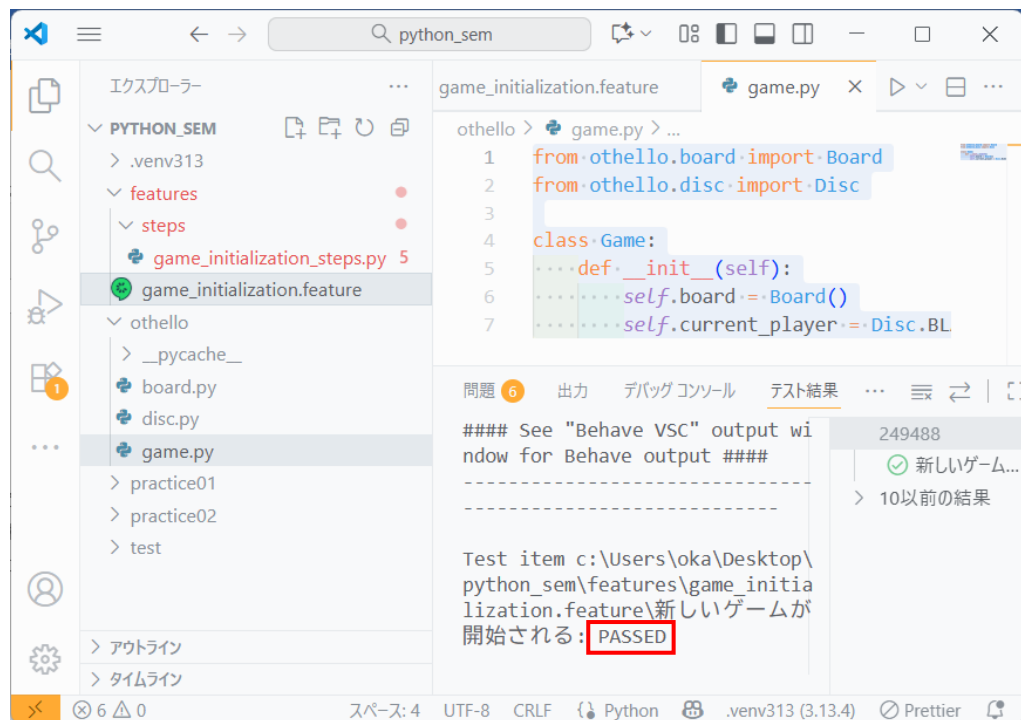
```
from othello.board import Board
from othello.disc import Disc

class Game(object):
    def __init__(self):
        self.board = Board()
        self.current_player = Disc.BLACK
```

入力したらテストを実行します。テストの実行はまた feature ファイルを開いてやってもいいのですが、より簡単に game\_initialization.feature を右クリックして【テストの実行】をクリックします。これで game.py を開いたまま手軽にテストできます。



実行して、「PASSED」の表記が見えたらテストに通ったことになります。



リファクタリングは後回しにして、残りの機能を実装していきます。

## Feature 2: ターン進行

次にターン進行機能をふるまいから実装していきます。

### ① ふるまいを作成する

features フォルダの下に turn\_progression.feature というファイルを作成し下記を入力します。

turn\_progression.feature

Feature: ターン進行

Scenario: プレイヤーが有効な手を打つ

Given 新しいゲームが開始されている

When 黒プレイヤーが (3, 2) に石を置く

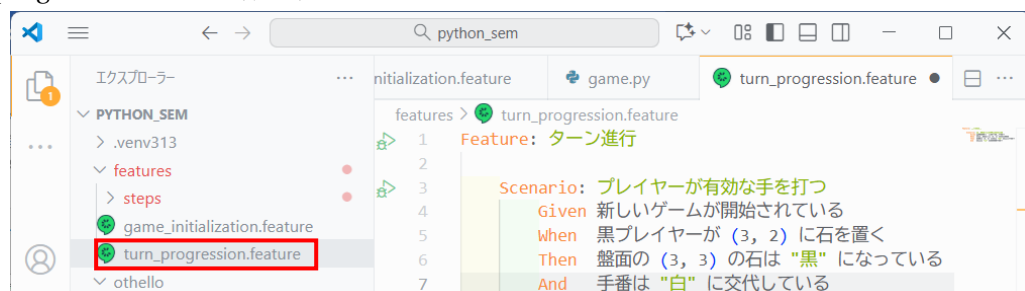
Then 盤面の (3, 3) の石は "黒" になっている

And 手番は "白" に交代している

新たに **When** というキーワードが出てきました。**When** は「ユーザやシステムが何を行うか」を記述するキーワードです。今回の feature ファイルの内容は、下記の文言を Gherkin 記法で書いたものになります。

ターンの進行について (**Feature: ターン進行**)、  
プレイヤーが有効な手を打つときを考える (**Scenario: プレイヤーが有効な手を打つ**)。  
当然ゲーム自体が始まっているのが前提なんだけど (**Given** 新しいゲームが開始されている)、  
まず、黒のプレイヤーが (3, 2) に駒を置いたとき、(**When** 黒プレイヤーが (3, 2) に石を置く)、  
(3, 3)の白い駒が黒にひっくり返ってほしい (**Then** 盤面の (3, 3) の石は "黒" になっている)  
さらに手番は白に交代してほしいね (**And** 手番は "白" に後退している)。

turn\_progression.feature 作成後の図は下記のとおりです。



## ② テストを作成する

turn\_progression.feature に基づいてテスト用の Python ファイルを作成します。features/steps フォルダの下に turn\_progression\_steps.py を作成し、下記のひな型を入力します (behave --dry-run の出力をコピーしても構いません)。

turn\_progression\_steps.py

```
from behave import given, when, then
from othello.game import Game
from othello.disc import Disc

@given('新しいゲームが開始されている')
def step_impl(context):
    ...

@when('{color}プレイヤーが ({x:d}, {y:d}) に石を置く')
def step_impl(context, color, x, y):
    ...

@then('盤面の ({x:d}, {y:d}) の石は "{color}" になっている')
def step_impl(context, x, y, color):
    ...

@then('手番は "{color}" に交代している')
def step_impl(context, color):
    ...
```

`{x:d}` についてプレースホルダーが受け取るデータは基本的に文字列オブジェクトですが、**変換指定子**で型変換を行うことができます。整数のデータが送られてくる場合、`:d` という指定子を使えば関数内で整数として利用できます (文字列で送られてきた場合、`int(x)` のように整数型に変換する必要が生まれる)。このほかの指定子には `:f` というものがあり、こちらは小数点数としてデータを扱います。

テスト用の Python ファイルのひな型が作れたら、そのステップに相当するテストの内容を書いていきます。Ellipsis を削除して下記のように編集します。

```

from behave import given, when, then
from othello.game import Game
from othello.disc import Disc

@given('新しいゲームが開始されている')
def step_impl(context):
    context.game = Game()

@when('{color}プレイヤーが ({x:d}, {y:d}) に石を置く')
def step_impl(context, color, x, y):
    context.game.play_turn(x, y)

@then('盤面の ({x:d}, {y:d}) の石は "{color}" になっている')
def step_impl(context, x, y, color):
    expected = Disc.BLACK if color == "黒" else Disc.WHITE
    actual = context.game.board.grid[y][x]
    assert actual == expected

@then('手番は "{color}" に交代している')
def step_impl(context, color):
    expected = Disc.BLACK if color == "黒" else Disc.WHITE
    assert context.game.current_player == expected

```

初めの `context.game = Game()` でゲームを開始します。これ以降のステップでは `context.game` で、ここで開始したゲームを取得できます。

次の `context.game.play_turn(x, y)` で盤面に駒を置いています。この段階ではまだ `play_turn` メソッドは定義していないので、ここでテストに失敗すると予想されます。

`expected = Disc.BLACK if color == "黒" else Disc.WHITE` は、`.feature` ファイルではオセロの黒の駒を“黒”と呼んでおり、プログラムでは `Disc.BLACK` と定義しているので、その整合性を保つための1文です。

`actual = context.game.board.grid[y][x]` は `(x, y)` に置かれている駒を取得しています。

`assert actual == expected` では、期待する駒の色 (`expected`) と実際に置かれている駒の色 (`actual`) が等しいか比較しています。期待する駒じゃなかった場合に例外を送出します。

`assert context.game.current_player == expected` では期待する現在の手番の色 (`expected`) と、実際の現在の手番の色 (`context.game.current_player`) が等しいか比較しています。

以上でテストファイルができました。`turn_progression.feature` でテストを実行してみます。RED であることが確認できます。

### ③ モジュールを作成する

テストをパスするようにクラスを編集します。

```
def switch_player(self):
    self.current_player = Disc.WHITE if self.current_player == Disc.BLACK else Disc.BLACK

def play_turn(self, x, y):
    # Board.place_disc は石を置いて反転させる責務を持つ
    if self.board.place_disc(x, y, self.current_player):
        self.switch_player()
        return True
    return False
```



**Feature 3:        ゲームの終了と勝利判定**

ゲームの終了時の動作と、勝利判定についての振舞を記述します。

**① ふるまいを作成する**

features フォルダの下に `game_termination.feature` というファイルを作成し下記を入力します。今回はゲームの終了と勝者の判定を行うために3つのシナリオを作成しています。各シナリオで登場するキーワードは既出なので詳しい説明は省きます。

*game\_termination.feature*

**Feature: ゲームの終了と勝者判定**

**Scenario: 両者とも手がない場合にゲームが終了する**

**Given** 盤面が埋まり、両者とも有効な手がない

**When** ゲームの終了を判定する

**Then** ゲームは終了しているべきである

**Scenario: 黒の勝利**

**Given** 黒石が白石より多い状態でゲームが終了した

**When** 勝者を判定する

**Then** 勝者は "黒" である

**Scenario: 引き分け**

**Given** 黒石と白石が同数の状態でゲームが終了した

**When** 勝者を判定する

**Then** 勝者はいない（引き分け）

## ② テストを作成する

features/steps フォルダの下に game\_termination\_steps.py という Python ファイルを作成し、先の game\_termination.feature ファイルの内容を反映させます。

```
from behave import given, when, then
from othello.game import Game
from othello.disc import Disc

@given('盤面が埋まり、両者とも有効な手がない')
def step_impl(context):
    context.game = Game()
    # テストのために意図的に盤面を操作
    context.game.board.grid = [[Disc.BLACK for _ in range(8)] for _ in range(8)]

@when('ゲームの終了を判定する')
def step_impl(context):
    context.is_over = context.game.is_game_over()

@then('ゲームは終了しているべきである')
def step_impl(context):
    assert context.is_over is True

@given('{winner_color}駒が他方より多い状態でゲームが終了した')
def step_impl(context, winner_color: str):
    context.game = Game()
    context.game.board.grid = [[Disc.BLACK for _ in range(8)] for _ in range(8)]
    if winner_color == "白":
        # 盤面を白勝ちの状態にする
        context.game.board.grid[0][0] = Disc.WHITE
        context.game.board.grid[0][1] = Disc.WHITE
        context.game.board.grid[0][2] = Disc.WHITE

@when('勝者を判定する')
def step_impl(context):
    context.winner = context.game.get_winner()
```

```
@then('勝者は "{color}" である')
def step_impl(context, color: str):
    expected = Disc.BLACK if color == "黒" else Disc.WHITE
    assert context.winner == expected

@given('黒駒と白駒が同数の状態でゲームが終了した')
def step_impl(context):
    context.game = Game()
    # 盤面を引き分けの状態にする
    for i in range(4):
        for j in range(8):
            context.game.board.grid[i][j] = Disc.BLACK
    for i in range(4, 8):
        for j in range(8):
            context.game.board.grid[i][j] = Disc.WHITE

@then('勝者はいない(引き分け)')
def step_impl(context):
    assert context.winner is None
```

ステップ数が多く長いテストファイルになりましたが、内容自体に新規性はないため詳細は省きます。

作成出来たら一度テストを行います。現状ではテストに失敗します。

## ③ モジュールを作成する

最後にこのテストをパスするモジュールを作成します。

*game.py* の末尾に追記

```
def is_game_over(self):
    # 両プレイヤーに有効な手がないかチェック
    black_moves = self.board.get_valid_moves(Disc.BLACK)
    white_moves = self.board.get_valid_moves(Disc.WHITE)
    return not black_moves and not white_moves

def get_winner(self):
    # Board クラスは石の数を数える責務を持つ
    black_discs, white_discs = self.board.count_discs()
    if black_discs > white_discs:
        return Disc.BLACK
    elif white_discs > black_discs:
        return Disc.WHITE
    else:
        return None # 引き分け
```

再度テストを実行します。テストが通ったことを確認できます。

## 2-4-2 型ヒント

リファクタリングは主に識別名の変更です。しかし Python のような動的型付け言語では「変数が何型オブジェクトを参照するのか」と「メソッド・関数の戻り値は何か」を明示する型ヒントをつけることも多いです。Game クラスに型ヒントをつけていきます。

型ヒントの使い方は以下の通りです。

### 変数の型ヒント

```
変数名: 型
```

### メソッド・関数の型ヒント

```
メソッド名(...) -> 型:
    ...
```

型ヒントをつけておくと IDE によっては警告を出してくれたり、mypy などの外部ツールを使うと実行前にプログラムの解析を行ってくれます。

チームで開発する際や、個人でも規模の大きい開発を行う際には必ずと言っていいほど使います。今回はリファクタリングのタイミングで型ヒントをつけますが、現場によっては最初のテストを通るモジュール作成段階でつけておく場合も多いです。

まずはクラス宣言部から `__init__` メソッドに型ヒントをつけます。追記部分を赤枠で囲っています。

*game.py* のクラス宣言部から `__init__` メソッドまで

```
class Game:
    board: Board
    current_player: Disc

    def __init__(self) -> None
        self.board = Board()
        self.current_player = Disc.BLACK
```

インスタンス変数に型ヒントをつける場合、クラス宣言部の直後に型ヒントを並べます(上部の赤枠)。これによって、変数 `board` は `Board` 型、`current_player` は `Disc` 型であることが分かります。あとからプログラムを見たチームメンバーも、型ヒントがあるクラスではそのおインスタンス変数が何か一目でわかります。

次の `def __init__(self) -> None:` について、`__init__` メソッドの戻り値が何もないことを表しています（`return` 文がないメソッド）。

次の型ヒント追記箇所は下記のとおりです。

*game.py* (一部)

```
def switch_player(self) -> None:
    ...

def play_turn(self, x: int, y: int) -> bool:
    ...
```

メソッドの中身は省略しています。まず `switch_player` メソッドですが、これも `return` 文がなく戻り値がないため `-> None` と書いています。

次の `play_turn` メソッドですが、最終的に `True` か `False` をリターンしているので戻り値は `bool` 型オブジェクトです。よって `-> bool` と書いています。次にメソッドの引数部分について、メソッドの引数でも型ヒントをつけることができます。

自作のメソッドはとくに、そのメソッドの引数の意味が外部から読み取りづらいです。そこで、自分が作るクラスに対してはなるべく型ヒントをつけてあげるようにしましょう。

最後の追記箇所です。

*game.py* (一部)

```
from typing import Optional

...

def is_game_over(self) -> bool:
    ...

def get_winner(self) -> Optional[Disc]:
    ...
```

今回新しく `Optional` というものがあらわれました。`Optional` は「値があるかもしれないし、ない(`None`)かもしれない」変数や関数・メソッドに対して使う型ヒントです。

`get_winner` で勝者が分かりますがそれは、白 (Disc.WHITE) か黒 (Disc.Black) か引き分け (None) のいずれかなので、このメソッドの戻り値の型ヒントは `Optional[Disc]` となります。

このほかにも型ヒントには、複数の型があり得る場合に使う Union や、どんな型でも許容する Any などがあります。ここでは型ヒントについての簡単な紹介にとどめておきます。

以上で Board クラスの作成は終了です。

## 2-5 オセロのプレイヤー 継承・多態性・カプセル化

次にオセロのプレイヤーを開発します。

「プレイヤー」には「人間プレイヤー」と「コンピュータープレイヤー」が存在し、「コンピュータープレイヤー」には「強いコンピュータープレイヤー」と「弱いコンピュータープレイヤー」が存在するという状況です。

振舞駆動開発でクラスを作成していきませんが、今回のように複雑に名詞が関係している場合、そもそも「どんなクラスを作成するか」という考え方が重要になります。そこで、ここから少し脱線して（ようやく）オブジェクト指向の三大要素の一つである**継承**を利用していきます。

### 2-5-1 プレイヤークラス 継承

広辞苑開いて出てくる名詞は全てクラスにすることができます。これはクラスをどのように作成するか（クラス設計）と、各名詞間の言語学的な関係が非常に密接な関係にあることを意味します。

オブジェクト指向が表現できる言語学的な関係として主に下記の二つがあります。

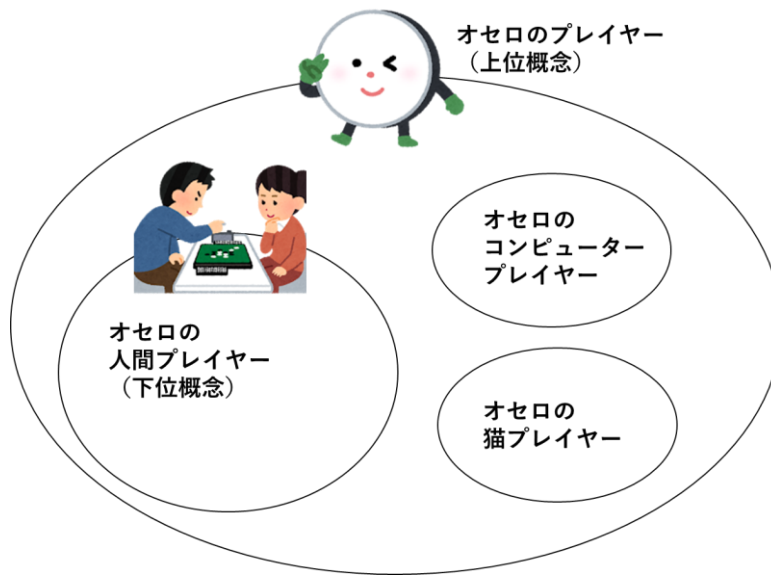
- ・ **上位概念と下位概念（Hyponymy, Hypernymy）**
- ・ **部分と全体（Meronymy and Holonymy）**

ややこしそうに見えますが、これらは非常にシンプルで直感的にわかりやすいです。まずは上位概念と下位概念の考え方からマスターします。

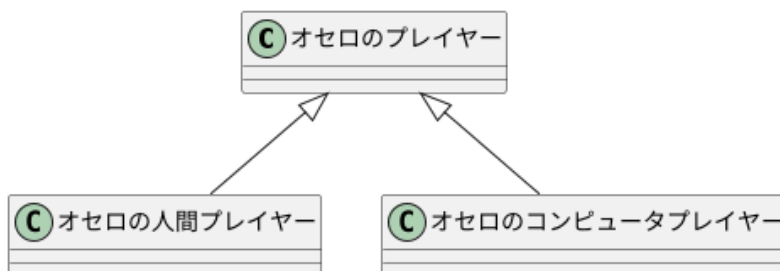
今回、「オセロの人間プレイヤー」と「オセロのコンピュータープレイヤー」をこれから作成するつもりですが、このように直感的に似た概念をクラスにするとき、共通部分を抽出してできる概念が**上位概念（上位語）**です。ハイパニミー（Hyponymy）とも言います。今回作ろうとする二つの概念から共通部分が「オセロのプレイヤー」になりますが、これが上位概念にあたります。

逆に「オセロのプレイヤー」から見て「オセロの人間プレイヤー」は**下位概念（下位語）**と言えます。ハイボニミー（Hyponymy）とも言います。





クラスの間にある上位語 ⇔ 下位語の関係は**クラス図**と呼ばれる図で分かりやすく表現できます。下位概念から上位概念に向かって先が△の矢印を伸ばします。



上位・下位の関係にあるクラス間の関係は、オブジェクト指向プログラミングの世界では**is-a(イズ ア)の関係**にあると表現されます。これは上位概念・下位概念のクラスには

下位概念のクラスは、(一種の) 上位概念のクラスである

という文法が矛盾なく自然に成立するためです。今回のプレイヤークラスで言えば、

人間のオセロプレイヤーは、(一種の) オセロプレイヤーである

といった具合です。オブジェクト指向プログラミングの言葉では上位概念のクラスを**上位クラス**、**基底クラス**、**親クラス**、**スーパークラス**などといい、下位概念のクラスを**下位クラス**、**派生クラス**、**子クラス**、**サブクラス**といいます。

これを Python で表現する場合、次のように書きます。

```
class 下位クラス(上位クラス):
    ...
```

**上位クラス**の部分は今まで無条件に **object** と書いていた部分ですね。Python では「モノ・概念」「名詞」そのものを表すクラスに **object** クラスというがあります。理論上の最上位概念で、あらゆるクラスと is-a の関係にあります。

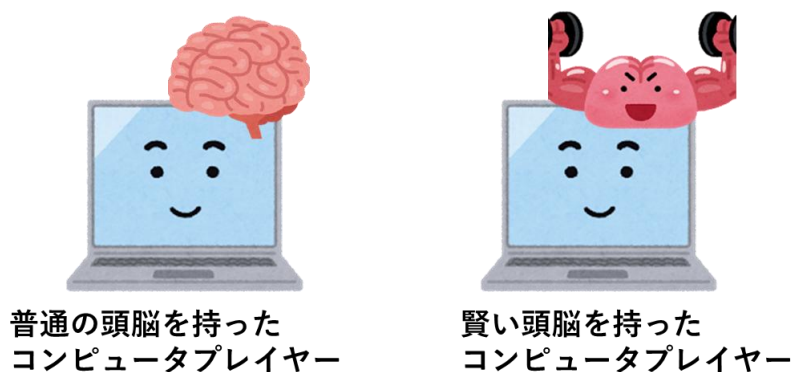
オセロの駒は、(一種の) モノ・概念である  
プレイヤーも色も空も猫も杓子も、(一種の) モノ・概念である

親クラス・子クラスという言い方を採用するなら、オブジェクトクラスは全てのクラスの祖先クラスといった感じです。もしクラス名のあとのカッコを省略した場合、自動でオブジェクトクラスの下位クラスとなります。

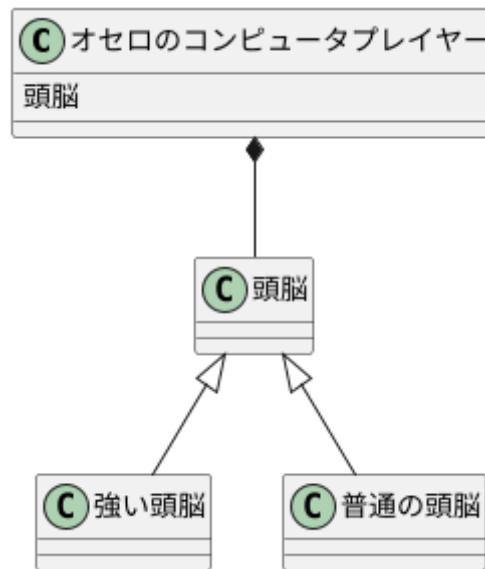
```
class クラス名: ← カッコを省略した場合、自動で (object) が挿入される
    ...
```

(基本的には「どんなクラスも object クラスの下位クラスである」ことは明白なので、わざわざ(object)を書くことはほとんどありません。)

もう一つの**部分と全体 (Meronymy and Holonymy)**についてもさらっておきます。これはオブジェクト指向の世界では has-a の関係と呼ばれ、簡単に言えば「あるオブジェクトがあるオブジェクトを所有している」という関係になります。下図は「コンピュータープレイヤー has a 頭脳」のイメージです。

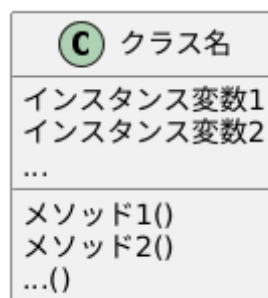


has-a の関係にあるクラスをクラス図で表現するとき、先が◇の矢印でクラス間をつなぎます※。「強い頭脳」も「普通の頭脳」も「頭脳」の下位概念であることが明白なので、先の図は下図のようなクラス図として表現できます。



※ 先の◇が黒く塗られている場合と白抜きされている場合で意味が異なります。黒塗りされている方が強い has-a の関係で **コンポジション（合成）** と呼ばれます。コンポジションにある has-a の関係の例としては、「家」has a 「部屋」、「本」has a 「ページ」、「会社」has a 「部署」などがあげられます。これらは**所有者から独立できない**という特徴があります。「家」が消えれば「部屋」も消える、といった具合です。これは**ライフサイクルをともにする**とも言います。白抜きの◇の場合は**アグリゲーション（集約）**と呼ばれず。例としては、「図書館」has a 「本」、「学校」has a 「生徒」、「会社」has a 「従業員」などです。これらは**所有者から独立している**という特徴があります。「図書館」が消えても「本」は消えません。これは**ライフサイクルが別である**とも言います。

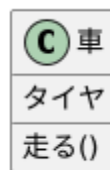
クラス図ではそのクラスが持つデータとふるまいを書くことができます。今までの図では省略してきましたが、実際にクラス図を書く際には省略せずにしっかりと書きます。



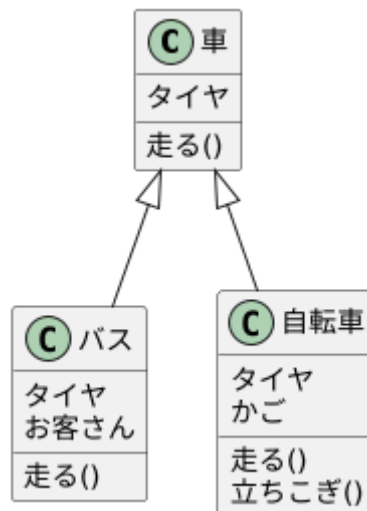
## 2-5-2 オーバーライドと多態性

継承は、名詞が持つ上位概念・下位概念の関係をプログラムに落とし込んだものです。もう少しこの上位概念と下位概念について深堀します。

例として、上位概念の「車」と下位概念の「バス」「自転車」について考えます。「車」には「タイヤ」がありふるまいとして「走る」ことができます。これをクラス図にすると下図のようになります。



車の一種である「バス」や「自転車」も、「タイヤ」を持ち「走る」ことができます。上位概念と下位概念の重要な関係に「上位概念が持つものは下位概念も持ち」、「上位概念ができることは下位概念もできる」というものがあります。一般に逆は成り立ちません。

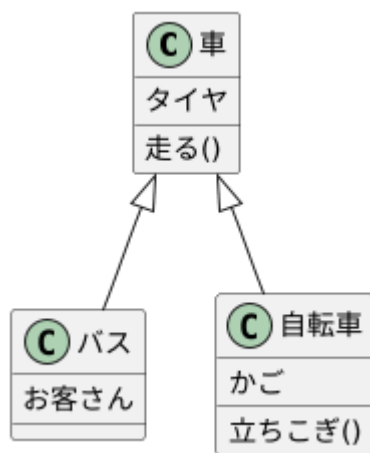


「自転車」は「立ちこぎ」できますが、「車」がすべて「立ちこぎ」できるわけではありません。一般化すればするほど（上位概念に向かうほど）できることは少なくなり、特殊化すればするほど（下位概念に向かうほど）できることが増えます。

クラス図では下位概念から上位概念に矢印が伸びますが、この矢印は汎化 (generalization) と呼ばれており、クラスの概念を一般化するという意味を持ちます。

クラスが持つインスタンス変数とメソッドを**資産**と言います。クラスの継承を行うと、「上位概念が持つものは下位概念も持ち」、「上位概念ができることは下位概念もできる」という特徴を反映して、上位クラスの資産が下位クラスに継承されます。

つまり、先ほどのクラス図では「タイヤ」と「走る」を「車」空継承することになるので、下図のように書いても構わないことになります。



自転車やバスは、車から資産（「タイヤ」と「走る」）を継承するので、クラス図に書かなくても持っていることがわかります。

このように定義したクラスで自転車型インスタンスに走るメソッドを呼び出すことが可能です。

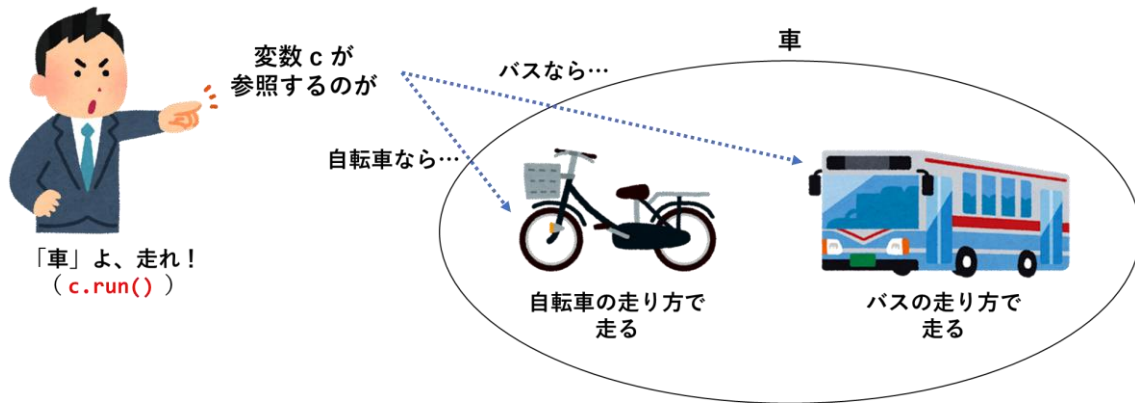
「バス」も「自転車」も「車」なので「走る」ことができます。

しかし、**走り方**はバスと自転車で明らかに異なります。このような場合、上位クラスが持つメソッドを下位クラスで再定義して、ふるまいの内容を変更することができます。これをメソッドの**オーバーライド**と言います。

適切にメソッドをオーバーライドすることで、車型オブジェクトを格納した変数 `c:Car` について、`c.run()` のようなメソッド呼び出しを行うと、その車がバスか自転車かで異なるふるまいをしてくれます。

このように、同じ命令でも参照するオブジェクトによって異なるふるまいをすることを**多態性（ポリモーフィズム）**と言います。多態性の実現には**動的結合（命令されたときのふるまいをプログラムの実行時に決める）**という技術が使われています。

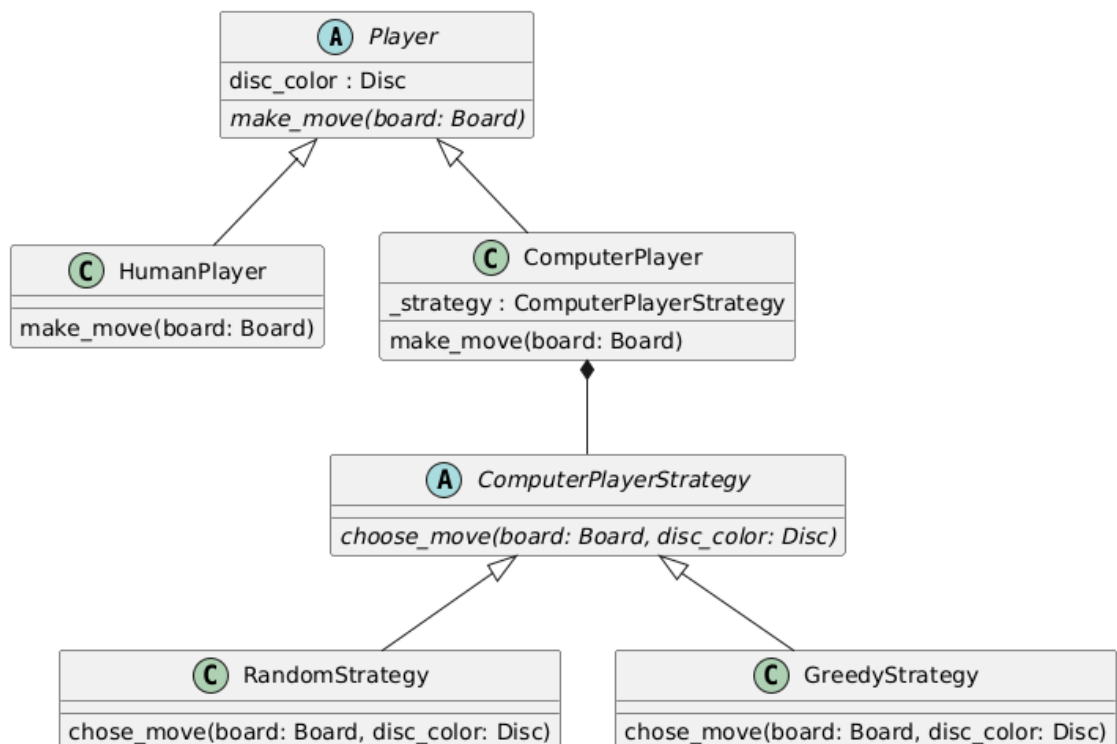
オブジェクト指向の三大要素の二つ目が多態性です。



同じ `c.run()` という命令でも参照するオブジェクトによって動きが変わる

↓  
多態性 (ポリモーフィズム)

それでは実際にプレイヤークラスを作成していきます。内部設計書でクラス図ができている状態から、振舞駆動開発で各クラスのふるまい（メソッド）を実装するという手法が実践的ですが、今回はオブジェクト指向の内容を学習しながら進みたいので、クラス図から一つずつクラスを完成させるという手法を取ります。下記が Player 関連クラスのクラス図です（一部簡略化しています）。



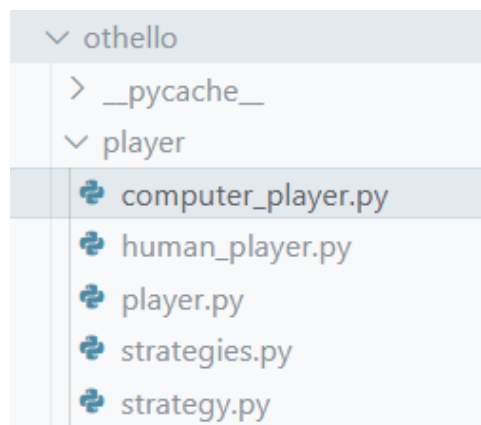
いくつか重要な部分を抜粋して説明します。HumanPlayer クラスと ComputerPlayer クラスは Player クラスから派生しています。Player クラスは自分の駒の色(disc\_color)を持ち、ボードに駒を置く (make\_move()) ことができます。

同じ「プレイヤーがボードに駒を置く」という動作でも、コンピュータと人間で異なるはずなので、HumanPlayer クラスと ComputerPlayer クラスで make\_move() をオーバーライドしています。

またコンピュータは強さによって異なる「戦略」を持ちます。ComputerPlayer が持つ戦略を ComputerPlayerStrategy クラスとして、その下位クラスに「強い戦略」と「弱い戦略」を表す GreedyStrategy クラスと RandomStrategy クラスを作成しています。

「戦略」は駒を置く (make\_move()) ときに、どこに駒を置くべきか (chosed\_move()) を考えます。考え方は戦略により異なるので、どこに駒を置くべきか (chosed\_move()) は下位クラスごとにオーバーライドしています。

クラス図に従って、各クラスのひな型を書いていきます。まずは othello フォルダの下に Player 関連クラスを入れるための player フォルダを作成し各種 python ファイルを作成します。



strategies.py には RandomStrategy と GreedyStrategy の二つのクラスを定義する予定です。

もし振舞駆動開発やテスト駆動開発で開発を進める場合、クラス図から簡単にクラスのひな型を作ってから、各クラスの振舞を定義していくという流れになります（別紙参照）。

### 2-5-3 Player クラス作成 抽象クラス

Python ファイルが作成できたら、具体的にクラスを作成していきます。Player クラスから始めます。

```
class Player:
    """
    プレイヤーの基底クラス
    """
    def __init__(self, disc_color):
        self.disc_color = disc_color

    def make_move(self, board):
        """次に出す手を決める
        """
        Pass
```

Player クラス自体はこれで終わったように見えますが、もう少しだけ手を加えます。今回のオセロゲームには「人間プレイヤー」と「コンピュータープレイヤー」は登場しますが、「プレイヤー」という漠然とした存在は登場しません。つまり **Player クラスは直接インスタンス生成されると困るクラス**です。

また `make_move(self, board)` メソッドについて、このメソッドの中身を `pass` 文で「何もしない」としてはいますが、より正確には「**下位クラスがこのメソッドをオーバーライドして具体的な内容を書く**」としたいところです。

こんな時に便利なのが**抽象クラス**と呼ばれるクラスの書き方です。**抽象クラスにしたクラスはインスタンス生成できません**。また抽象クラスには**抽象メソッド**と呼ばれるメソッドを定義することができ、**抽象メソッドはサブクラスでの実装を強制**できます。

Python で抽象クラスを利用する場合、`abc` モジュールをインポートする必要があります。`abc` は Abstract Base Class (抽象基底クラス) の略です。では Player クラスを抽象クラスに変更します。下記のように追記してください。



```
from abc import ABC, abstractmethod
```

```
class Player(ABC):
```

```
    """
```

```
    プレイヤーの抽象基底クラス
```

```
    """
```

```
    def __init__(self, disc_color):
```

```
        self.disc_color = disc_color
```

```
    @abstractmethod
```

```
    def make_move(self, board):
```

```
        """次に出す手を決める
```

```
        """
```

```
        pass
```

抽象クラスにするには abc モジュールの中の ABC クラスを継承します。これによってそのクラスは抽象クラスとなります（インスタンス生成できなくなります）。

抽象メソッドには `@abstractmethod` というデコレータをつけます。このデコレータが付いたメソッドはある意味で不完全なメソッドとなり、「サブクラスで実装するまで使えないよ」という意味になってくれます。また、サブクラスで実装しなかった場合、そのサブクラスも `@abstractmethod` な資産を継承して抽象クラスとなります。

## 2-5-4 HumanPlayer, ComputerPlayer の作成 カプセル化

次に HumanPlayer クラスを作成します。抽象メソッド `make_move` をオーバーライドしますが、具体的な処理はユーザからの入力によるので結局ここも空になります。

```
from .player import Player

class HumanPlayer(Player):
    """
    人間のプレイヤーを表すクラス
    """

    def __init__(self, disc_color):
        super().__init__(disc_color)

    def make_move(self, board):
        """
        人間の手はユーザ入力によって処理されるため、
        このメソッドでは何もしない
        """
        pass
```

`__init__`メソッドの中の処理で、`super().__init__(disc_color)` という文があります。まず `super()` はスーパークラスへの参照を表す部分です。

次に ComputerPlayer クラスを作成します。ComputerPlayer は自身の持つ戦略に基づいて、どこに駒を置くか (`make_move(self, board)`) を決めます。

```
from .player import Player

class ComputerPlayer(Player):
    """
    コンピュータプレイヤーを管理するクラス
    """

    def __init__(self, strategy, disc_color):
        super().__init__(disc_color)
        self.strategy = strategy

    def make_move(self, board):
        """
        設定された戦略に基づいて次の手を決定する
        """
        return self.strategy.choose_move(board, self.disc_color)
```

`__init__(self, strategy, disc_color)` でコンピュータの駒の色と戦略を初期化しています。

`make_move(self, board)` メソッドでは戦略に基づいて置く場所が決まるように、`return self.strategy.choose_move(board, self.disc_color)` としています。

まだ `strategy.choose_move(board, self.disc_color)` の内容は定義していませんが、予定としてはボードの状況 (`board`) と自分の駒の色 (`self.disc_color`) から、自分の戦略 (`self.strategy`) に応じて駒を置く場所を決めるイメージです。

これで ComputerPlayer クラスは完成しましたが、オブジェクト指向の三大要素の最後の一つであるカプセル化について触れておきます。カプセル化とは、オブジェクトが持つインスタンス変数に外部から直接参照させない仕組みのことです。データの隠蔽などとも言われることもあります。

では ComputerPlayer クラスのインスタンス変数である strategy をカプセル化してみます。すぐ後にわかりますが、データの隠蔽はできていません。

```
from .player import Player

class ComputerPlayer(Player):
    """
    コンピュータプレイヤーを管理するクラス
    """

    def __init__(self, strategy, disc_color):
        super().__init__(disc_color)
        self._strategy = strategy

    def make_move(self, board):
        """
        設定された戦略に基づいて次の手を決定する
        """
        return self._strategy.choose_move(board, self.disc_color)
```

インスタンス変数 strategy の先頭にアンダースコア `_` を付けました。Python ではサブクラス以外からいじられたくない変数の先頭にアンダースコアをつける風習があります。しかしこれはあくまで風習であり、普通に外部からアクセスできます。

アンダースコアを先頭に一つつけた変数は、「サブクラス以外から直接いじらないでね」という意味になります。とはいえ、まったく使用できないのでは無意味なので、カプセル化（隠蔽）されたデータへの**アクセッサ**を定義します。

アクセッサには2種類あり、隠蔽されたデータの取得を行うのが**ゲッター**、隠蔽された変数にデータを入れるのが**セッター**です。

アクセッサを追加した ComputerPlayer クラスは次の通りです。

```
from .player import Player

class ComputerPlayer(Player):
    """
    コンピュータプレイヤーを管理するクラス
    """

    def __init__(self, strategy, disc_color):
        super().__init__(disc_color)
        self._strategy = strategy

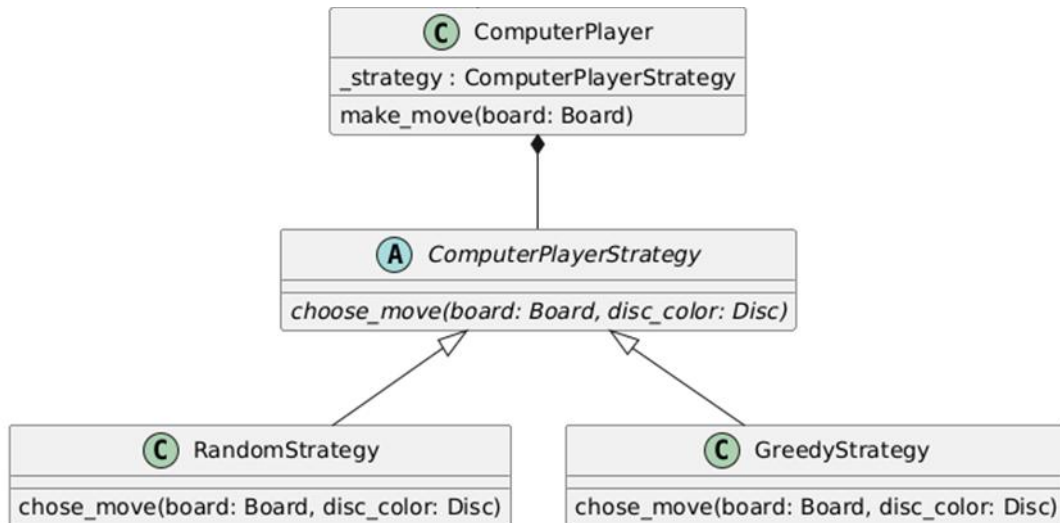
    @property
    def strategy(self):
        return self._strategy

    @strategy.setter
    def strategy(self, strategy):
        self._strategy = strategy

    def make_move(self, board):
        """
        設定された戦略に基づいて次の手を決定する
        """
        return self._strategy.choose_move(board, self.disc_color)
```

## 2-5-5 strategy 関連クラスの作成 Strategy パターン

次に strategy 関連クラスの作成を行います。再度クラス図を見てみます。



ComputerPlayer はどこに駒を置く (make\_move メソッド) かを戦略によって変えます。「戦略」とはさらにかみ砕けば「駒をどこに置くかを決めるアルゴリズム」のことです。

今回の例のように、同じ抽象クラスを実装する交換可能な「アルゴリズム」を複数定義して、プログラムの実行時に適切なアルゴリズムを選択するクラス設計のやり方を **Strategy パターン**と言います。Strategy パターンはデザインパターン※の一つです。

※ デザインパターン … ソフトウェア開発における「よくある問題に対する再利用可能な解決テンプレート」のことです。簡単に言えば「うまくいった設計のアイディア集」です。有名なものに **GoF (Gang of Four) の 23 のデザインパターン**があります。

Strategy パターンで開発すると、後から「超強い戦略」を持ったコンピュータを作成する必要が出てきたときも ComputerPlayerStrategy を継承して chose\_move メソッドをオーバーライドするだけで済みます。

では開発していきます。まずは strategy.py の中で ComputerPlayerStrategy クラスを作成します。この時点では思考ロジックは抽象的なものなので、抽象クラスとして作成します。

```

from abc import ABC, abstractmethod

class ComputerPlayerStrategy(ABC):
    """
    コンピュータプレイヤーの思考ロジックの抽象基底クラス (Strategy)
    """

    @abstractmethod
    def choose_move(self, board, disc_color):
        """
        利用可能な手の中から次の手を選択する

        :param board: 現在のゲーム盤 (Board)
        :param disc_color: コンピュータの石の色 (disc)
        :return: (x, y) タプルで表される選択した手
        """
        pass

```

ドキュメントを少し正確に書きましたが、内容自体は抽象メソッドが一つあるだけの抽象クラスです。

次にランダムに駒を置く戦略である RandomStrategy クラスを作成します。strategies.py を編集します。

*strategies.py*

```

import random
from .strategy import ComputerPlayerStrategy

class RandomStrategy(ComputerPlayerStrategy):
    """
    ランダムに手を選択する戦略
    """

    def choose_move(self, board, disc_color):
        valid_moves = board.get_valid_moves(disc_color)
        if not valid_moves:
            return None
        return random.choice(valid_moves)

```

`random.choice(valid_moves)` は駒を置ける場所のリスト (`valid_moves`) からランダムに一つ選択するという意味です。

次に最も多くの石を裏返せる手を打つ戦略である `GreedyStrategy` クラスを作成します。

*strategies.py* に追記

```
class GreedyStrategy(ComputerPlayerStrategy):
    """
    最も多くの石を裏返せる手を選択する戦略
    """
    def choose_move(self, board, stone_color):
        valid_moves = board.get_valid_moves(stone_color)
        if not valid_moves:
            return None

        # 各有効な手について、裏返せる石の数を計算
        move_scores = {
            move: board.get_num_flippable_stones(move[0], move[1], stone_color)
            for move in valid_moves
        }

        # 最もスコアの高い手(複数ある場合はその中の一つ)を返す
        max_score = max(move_scores.values())
        best_moves = [move for move, score in move_scores.items() if score == max_score]

        return random.choice(best_moves)
```

以上で strategy 関連クラスの作成は終了です。しかし将来的なことを考えると、各ストラテジーとゲームの難易度を紐づけておいた方がよさそうですね。



## 2-5-6 自作デコレータ

各ストラテジーとゲームの難易度を紐づけます。STRATEGIES という辞書を strategies モジュールに定数として記述して使用できる形にします。

*strategies.py の末尾に追記*

```
STRATEGIES = {"Easy (Random)": RandomStrategy, "Normal (Greedy)": GreedyStrategy}
```

確かにこれで、各ストラテジーと難易度の紐づけはできました。しかしこれだと後から新しいストラテジーを思いついたときに不便です。また、strategies.py の末尾に STRATEGIES 辞書があるため、後から違う人が見たときに見落とされる危険などもあります。

理想としては、新しいストラテジーを作成したら自動で辞書に登録してほしいですね。これを実現するのに適しているが自作のデコレータです。デコレータの自作について詳しくは取り扱いませんが、自作デコレータを使用することで下記のように書くことができます。(デコレータの詳細については付録 B を参照)

*strategies.py (完成版)*

```
import random
from .strategy import ComputerPlayerStrategy

# 空の辞書
STRATEGIES = {}
def register_strategy(name):
    def decorator(cls):
        STRATEGIES[name] = cls
        return cls
    return decorator

@register_strategy("Easy (Random)")
class RandomStrategy(ComputerPlayerStrategy):
    """
    ランダムに手を選択する戦略
    """
    def choose_move(self, board, disc_color):
        valid_moves = board.get_valid_moves(disc_color)
```

```

        if not valid_moves:
            return None
        return random.choice(valid_moves)

@register_strategy("Normal (Greedy)")
class GreedyStrategy(ComputerPlayerStrategy):
    """
    最も多くの石を裏返せる手を選択する戦略
    """
    def choose_move(self, board, disc_color):
        valid_moves = board.get_valid_moves(disc_color)
        if not valid_moves:
            return None

        # 各有効な手について、裏返せる石の数を計算
        move_scores = {
            move: board.get_num_flippable_discs(move[0], move[1], disc_color)
            for move in valid_moves
        }

        # 最もスコアの高い手(複数ある場合はその中の一つ)を返す
        max_score = max(move_scores.values())
        best_moves = [move for move, score in move_scores.items() if score == max_score]

        return random.choice(best_moves)

```

自作のデコレータ `@register_strategy("Normal (Greedy)")` は STRATEGIES 辞書に、ゲームの難易度：ストラテジー の形でデータ登録します。

以上でオセロゲームに必要なオブジェクトが大方出そろいました。あとは CUI アプリでも GUI アプリでも Web アプリでもスマホアプリでも、好きなインターフェースでオセロゲームを作るのに使用できます。

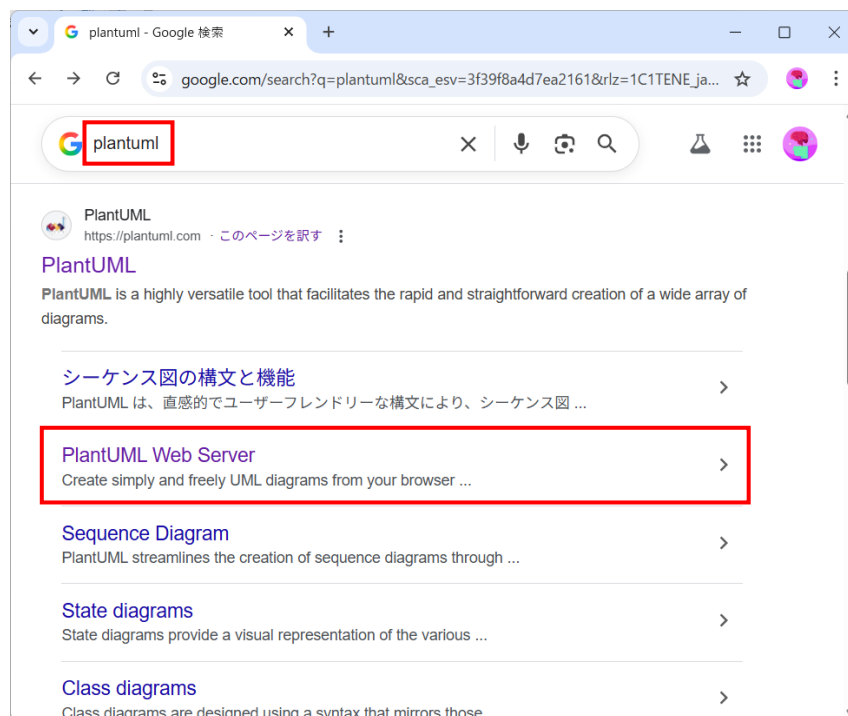
今回作成した othello 関連クラスで CUI アプリケーションと GUI アプリケーションにするための手順書を

### 3 付録

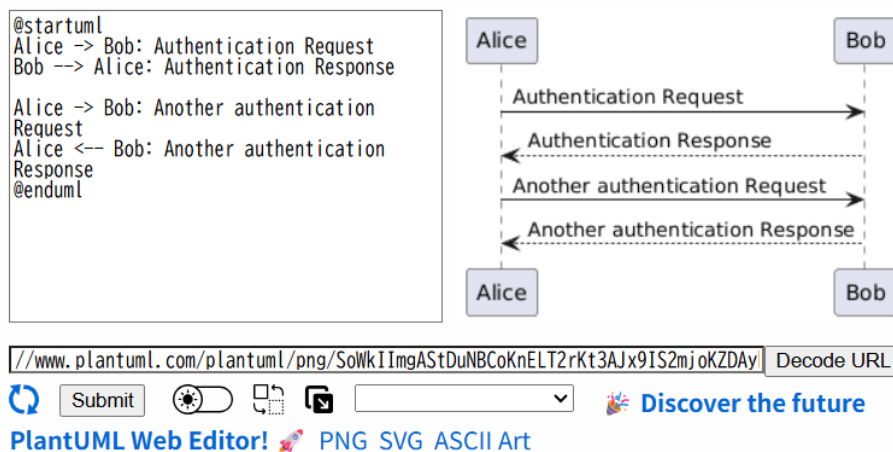
#### 付録 A. PlantUML

クラス図の作成に使えるツールはいくつもありますが、手軽なのはブラウザ上で簡単に作成できる PlantUML です。検索エンジンで「plantuml」と入力して検索すると公式サイトが現れます。

【PlantUML Web Server】をクリックします。



エディタが現れます。(下図では広告の削除のためエディタ部分のみ表示)



エディタ部分に下記を入力すると、Player 関連クラスのクラス図を作成することができます。

```
@startuml
skinparam classAttributeIconSize 0

abstract class Player {
    disc_color : Disc
    {abstract} make_move(board: Board)
}

class HumanPlayer {
    make_move(board: Board)
}

class ComputerPlayer{
    _strategy : ComputerPlayerStrategy
    make_move(board: Board)
}

abstract class ComputerPlayerStrategy{
    {abstract} choose_move(board: Board, disc_color: Disc)
}

class RandomStrategy{
    chose_move(board: Board, disc_color: Disc)
}

class GreedyStrategy{
    chose_move(board: Board, disc_color: Disc)
}

Player <|-- HumanPlayer
Player <|-- ComputerPlayer
```

```
ComputerPlayer *-- ComputerPlayerStrategy
ComputerPlayerStrategy <|-- RandomStrategy
ComputerPlayerStrategy <|-- GreedyStrategy

@enduml
```

## 付録 B. デコレータとその周辺知識

デコレータとは関数を修飾する仕組みのことです。原理的には**高階関数**と**クロージャ**を組み合わせたものであるといえます。それぞれを簡単に説明します。ついでに後悔関数と相性が高い**ラムダ式**についても説明します。

## \* ラムダ式

Python の**ラムダ式**とは**無名関数**のことで、`def ...` を使わないで簡単な関数を定義することができます。書き方は下記のとおりです。

`lambda 引数: 式(戻り値)`

非常に簡単に関数を定義できるため、後述する高階関数と相性が良いです。いくつか例を出しておきます。

関数  $y = 2x + 1$  をラムダ式で表現

```
>>> f = lambda x: 2 * x + 1
>>> f(5)
11
>>> f(-1)
-1
```

手紙の冒頭を作る無名関数の定義

```
>>> begin_letter = lambda name: "拝啓、{}様".format(name)
>>> begin_letter("山本")
'拝啓、山本様 '
>>> begin_letter("Bob")
'拝啓、Bob様 '
```

## \* 高階関数

**高階関数**とは関数を引数に取る関数のことです。高階関数を使うことで非常に柔軟なプログラミングができるようになります。

高階関数の例。第一引数のリストを第二引数の関数で加工する。

```
>>> def processing_list(lists, func):
...     return [func(i) for i in lists]
...
>>> processing_list([1, 2, 3], lambda x:x**2)
[1, 4, 9]
>>> processing_list([1, 10, 100, 1000], lambda x:x//2)
[0, 5, 50, 500]
```

高階関数の例。数学の $\Sigma$ 計算。

```
>>> def sigma(func, i=0, i_max=10):
...     sum = 0
...     for j in range(i, i_max + 1):
...         sum += func(j)
...     return sum
...
>>> sigma(lambda x:x)
55
>>> sigma(lambda x:x**2, i_max = 5)
55
>>> sigma(lambda x:1/x, i = 1, i_max = 100)
5.187377517639621
```

Python は便利な高階関数をいくつか提供してくれています。

Python が提供する map と filter という高階関数。

リスト nums は 0 から 9 までの整数値のリスト。

高階関数 map で第一引数の関数を第二引数のリストの要素に作用させている。

高階関数 filter で第一引数の関数の条件にある第二引数のリストの要素を抽出。

```
>>> nums = [i for i in range(10)]
>>> list(map(lambda x: x*2, nums))
[0, 2, 4, 6, 8, 10, 12, 14, 16, 18]
>>> list(filter(lambda x: x % 3 == 0, nums))
[0, 3, 6, 9]
```

## \* クロージャ

関数に状態を持たせることができるのがクロージャ（関数閉方）というものです。関数の内部で関数を定義して、外側の関数のローカル変数の値を保持することができます。外側関数のローカル変数の値を変更したい場合には `nonlocal` キーワードを使用します。

クロージャ使用例。傾き `a` と切片 `b` を記憶させて任意の 1 次関数を作る。

```
>>> def linear_func(a, b):
...     def func(x):
...         return a * x + b
...     return func
...
>>> f1 = linear_func(2, 1)
>>> f1(2)
5
>>> f1(10)
21
>>> f2 = linear_func(-1, 3)
>>> f2(3)
0
>>> f2(-5)
8
```

クロージャの例。関数 `counter` のローカル変数 `count` の値を `nonlocal` としてクロージャ内でカウントアップする。関数を呼び出した回数を記録できる。

```
>>> def counter():
...     count = 0
...     def increment():
...         nonlocal count
...         count += 1
...         return count
...     return increment
...
>>> c1 = counter()
>>> c2 = counter()
>>> c1()
1
>>> c1()
2
>>> c1()
3
>>> c2()
1
```



## \* デコレータ

以上を踏まえて、最後にデコレータについてです。デコレータは関数の前と後に処理を追加できる機能です。

デコレータを自作する際のひな型は以下の形です。

### 自作デコレータのひな型

```
import functools

def my_decorator(func):
    @functools.wraps(func)  # 元の関数名やdocstringを保持
    def wrapper(*args, **kwargs):
        # 前処理(関数実行前にやりたいこと)
        print(f"{func.__name__} を実行します")

        result = func(*args, **kwargs)  # 元の関数を実行

        # 後処理(関数実行後にやりたいこと)
        print(f"{func.__name__} が終了しました")

    return result
    return wrapper
```

`@functools.wraps(func)` は必須ではありませんが、元の関数名を保持できるので自作のログのためのデコレータや、自作のベンチマークテストでは必須です（例は最後に乗せます）。ひな型を見れば明らかですが、引数が関数になっていて（高階関数）、関数内部に関数を持っています（内部関数、クロージャ）。

上記のひな型で基本的にどんなデコレータも作成することができます。以下で簡単なものから例を見ていきます。エディタは Jupyter Notebook を利用します。

Practice01 フォルダの下に decorator.ipynb というフォルダを作成して、この先のコードを試してみましょう。

引数なしメソッドの前後で "処理開始" と "処理終了" を表示するデコレータ

```
def simple_decorator(func):
    def wrapper():
        print("処理開始")
        func()
        print("処理終了")
    return wrapper

@simple_decorator
def hello():
    print("こんにちは!")

hello()
```

関数の実行時間を計測するベンチマーク用のデコレータ

```
import time
import functools

def benchmark(func):
    @functools.wraps(func) # 元の関数情報を保持
    def wrapper(*args, **kwargs):
        start = time.perf_counter() # 高精度タイマー
        result = func(*args, **kwargs)
        end = time.perf_counter()
        print(f"{func.__name__} 実行時間: {end - start:.6f} 秒")
        return result
    return wrapper

@benchmark
def heavy_task():
    total = 0
    for i in range(1, 10**6):
        total += i
    return total

heavy_task()
```

## ログを取るデコレータ

```
import functools
import datetime

def log_decorator(func):
    @functools.wraps(func) # 元の関数情報を保持
    def wrapper(*args, **kwargs):
        start_time = datetime.datetime.now()
        print(f"[{start_time}] {func.__name__} 開始 args={args},
kwargs={kwargs}")
        result = func(*args, **kwargs)
        end_time = datetime.datetime.now()
        print(f"[{end_time}] {func.__name__} 終了 result={result}")
        return result
    return wrapper

@log_decorator
def add(a, b):
    return a + b

@log_decorator
def greet(name):
    return f"こんにちは {name}"

print(add(3, 5))
print(greet("Alice"))
```

デコレータを使用することで、簡単に関数に処理を追加することができます。何よりデコレータは DRY 原則（Don't Repeat Yourself（繰り返すな）の原則）の実現のためにも非常に有用な技術です。オリジナルのデコレータをいくつか作成してみて使いこなせるようになっておきましょう。